

SLAI T-Rex: Full-Parameter Post-training of the DeepSeek-V4 Family on Ascend SuperPOD

AI Training Platform Team, Shenzhen Loop Area Institute

 Models  Codes

Abstract

Full-parameter post-training of trillion-parameter-scale Mixture-of-Experts (MoE) models introduces substantial system-level challenges for large-scale distributed training, including severe memory pressure, non-overlapped communication overhead, and inefficient kernel execution. While most large-scale LLM training systems are built around GPU-based clusters, this report presents an end-to-end optimization practice on the Ascend NPU SuperPOD. Using the DeepSeek-V4 model family as the target workload, we develop a hierarchical optimization framework spanning model-level parallelism, computation-communication orchestration, and low-level kernel execution. The resulting system achieves 34.22% Model FLOPs Utilization (MFU) with a 2.93 $\times$ improvement over the open-source baseline recipe while maintaining training stability. Building on this optimized infrastructure, we further establish a Continued Pre-Training (CPT) and Supervised Fine-Tuning (SFT) workflow for complex Operations Research (OR) tasks. We refer to the integrated framework as **SLAI T-Rex**, which couples full-stack Ascend SuperPOD optimization with solver-grounded CPT-SFT specialization. OR modeling is selected as the target domain because it jointly requires mathematical formulation, structured reasoning, and code generation. Using DeepSeek-V4-Flash as the specialization platform, we develop OR-oriented CPT and SFT data pipelines that combine collected domain resources with solver-verified synthetic optimization documents. The resulting dataset contains 10K high-quality SFT samples spanning four task categories and three problem representations. The specialized model achieves the highest average zero-shot Pass@1 score among the evaluated models, reaching 71.81% and outperforming GPT-5.4-Mini and the base DeepSeek-V4-Flash model by 3.98 and 11.27 percentage points, respectively. Further evaluation indicates that CPT establishes transferable OR-domain modeling priors, improving both solver-facing feasibility and structural equivalence after SFT. Overall, this work demonstrates a full-stack pathway from efficient trillion-parameter model post-training on Ascend infrastructure to domain-specialized Flash models for solver-grounded mathematical modeling, advancing frontier-model systems for complex reasoning.

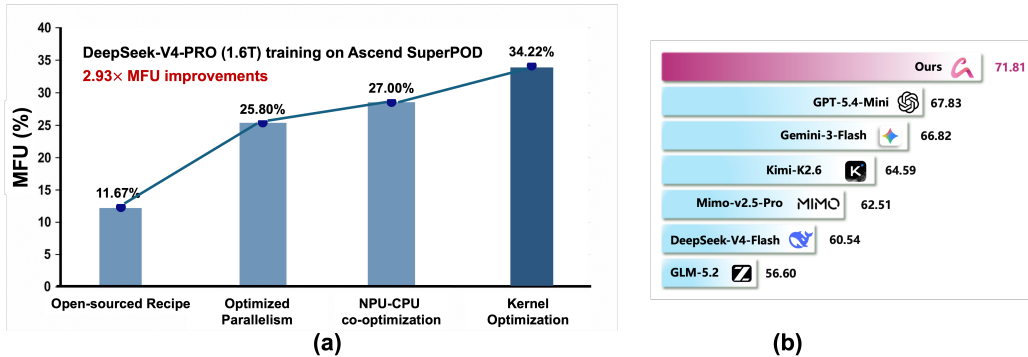


Figure 1: (a) MFU improvements during DeepSeek-V4-Pro (1.6T) training on Ascend SuperPOD. Starting from open-sourced recipes, optimized parallelism, NPU-CPU co-optimization, and kernel optimization increase MFU from 11.67% to 34.22%. (b) Overall accuracy, computed as the mean of NL4OPT (Ramamonjison et al., 2022), OptiBench (Yang et al., 2025c), Bench4Opt (B4O)-Feasible (Wang et al., 2025b), and Bench4Opt (B4O)-ORGEval (Wang et al., 2025b). SLAI T-Rex achieves the highest overall score while using a smaller disclosed activated-parameter budget.

Contents

1	Introduction	4
2	Training Infrastructure on Ascend CloudMatrix384 SuperPOD	5
2.1	Bottlenecks of Training Trillion-parameter LLM on Ascend SuperPOD	6
2.2	Optimizing Trillion-parameter LLM Training System on Ascend SuperPOD	8
2.3	Kernel Optimization for Accelerated LLM Training	11
2.3.1	Bottleneck Kernel Analysis	11
2.3.2	AuraKernel: Performance Optimization Agent for Complex Ascend Kernels	13
2.3.3	Ascend NPU-aware Kernel Fusion and Rewrite	18
3	OR-Oriented Post-Training Workflow	23
3.1	Capability Boundary Analysis of the Original Checkpoint	24
3.1.1	Benchmark-Level Signal	24
3.1.2	Error Spectrum Across Settings	25
3.1.3	Inference-Setting Shift	25
3.1.4	Two-Stage Training Adaptation: Why Both CPT and SFT	27
3.2	OR-Oriented Continued Pre-Training	28
3.2.1	OR-CPT Data Construction and Domain-Dominant Mixture	28
3.2.2	Training Execution in the Ascend Ecosystem	30
3.2.3	Checkpoint Selection and CPT-to-SFT Transfer Validation	30
3.3	Data-Efficient Supervised Fine-Tuning for Operations Research	31
3.3.1	Solver-Oriented SFT Objective	31
3.3.2	Self-Distilled OR Modeling Data Flywheel	32
3.3.3	Contract-Aware Cleaning and CoT Enhancement	33
3.3.4	Progressive SFT with General Capability Anchoring	34
3.4	AI-Assisted Data Construction and Refinement	34
4	Experimental Results	36
4.1	Efficient Kernels via AuraKernel	36
4.2	Performance Improvements from Kernel Fusion	38
4.3	Long-term Training Stability with Ascend SuperPOD	41
4.4	SFT Scaling and Cleaning Analysis	41
4.5	Effect of CPT Initialization under Matched SFT Conditions	43
4.6	Performance of the Proposed Post-Training Pipeline	44
5	Conclusion, Limitations, and Future Directions	46
	References	47
	Appendix	59

A Author List	59
B Post-training Recipe Details	59
C Solver-Verified OR-CPT Data Synthesis: Engine Design and Illustrative Cases	59
C.1 Role in the post-training workflow	60
C.2 End-to-end engine architecture	60
C.3 Mathematical seeds and domain contracts	61
C.4 Prompt-controlled bidirectional synthesis	62
C.5 Controlled scenarios and CPT rendering	62
C.6 Illustrative case I: capacitated facility location	63
C.7 Illustrative case II: executable but structurally wrong models	64
D CPT-SFT-Deployment-Evaluation Provenance	64
D.1 Design principles	64
D.2 Provenance chain and registry layout	65
D.3 Stage-level linkage mechanism	65
D.4 Confidence levels	65
D.5 Dashboard monitoring and retroactive registration	65
E Training Schedule and Monitoring Design	66
E.1 Static Curriculum Learning Design	66
E.2 Monitoring Dashboard	67
F Representative Original-Checkpoint Error Cases	68

1 Introduction

Post-training has become a standard stage for enhancing LLMs after pretraining (Lai et al., 2025; Cheng et al., 2025), improving instruction-following, reasoning power, alignment with human preferences, and the power of agentic tasks with long-horizon executions. In particular, recent efforts on LLM post-training mainly focus on Continued Pre-Training (CPT), Supervised Fine-Tuning (SFT) (Mecklenburg et al., 2024) and reinforcement learning (Schulman et al., 2017; Shao et al., 2024; Li et al., 2026b), together with dedicated agentic environments (GLM-5-Team et al., 2026; Dong et al., 2026) and data pipelines (NVIDIA et al., 2026) for general or domain-specific tasks.

As LLMs evolve from simple chatbots into agentic workforces, the growing demand for long-context agentic reasoning continues to drive model scaling in pursuit of stronger intelligence. Recent frontier LLMs (DeepSeek-AI et al., 2025; GLM-5-Team et al., 2026) have pushed model sizes toward the trillion-parameter scale while adopting the hybrid attention mechanism (Yang et al., 2025b; Dao & Gu, 2024) or intricate residual connection design (Xie et al., 2025) to improve the efficiency of long-context computation and related optimizations (Chen et al., 2026c; Li et al., 2026a). In the meantime, Mixture-of-Experts (MoE) (Fedus et al., 2022) architecture introduces a different route for scaling by routing each token to a selected subset of expert networks rather than activating all parameters, per-token computation grows sub-linearly with model size, enhancing reasoning power while preserving memory efficiency.

Most prior studies on large-scale MoE post-training remain tightly coupled with CUDA- or TPU-oriented training infrastructure (DeepSeek-AI et al., 2025; Kimi Team et al., 2026; GLM-5-Team et al., 2026), which are built around either Single-Instruction, Multiple-Thread (SIMT) execution or systolic-array-based accelerator architectures. In contrast, post-training trillion-parameter-class LLMs on Single-Instruction, Multiple-Data (SIMD) hardware remains comparatively underexplored. Motivated by this research gap, this report first presents a system-level infrastructure optimization for trillion-parameter-class LLM post-training based on Ascend CloudMatrix384 SuperPOD with Ascend 910C NPUs. As one of the recent state-of-the-art LLMs for agentic tasks and reasoning, DeepSeek-V4 (DeepSeek-AI et al., 2026) reaches top-tier intelligence via intricate sparse attention and residual connection mechanisms. Full-parameter post-training DeepSeek-V4-Pro model on SuperPOD cluster introduces a host of challenges, spanning AscendC kernel design, communication orchestration, memory management, and multi-dimensional parallelism, as illustrated in Figure 1.

Beyond the challenges of training infrastructure, recent LLM post-training efforts have increasingly concentrated on verifiable reasoning domains, particularly mathematical reasoning, coding, and agentic software-engineering tasks. However, many long-tailed industrial decision-making domains remain comparatively under-represented. Operations Research (OR), for example, is central to real-world optimization problems such as scheduling, transportation, supply-chain management, and resource allocation, yet its integration with LLM training and evaluation is still at an early stage. Recent studies on LLM-based OR mainly focus on automatic model formulation and solver-oriented program synthesis, where LLMs translate natural-language problem descriptions into mathematical formulations or executable optimization code. Representative works (Xiao et al., 2024; Zhang & Luo, 2025) further introduce domain-knowledge retrieval, multi-agent collaboration, and execution-feedback-based refinement into the OR modeling pipeline. However, these approaches remain limited by small-scale benchmarks, brittle semantic-to-structure mapping, and insufficient guarantees of formulation correctness in complex industrial OR scenarios. Furthermore, existing LLM-based OR studies primarily emphasize agentic-system design, while domain-specific training for OR remains under-explored. This gap becomes more pronounced for trillion-parameter-class models (e.g., DeepSeek-V4-Pro), where OR-oriented post-training requires not only domain knowledge acquisition via SFT or CPT, but also a dedicated data preparation pipeline, and verifiable feedback from optimization environments.

In this work, we present full-stack end-to-end training for the DeepSeek-V4 model family based on Ascend SuperPOD with 910C NPUs (Zuo et al., 2025). We refer to the overall framework as **SLAI T-Rex**, which couples full-stack Ascend SuperPOD training optimization with solver-grounded CPT-SFT specialization for OR. To improve system-level training efficiency, this work first presents a full-stack optimization for SuperPOD-based training infrastructure with both kernel-level optimization and top-level training pipeline orchestration. In particular, we propose **AuraKernel**, a specialized AscendC kernel optimization agent which automatically optimizes the dominant bottleneck kernels of the DeepSeek-V4 model family. By combining the hierarchical optimization

strategies, we achieve a $2.93\times$ Model FLOPs Utilization (MFU) improvement with the DeepSeek-V4-Pro model. Exploiting the optimized system-level training stack on Ascend SuperPOD, we further validate an OR-oriented post-training workflow on DeepSeek-V4-Flash, covering solver-grounded data construction, SFT data cleaning, CPT-to-SFT transfer, and benchmark evaluation. In the CPT-to-SFT transfer evaluation under identical SFT conditions, direct SFT improves B4O-Feasible from 60.47% to 65.93% and B4O-ORGEval from 34.26% to 48.73%, while CPT-initialized SFT further raises them to 71.22% and 59.39%, respectively. Aggregating over multiple OR evaluation metrics, our model attains an average accuracy of 71.81%, outperforming GPT-5.4-Mini by 3.98 points and the base DeepSeek-V4-Flash by 11.27 points. These results show that SFT provides supervised task alignment, while CPT contributes OR-domain modeling priors that improve both solver-facing feasibility and structural equivalence, with the largest gain on B4O-ORGEval dataset. Together, the Pro and Flash post-training validate two complementary benefits: infrastructure scalability and domain adaptation. To the best of our knowledge, this is among the first publicly documented full-stack engineering studies of full-parameter post-training for a trillion-parameter-class MoE model on Ascend SuperPOD and it demonstrate the effectiveness of SIMD-based NPUs for large-scale LLM training.

Overall, the contributions of this work are:

- **Ascend-native system optimization for training complex trillion-parameter MoE LLMs:** Based on Ascend CloudMatrix384 SuperPOD, we present a full-stack optimization framework for trillion-parameter-scale DeepSeek-V4-Pro training, achieving 34.22% MFU with $2.93\times$ improvement over the default baseline. In addition to the training acceleration, this work characterizes the key principles of Ascend-friendly system-level optimization with jointly optimized parallelism, communication orchestration, memory movement, and kernel execution under the architectural constraints of Ascend SuperPOD.
- **Hardware-aware analysis of MoE, sparse attention, and memory hierarchy on Ascend:** Through detailed profiling of DeepSeek-V4 training on Ascend, we provide an end-to-end analysis of how model architecture interacts with Ascend kernel execution, communication patterns, memory hierarchy, and distributed orchestration. The resulting insights bridge model-level design choices with hardware-level computation efficiency, and comprehensive profiling and bottleneck analysis further motivate the proposed optimization strategies and future research.
- **Solver-guided AscendC kernel optimization beyond manual and heuristic optimization:** We present one of the first attempts to apply Operations Research to Ascend kernel optimization. In contrast to prior kernel-agent approaches that mainly rely on harness construction and loop-level engineering, the proposed AuraKernel introduces OR-guided tiling optimization into the agentic kernel optimization workflow. The AscendC OR solver and dedicated harness design enable accurate tiling selection, long-horizon kernel optimization, and more systematic exploration of the Ascend kernel search space.
- **SuperPOD-aware orchestration for complex MoE communication and parallelism:** We present an end-to-end optimization recipe for Ascend SuperPOD with multi-dimensional parallelism and optimized training orchestration for large-scale MoE architectures.
- **Reproducible post-training workflow for domain-specific enhancement:** We construct an OR-oriented CPT-SFT workflow on DeepSeek-V4-Flash, covering solver-grounded CPT data construction, self-distilled SFT data generation, contract-aware cleaning, and matched benchmark evaluation.
- **Empirical evidence for CPT-SFT synergy in structured OR reasoning:** We show that SFT provides supervised task alignment and improves the original checkpoint, while CPT supplies additional OR-domain modeling priors that further improve both solver-facing feasibility and structural equivalence, with the largest additional gain observed on B4O-ORGEval dataset.

2 Training Infrastructure on Ascend CloudMatrix384 SuperPOD

Mixture-of-Experts (MoE) and hybrid attention architectures improve the efficiency–intelligence trade-off of large language models by decoupling total model capacity from per-token computation

and reducing the cost of long-context processing (Krajewski et al., 2024; DeepSeek-AI et al., 2026). These architectural benefits, however, introduce substantially more complex training workloads. Expert routing requires frequent token dispatch and All-to-All (A2A) communication, while sparse attention produces irregular computation patterns and heterogeneous kernel behavior. Efficient training therefore requires coordinated optimization across parallelism, communication scheduling, memory management, runtime orchestration, and kernel execution.

Existing large-scale LLM training systems primarily optimize multi-dimensional parallelism and computation-communication overlap on GPU- or TPU-based clusters (Yan et al., 2026). By comparison, full-parameter training of trillion-parameter-class MoE models on SIMD-oriented accelerator clusters remains less explored. This section presents an end-to-end study of such training on the Ascend CloudMatrix384 SuperPOD (Zuo et al., 2025), with Ascend-native optimizations spanning both the distributed training system and architecture-specific kernels.

We use DeepSeek-V4-Pro as the target workload, which has 1.6 trillion parameters and combines large-scale MoE layers with hybrid sparse attention (DeepSeek-AI et al., 2026). Its architecture stresses three major dimensions of the training infrastructure: ① parameter and optimizer-state management at trillion-parameter scale, ② fine-grained expert routing with intensive A2A communication, and ③ irregular sparse-attention computation requiring specialized kernel optimization. These characteristics expose the principal bottlenecks of large-scale MoE training, including memory pressure, expert-parallel scalability, token-dispatch efficiency, communication-computation overlap, and kernel performance.

Section 2.1 first characterizes the end-to-end training bottlenecks on Ascend SuperPOD. Section 2.2 then presents the proposed parallelism, communication, memory, and runtime-orchestration optimizations. Finally, Section 2.3 introduces the kernel-level optimization workflow, including AuraKernel, our operations-research-guided agent for long-horizon AscendC performance tuning.

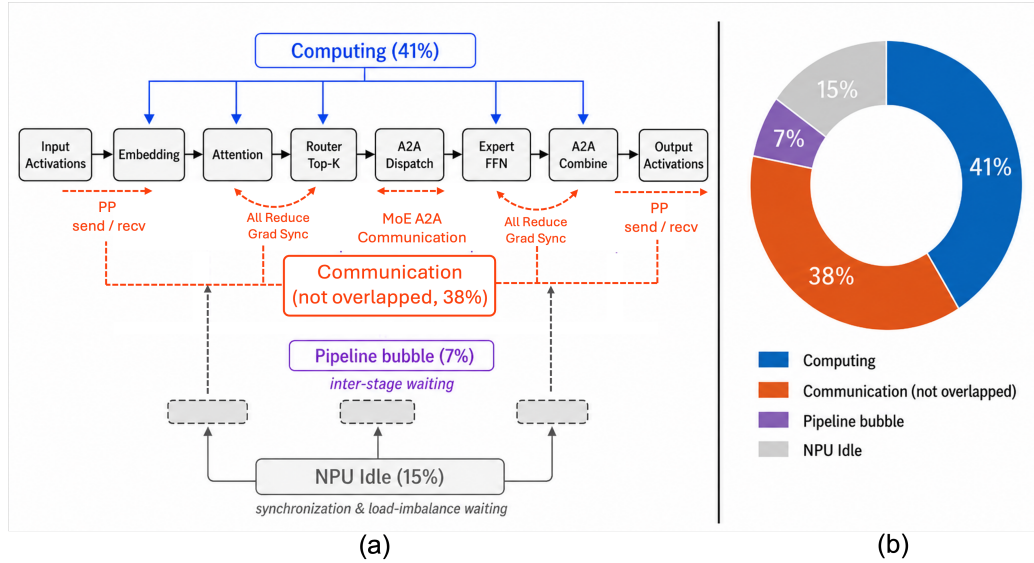


Figure 2: Latency breakdown of training trillion-parameter-class LLM model on Ascend SuperPOD NPU cluster. (a) Partition of latency in the end-to-end LLM training pipeline. (b) Averaged end-to-end latency breakdown, including computation, communication, and NPU idle bubble.

2.1 Bottlenecks of Training Trillion-parameter LLM on Ascend SuperPOD

Latency overhead induced by the non-overlapped communication. Given the scale of the target 1.6T-parameter model, we first adopt the baseline multi-dimensional parallelism to ensure training stability while keeping model states, activations, and optimizer states within the device memory budget, as summarized in Figure 2. However, training MoE models under such a multi-dimensional parallelism strategy inevitably introduces latency overhead across different partitions, due to fre-

Table 1: Communication latency breakdown by parallelism dimension in DeepSeek-V4-Pro training on Ascend SuperPOD, aggregated over the profiled ranks. Communication accounts for 52.2% (40.76 s) of the total device kernel time. Each collective/point-to-point primitive is attributed to the parallelism dimension that induces it: expert parallelism (AllToAllV token dispatch/combine), pipeline parallelism (point-to-point Send/Receive of inter-stage activations and gradients), and tensor/data parallelism (AllReduce, AllGather, and ReduceScatter for intra-layer reductions and sharded parameter/gradient exchange).

Parallelism Dimension	Communication Primitives	Latency (s)	Share of Comm. (%)	Share of Total (%)
Tensor / Data (TP+DP)	AllReduce, AllGather, ReduceScatter	19.71	48.4	25.2
Pipeline (PP)	Send, Receive (P2P)	14.95	36.7	19.2
Expert (EP)	AllToAllV (MoE dispatch/combine)	5.91	14.5	7.6
Misc.	Broadcast, others	0.18	0.4	0.2

quent synchronization, collective communication, and data redistribution among parallel groups, as summarized in Table 1.

The profiling results clearly indicate that the primary latency bottleneck is collecting partial results from the TP and DP partitions. Meanwhile, the send and receive latency introduced by the traditional 1F1B pipeline parallelism (PP) (Narayanan et al., 2021) contributes a large portion toward the overall latency per training step. Benefiting from the high-speed interconnects of the Ascend SuperPOD, the communication overhead introduced by MoE remains minimal.

Based on this analysis, the latency of one training step can be decomposed into four major components:

$$T_{\text{step}} = T_{\text{comp}} + T_{\text{comm}}^{\text{non-overlapped}} + T_{\text{bubble}} + T_{\text{idle}},$$

where T_{comp} corresponds to useful dense and sparse model computation, $T_{\text{comm}}^{\text{exposed}}$ denotes communication that is not fully hidden by computation, T_{bubble} captures inter-stage waiting caused by pipeline parallelism, and T_{idle} represents device-side stalls caused by synchronization, load imbalance, and unresolved runtime dependencies.

In addition to the absolute latency introduced by the parallelism-induced communication (hcom_kernels), the imbalanced computation workload between different PP stages further introduces a latency bubble throughout the training process. As depicted in Figure 2(b), the computation bubble exists uniformly across different stages of pipeline parallelism.

Dominant latency from model-specific kernels. Taking a closer look at the detailed kernel execution time, we observe that the latency of computation is dominated by a small set of architecture-specific kernels. In particular, DeepSeek-V4 introduces the hierarchical sparse attention with heavy execution time on Ascend NPUs. As depicted in Table 2, the backward kernel of sparse attention constitutes one of the most significant bottlenecks among all AscendC operators, where the total execution time is even comparable to the generic matrix multiplication operators. Throughout this analysis, an operator’s Task-Duration share is its summed core-busy time—the per-kernel AI-Core or vector-core execution time—normalized over the 206,503 compute kernels of one step after excluding communication and AI-CPU entries, with invocation count and average per-call duration listed alongside to expose the fragmented, long-tailed operators.

Concretely, the sparse-attention family contributes roughly a quarter of the non-communication kernel time, exceeding the combined share of the dense MatMulV3 and MoE GroupedMatmul operators. This concentration is intrinsic to the model architecture rather than an artifact of a particular layer configuration: the lightning indexer that selects the sparse key-value context and the shared-KV attention that consumes it are invoked in every attention block (DeepSeek-AI et al., 2026), so their cost scales directly with the depth of the network.

The profiling results also make it clear why these kernels are slow, and their bottlenecks differ fundamentally from those of matrix-multiplication operators. The dense MatMulV3 and MoE GroupedMatmul kernels are compute-bound and run on the CUBE core near saturation (over 90% utilization), converting their latency into useful matrix throughput. Among these, GroupedMatmul

is genuinely MAC-dominated, whereas MatMulV3 is operand-load-bound, its MTE2 load share (96.3%) far exceeding its MAC share (52.5%). The sparse-attention kernels do the opposite: their compute engines sit largely idle (vector-compute pipeline ratio of about 9.5%), and their runtime is instead spent moving key-value tiles through the local buffers and executing the scalar index and gather/scatter logic needed to assemble the sparse attention pattern. In other words, these operators are memory- and control-bound rather than compute-bound—they occupy the top of the latency profile not because they perform heavy computation, but because their data-dependent, irregular access pattern keeps the hardware’s matrix engine starved.

Motivated by these findings, this work proposes **AuraKernel**, an end-to-end AscendC kernel optimization agent that automatically restructures the large, memory- and control-bound AscendC kernels, as presented in Section 2.3.2.

Table 2: Representative top operators in DeepSeek-V4-Pro training on Ascend 910C at global batch size 1024, ranked by Task-Duration share. Cube rows report AIC counters (MAC, MTE2, Fixpipe store, cube utilization) and vector rows AIV counters (vector compute, MTE2, MTE3, scalar utilization); the store column is therefore Fixpipe for cube rows and MTE3 for vector rows. The final row aggregates the remaining 92 operator types. Share definition and profiling scope are given in the text.

Rank	Operator	Task Dur. Share	Calls	Avg (μ s)	Compute (MAC/Vec)	MTE2 Load	Store (MTE3/Fix)	Cube/Scalar Util.
1	SparseAttnSharedkvGrad (sparse-attn bwd)	16.86%	224	27,593	9.5%	21.3%	34.2%	15.1%
2	MatMulV3	12.77%	5,600	836	52.5%	96.3%	21.8%	85.4%
3	GroupedMatmul (MoE experts)	9.50%	1,344	2,592	86.8%	96.2%	11.4%	95.4%
4	Cast	6.02%	40,063	55	5.7%	89.7%	59.9%	1.7%
5	SparseLightningIndexerGradKLLoss	5.52%	96	21,082	5.5%	7.9%	5.4%	96.0%
6	Add	5.40%	16,579	119	14.0%	93.4%	23.0%	2.3%
7	GmmAddKernel (MoE weight-grad)	4.72%	448	3,859	58.8%	89.4%	24.0%	95.8%
8	PpMatmulAccumAtomicKernel	3.26%	896	1,335	63.5%	95.6%	9.2%	95.1%
9	BatchMatMulV2	3.06%	896	1,253	64.1%	98.7%	12.1%	94.3%
Long tail (92 remaining operator types)		32.88%	140,357	≈ 86	predominantly small, memory-bound vector & copy kernels			

The long tail of fragmented, memory-bound kernels. In addition to the top-ranked bottleneck kernels with insufficient utilization on Ascend NPUs, we observe a long tail of small kernel invocations that consistently appear in each training step, as shown in Table 2. While each individual kernel contributes only a short execution time, these fragmented kernel calls collectively account for a substantial fraction of the overall computation time.

A closer inspection shows that these long-tailed kernels are primarily element-wise and data-movement operators, such as Cast and Add, together with tensor copy and reshape operations. These operators are invoked frequently, yet each typically completes within only tens of microseconds. Unlike cube-bound matrix-multiplication kernels, these fragmented operators execute almost entirely on the vector core, where the load/store pipelines are significantly more active than the vector-compute pipeline. As shown in Table 2, the Cast operator alone contributes 5.7% of the total computation time. However, the vector-compute utilization of Cast remains nearly idle (computing core utilization $\approx 1.7\%$), whereas the memory-transfer pipelines remain heavily occupied, with AIV MTE2/MTE3 at 89.7%/59.9%. This imbalance indicates that such operators are primarily memory-bound and are natural candidates for fusion with neighboring kernels.

Overall, kernel fragmentation further degrades system-level efficiency due to the newly proposed model architecture (e.g., mHC (Xie et al., 2025) hyper-connections) in the DeepSeek-V4 model. These kernels have very short execution times, where the fixed costs of kernel launch and scheduling can no longer be effectively amortized. Furthermore, repeated round trips to memory introduce redundant data movement and inflate the aggregate overhead. As a result, the collective footprint of these fragmented, memory-bound operators is far from negligible, motivating a consolidation strategy based on dedicated kernel fusion to suppress launch overhead and eliminate redundant memory traffic, as presented in Section 2.3.3.

2.2 Optimizing Trillion-parameter LLM Training System on Ascend SuperPOD

Motivated by the bottleneck analysis in Section 2.1, we optimize end-to-end training performance through a hierarchical three-layer strategy: ① Optimal computation-communication orchestration

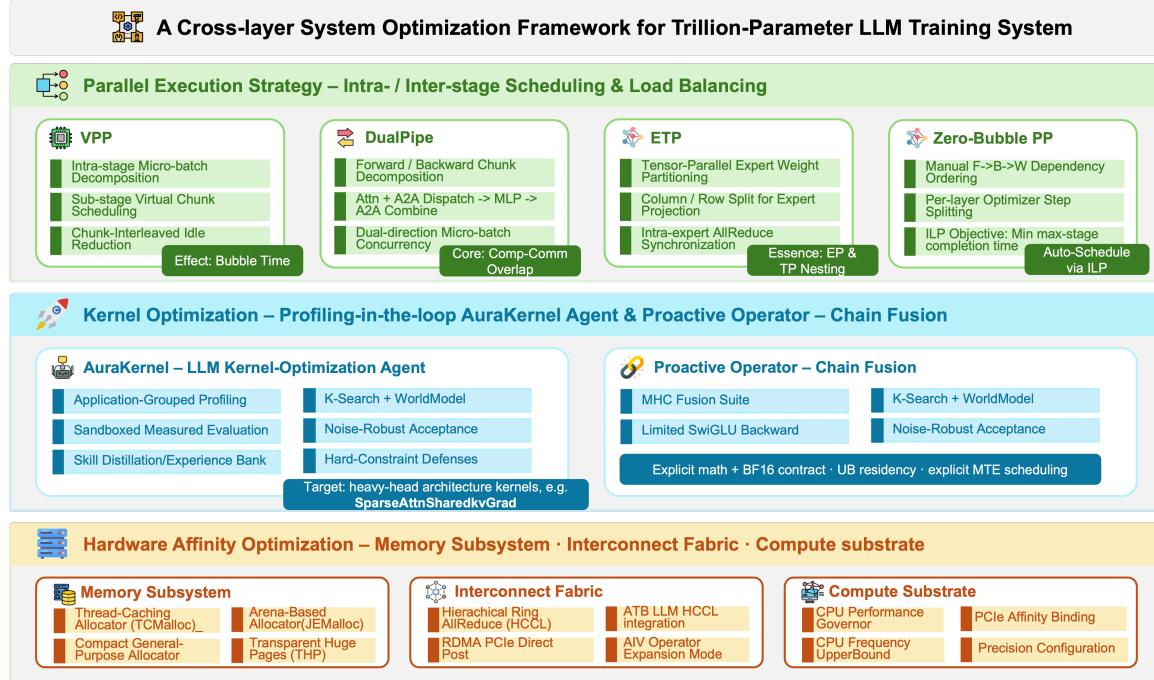


Figure 3: Overview of the proposed full-stack optimization framework, which includes three cooperating layers: parallel execution strategy, kernel optimization, and hardware-affinity tuning.

via optimized training parallelism, ② Agent-driven optimization of top-ranked bottleneck kernels with AuraKernel, and ③ Full-stack operator fusion to eliminate fragmented kernel overhead. The overall framework is illustrated in Figure 3.

Load-balanced pipeline scheduling and expert-tensor parallelism (ETP). As discussed in Section 2.1, reducing the exposed communication overhead introduced by tensor parallelism (TP) is a primary optimization target. Due to the limited memory capacity of each Ascend 910C NPU (64GB), a naive parallelization strategy requires a relatively high TP degree, such as TP=4, to fit the model states and activations within the device memory budget. However, this also increases the frequency and cost of TP collectives.

Following the Parallel Folding strategy in Megatron-LM (Shoeybi et al., 2019), we enable expert-tensor parallelism (ETP) for MoE layers while keeping the pipeline parallelism (PP) configuration fixed. This decouples the parallelism choices between attention and MoE modules: attention layers can adopt a lower TP degree (TP=2) to reduce communication overhead and improve GEMM efficiency on large dense matrices, while MoE layers use ETP=1 to preserve the full expert width and avoid further fragmenting expert computation. The resulting dispatch and execution strategy is illustrated in Figure 4.

Virtual Pipeline Parallelism (VPP) vs. DualPipeV. For the pipeline dimension (with a pipeline degree of 8), we adopt VPP (Narayanan et al., 2021) rather than the bidirectional DualPipeV (Qi et al., 2025; DeepSeek-AI et al., 2025) schedule, for two complementary reasons grounded in our profiling. First, DualPipeV (Qi et al., 2025; DeepSeek-AI et al., 2025) is memory-prohibitive at this scale: to sustain its bidirectional overlap, each device must retain two model chunks, roughly doubling the resident parameters and gradients, together with a larger population of in-flight micro-batch activations. Our traces show that, even under the more memory-frugal VPP schedule, the peak reserved device memory already reaches 51.7–54.6 GB per NPU across the profiled ranks, leaving only a narrow margin from the per-device HBM budget; the additional residency demanded by DualPipeV would readily overshoot this limit, forcing a smaller micro-batch size or heavier recomputation that erodes its purported efficiency gains. Second, and more fundamentally, DualPipeV’s headline capability, overlapping the expert-parallel all-to-all (MoE dispatch/combine)

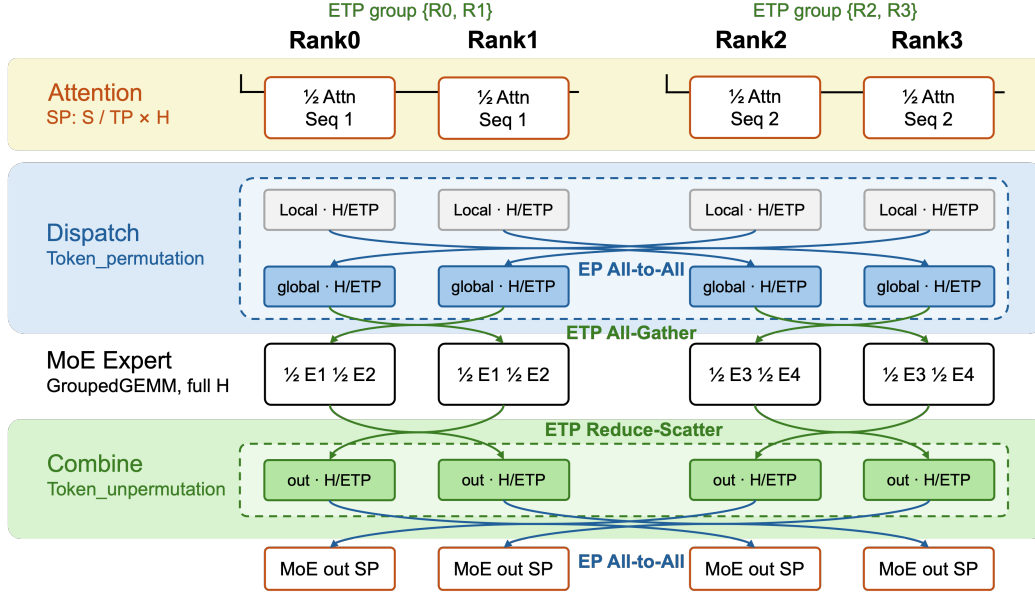


Figure 4: Communication pattern of the ETP token dispatcher (ETP2-EP2 example). Dispatch and combine wrap the expert grouped-GEMM with an EP all-to-all (token routing) and an ETP all-gather/reduce-scatter over the hidden dimension (Gale et al., 2023). With MoE Parallel Folding, ETP folds into EP as a single group, so one all-to-all performs dispatch and the separate ETP collectives are eliminated.

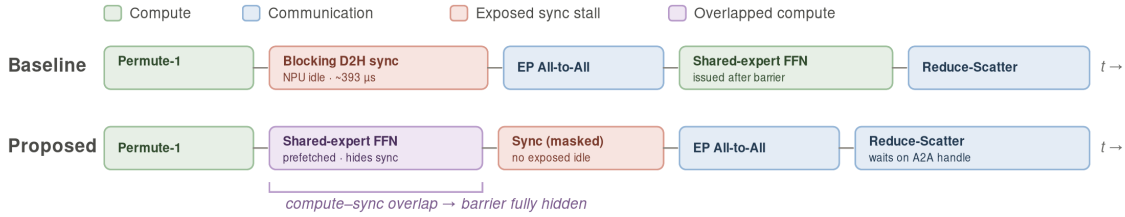


Figure 5: Compute-communication schedule of the MoE token dispatcher. Issuing the shared-expert forward ahead of the before_ep_alltoall synchronization fills the exposed sync stall with useful computation while communication overlap is preserved.

with computation, targets a cost that is simply not our bottleneck. In our profiling, the all-to-all accounts for only 14.5% of communication time, whereas the communication cost is dominated by pipeline point-to-point transfers (36.7% of communication, driven largely by receive-side stalls that reflect stage-level bubbles and load imbalance) and by tensor-/data-parallel AllReduce (48.4% of communication).

The pipeline-bubble is precisely what VPP mitigates by introducing fine-grained interleaving of virtual stages. Consequently, DualPipeV would incur a substantial memory penalty in exchange for accelerating a component that contributes marginally to our end-to-end latency, whereas VPP suppresses the dominant pipeline bubbles while keeping the per-device footprint within the hardware budget and training stability. We will further investigate the optimal pipeline parallelism strategy in the near future.

Overlapped synchronization and computation for MoE token dispatch. Expert parallelism turns routing into an all-to-all (A2A) communication in distributed LLM training. Our dispatcher structures one MoE layer as a pipeline (permutation, an A2A dispatch, grouped-GEMM expert computation, an A2A combine, and unpermutation), and applies three Ascend-centric choices to keep the collectives off the critical path.

For the MoE module, both token dispatch and combine are implemented using asynchronous all-to-all (A2A) collectives. Synchronization is deferred until the communicated results are consumed, allowing independent computation to execute concurrently on separate streams and thereby reducing the exposed communication latency. Specifically for Ascend-based MoE training, we further hide the synchronization stall preceding the dispatch operation, as illustrated in Figure 5. Because the per-expert A2A split sizes cannot be determined until token routing is completed, resolving these dynamic split sizes introduces a visible `before_ep_alltoall` synchronization stall on the device timeline. The shared expert, however, depends only on the MoE-layer input and is independent of the routing-derived split sizes. We therefore advance its forward computation to overlap with this synchronization interval. This scheduling strategy fills an otherwise idle period with useful Cube computation while preserving the subsequent computation-communication overlap, effectively converting the barrier-induced latency identified in Section 2.1 into productive execution.

Double-buffered Swap Optimizer. At the trillion-parameter scale, optimizer states become a dominant source of device-memory consumption. Distributed AdamW (Loshchilov & Hutter, 2017) maintains an FP32 master copy of the parameters together with the first- and second-moment states, amplifying the overall HBM usage compared to the model parameters alone. On Ascend NPU, enabling swap optimizer alleviates the memory pressure.

A conventional swap optimizer mitigates this pressure by offloading optimizer states (Ren et al., 2021), together with the FP32 master parameters when configured, to pinned host memory and reloading them only when needed for parameter updates. Although this approach reduces the HBM footprint, it introduces serialized host-to-device and device-to-host transfers around each optimizer update. These transfers extend the optimizer step and can offset the performance benefits gained from the reclaimed memory.

To reduce this overhead, we implement a double-buffered swap optimizer that overlaps state transfers with optimizer computation. The swapped states are partitioned into chunks of $\text{swap_numel} = N_{\text{swap}} / \text{swap_optimizer_times}$ elements, where N_{swap} denotes the total number of swapped parameters. While a fused AdamW kernel updates the current chunk, the next chunk is prefetched from host memory and the previously updated chunk is asynchronously written back through dedicated swap-in and swap-out streams. This pipelined schedule removes most transfer latency from the critical path. The chunk size controls the trade-off among the temporary HBM working set, transfer granularity, and achievable overlap. As a result, the optimizer substantially reduces HBM consumption with limited runtime overhead. The reclaimed memory can support more activations and larger micro-batches, thereby reducing activation recomputation and pipeline bubbles and increasing the useful-computation share T_{comp} .

2.3 Kernel Optimization for Accelerated LLM Training

2.3.1 Bottleneck Kernel Analysis

Following the training step-level decomposition in Section 2.1, we further investigate the device-compute bottlenecks at the kernel level. The profiling results reveal two distinct sources of inefficiency. ① The architecture-intrinsic kernels, including sparse attention, dense matrix multiplications, and grouped expert GEMMs, account for substantial device time within individual invocations. ② Model-specific stages remain implemented as PyTorch-level compositions and are lowered by the `eager` mode into long sequences of casts, views, tensor-movement, and reductions, resulting in excessive kernel launches and fragmented execution. These observations motivate two complementary optimization directions: kernel-internal tuning for dominant compute-intensive kernels, and fusion or re-expression for launch-fragmented operator chains.

We organize the profiling results at the granularity at which optimization decisions can be made. Task Duration identifies individual kernels that dominate the device timeline and are therefore candidates for kernel-internal optimization. Kernel launch counts and submodule annotations reveal how high-level model stages are decomposed into low-level runtime operations. For AIV-dominated kernels, the utilization ratios of the vector-compute, scalar, MTE2 load, and MTE3 store pipelines further indicate whether execution is constrained by arithmetic, scalar control, or data movement. Taken together, these measurements distinguish compute-intensive standalone kernels

from launch-fragmented operator chains and from memory-traffic bottlenecks introduced by the frontend implementation.

On the profiled NPUs, the trace contains 206,503 compute-kernel launches and 39.84 s of aggregate device-side compute duration after excluding communication and AI-CPU execution. The resulting kernel profile exhibits four recurring patterns. The first is a small heavy head of architecture-intrinsic kernels whose latency is dominated by computation within the kernel body. The second is a launch-fragmented tail produced by PyTorch eager computation, where overhead accumulates across a large number of short-lived kernels. The third is a memory-bound vector tail, in which execution time is dominated by data movement rather than arithmetic. The fourth consists of generic Triton-generated kernels whose functional computation is correct, but whose lowering does not fully exploit on-chip data residency or pipeline scheduling. The first category primarily requires kernel-internal optimization, whereas the remaining categories motivate operator fusion or re-expression as AscendC kernels that retain intermediate tensors in Unified Buffer whenever possible. Table 2 summarizes representative operators and their vector-pipeline characteristics. We first examine the dominant kernels and then turn to the fragmented and memory-bound long tail.

We begin with the dominant kernels, a small set of architecture-specific operations that account for a substantial fraction of device-compute time. In particular, `SparseAttnSharedkvGrad` is the largest individual contributor, accounting for 16.86% of the total kernel computation time due to the expensive gradient computation with respect to the DeepSeek sparse attention. Unlike ordinary operators (e.g., Matrix Multiplication) which are thoroughly optimized, architecture-specific kernels exhibit high code-level complexity in hierarchical memory layout and computation pipelines. This work addresses the challenge of kernel complexity via the proposed **AuraKernel** agent, as elaborated in Section 2.3.2.

The same trace also exposes a second execution regime. Rotary position embedding, weight-free RMS normalization, limited SwiGLU, and the mHC projection path are implemented as torch-level compositions, and autograd expands them into long sequences of generic operators. In the profiled step, non-matmul eager operators account for 91.3% of launches but only 66.7% of the Task Duration, against 33.3% for the far fewer matmul-like operators (Figure 6b). A dense small-kernel tail gives the regime its shape: 58.0% of launches finish in under 20 μ s yet together contribute only 2.2% of Task Duration. The performance cost is therefore caused by repetition and by the boundaries between kernels: tensor materialization, dispatch, scheduling, and memory traffic are paid at every link in the chain.

The vector-pipeline counters show that the long tail is primarily constrained by data movement rather than arithmetic. Vector-core kernels account for 36.5% of the total Task Duration, but their vector-compute utilization is only about 15%, substantially lower than the corresponding MTE load and store ratios of approximately 68% and 40%, respectively (Figure 6c). The imbalance indicates that most vector-kernel execution time is consumed by loading, storing, and rearranging data instead of actual computation.

The distribution of operators (frequency and latency) further confirms that the long tail is dominated by fragmented data-movement operations. `Cast` is invoked 40,063 times and contributes 6.02% of the total Task Duration, while operators such as `ViewCopy`, `TensorMove`, `ZerosLike`, and `Slice` introduce additional overhead through frequent tensor materialization and buffer movement. These operations commonly arise when high-level backward computations, such as rotary embedding and limited SwiGLU, are decomposed into multiple small kernels under eager execution.

The bottleneck analysis motivates two separate kernel-optimization paths. Architecture-intrinsic kernels are optimized within the operator through improved tiling, data layout, scheduling, and pipeline overlap, which can be performed via agentic pipeline with long-context understanding (Section 2.3.2). On the other hand, the fragmented long tail is addressed by fusing and rewriting each target stage as a hardware-aware AscendC kernel. For eager autograd chains, this requires recovering the high-level computation and eliminating unnecessary data movement and type conversion introduced by framework execution. For existing Triton kernels, the computation is reimplemented in AscendC with explicit control over Unified Buffer (UB) residency, tiling, and MTE scheduling. Both approaches reduce global-memory traffic by retaining intermediate results on-chip and consolidating the computation into fewer kernel passes. Section 2.3.3 describes this fusion-and-rewrite path in detail.

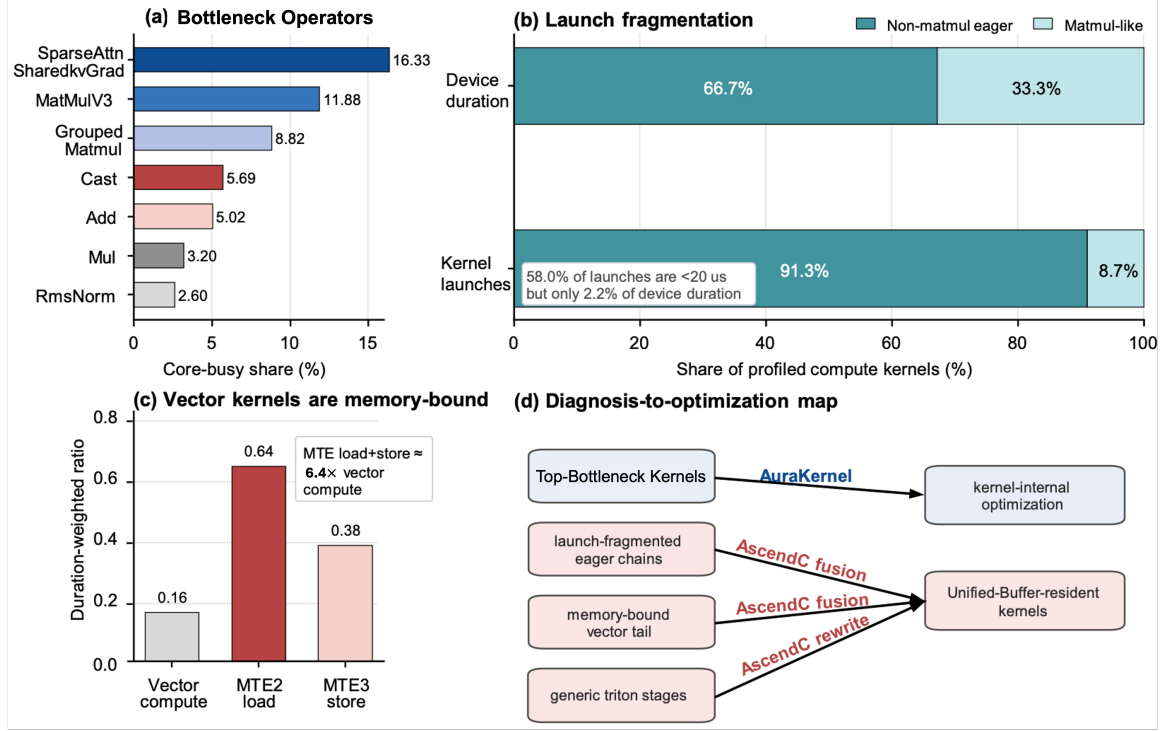


Figure 6: Bottleneck anatomy of one representative DeepSeek-V4-Pro training step on Ascend 910C (206,503 compute kernels, 39.84s device-side duration). Panel (a): the heavy head of architecture-intrinsic operators by core-busy share (the tool’s “Task Duration” column)—sparse-attention backward, dense matrix multiplication, and grouped MoE multiplication. Panel (b): the complementary launch-fragmentation view, contrasting the launch share of non-matmul eager operators with their much smaller share of device duration, and isolating the dense sub-20 μ s tail. Panel (c): duration-weighted vector-pipeline ratios—vector compute well below the MTE2 load and MTE3 store ratios, whose combined memory movement is about 7.4 $\times$ the vector-compute share. Panel (d): the resulting division of work—architecture-intrinsic heavy-head kernels routed to kernel-internal optimization by AuraKernel, and the launch-fragmented, memory-bound, and generic-Triton tails routed to re-expression as UB-resident AscendC kernels.

2.3.2 AuraKernel: Performance Optimization Agent for Complex Ascend Kernels

Achieving high kernel efficiency on Ascend 910C is fundamentally different from optimizing CUDA kernels. Benefiting from the well-established open-source community with diverse data sources, CUDA-based kernel writing has been widely studied and used to train in recent LLM models (Dao et al., 2022; Dao, 2024; Shah et al., 2024; Tillet et al., 2019; Hsu et al., 2024; Ye et al., 2025; Wei et al., 2025; Dong et al., 2025). However, agent-based kernel optimization for non-CUDA hardware remains challenging due to uncommon hardware architecture and programming languages. With fine-grained memory control and pipeline, optimizing Ascend operators via agent system faces two critical challenges:

- **Operator optimization with disaggregated host-kernel design:** Unlike CUDA kernels, a complete AscendC operator package consists of **both** host-side scheduling logic and device-side kernel implementation, enabling flexible control over execution parameters such as tiling, memory movement, and launch configuration. Optimizing AscendC operators therefore requires the agent to reason beyond isolated kernel code and understand the explicit contract between the host and device sides.
- **Long-context understanding with hierarchical agent memory:** Complex AscendC operators, such as sparse-attention kernels, often involve long-range code dependencies, multi-stage data movement, stream-level coordination, and pipeline scheduling. During iterative hardware-agent interaction, the agent must preserve and retrieve relevant design

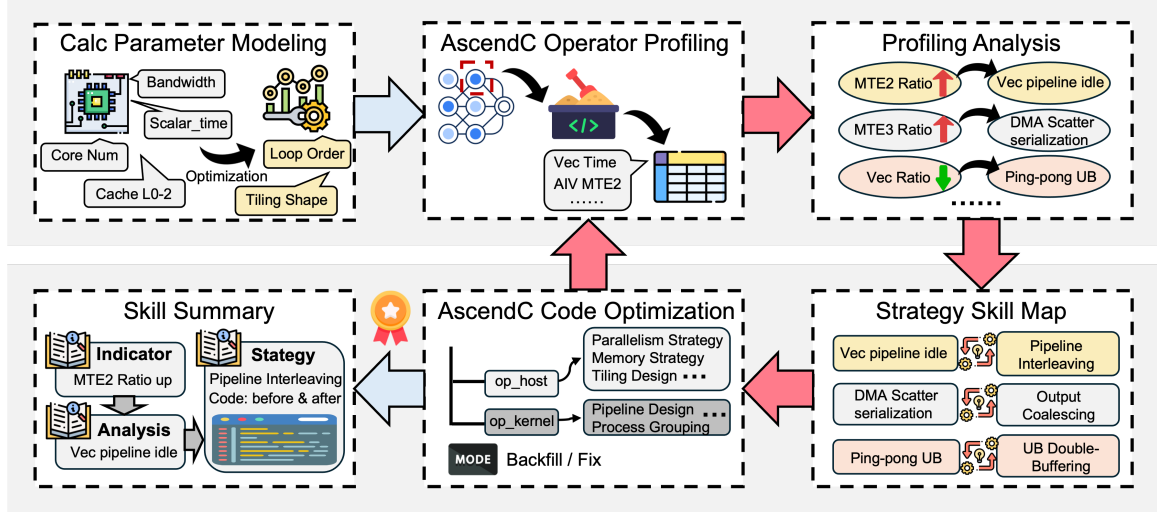


Figure 7: The architecture of AuraKernel. Starting from a functionally correct baseline kernel, AuraKernel generates an optimized operator in three stages. (1) **OR-based tiling optimization** diagnoses the operator’s dominant regime and instantiates a hardware-aware parameter space, yielding an initial configuration and a set of promising optimization directions. (2) **Hardware-grounded iterative optimization** runs a closed modify–verify–profile loop: a harness compiles each candidate, gates it on correctness with the returned profiling, while K-Search uses this feedback to explore a branchable, rollbackable tree of optimization directions and commits a checkpoint on every verified improvement. (3) **Skill distillation** converts each verified gain into a reusable skill that is retrieved to guide hypothesis generation in later episodes.

decisions, profiling observations, and correctness constraints across long optimization trajectories. Effective hierarchical memory is therefore essential for maintaining optimization consistency and avoiding repeated or conflicting modifications.

- **Heuristic optimization leads to suboptimal efficiency:** Existing AscendC operator optimization still heavily relies on expert-crafted heuristics for tiling, buffering, and pipeline scheduling. However, these heuristics are often hardware-, shape-, and operator-specific, making them difficult to generalize across diverse workloads. As a result, purely heuristic optimization can easily converge to locally efficient but globally suboptimal configurations, motivating a more systematic optimization framework that combines Kernel agent with mathematical modeling (for optimal initialization) followed by iterative hardware feedback.

Recent AscendC kernel agents focus on domain-specific training, DSL-guided generation, and profiling-in-the-loop search (Cao et al., 2026b; Wen et al., 2026; Wu et al., 2025a). However, production LLM training introduces a stricter setting: the target kernels must be optimized under real application shapes, memory pressure, layout constraints, and profiling signals from the end-to-end training workload rather than isolated microbenchmarks.

In this work, we propose **AuraKernel**, a performance optimization agent for AscendC kernels. Figure 7 presents the system-level design of AuraKernel. The agent first builds a hardware-aware parameter model to constrain the optimization space; it then enters a profiling-driven loop that diagnoses bottleneck indicators, maps them to concrete optimization strategies, and applies code rewrites; finally, successful indicator–strategy–code patterns are distilled into reusable skills.

Optimized Tiling Strategy via Operations Research. Given a pre-crafted AscendC kernel, AuraKernel initializes the optimization process with a solver-guided tiling strategy tailored to the target computation shape. Instead of relying solely on heuristic search, AuraKernel formulates operator tuning as a constrained performance-modeling problem. The optimizer determines how the workload is partitioned across AI Cores, how tiles are staged through GM, L2, L1, L0, and UB, how DMA transfers are overlapped with CUBE or vector execution, and how intermediate results are written back.

Given an operator shape, AuraKernel first identifies its dominant execution regime, such as compute-bound, memory-bound, scalar/control-bound, or communication-bound. It then instantiates a family-specific parameter space. For GEMM-like compute kernels, the parameters include L0 tile shapes, L1 blocking factors, core decomposition, loop ordering across memory levels, L0/L1 buffer counts, and ping-pong buffering settings. For vector- or reduction-style kernels, the same principle applies to row/block granularity, UB residency, reduction-axis placement, dtype-conversion points, and MTE2/MTE3 scheduling. In this way, the performance model serves as a structured interface between hardware constraints and agent actions.

Rather than optimizing a single opaque latency value, the model explicitly targets the bottleneck stage under a steady-state pipelined execution assumption:

$$\min_{\mathbf{p} \in \mathcal{P}(s)} \max \{T_{\text{MTE1}}(\mathbf{p}), T_{\text{MTE2}}(\mathbf{p}), T_{\text{MAC}}(\mathbf{p}), T_{\text{FixPipe}}(\mathbf{p}), T_{\text{scalar}}(\mathbf{p})\}, \quad (1)$$

subject to data-coverage, L0/L1/UB capacity, legal core partitioning, alignment, and pipeline-validity constraints. Each stage cost is constructed as an explicit clock-cycle-level function of the tiling decision, defined as a transfer or instruction count multiplied by a cycles-per-unit constant, rather than as a single wall-clock latency sample. The per-cycle constants are fitted separately for each level of the on-chip storage hierarchy, with GM, L2, L1, and the L0A/L0B/L0C buffers each assigned a distinct, independently measured read and write bandwidth, so that a given transfer's cost is determined by the specific storage level it traverses rather than by a single averaged rate.

MTE1/MTE2 cycle counts are fitted under a systematic sweep of tensor shapes crossed with layout, transpose, and traversal-order configurations, with each configuration profiled over repeated trials and averaged to suppress measurement noise before cycles are regressed against transfer count and byte volume; an alignment constraint on transfer size is derived directly from this fit, following the observation that misaligned transfers cost disproportionately more cycles per byte moved. MAC cost is fitted as cycles per L0-tile element, and FixPipe cost as cycles per output-tile element under a given writeback layout, using the same regression procedure. Fitting error identified in specific shape regimes is corrected with explicit sub-cycle terms, adjusting fixed per-core launch overhead and per-instruction MAC cost so that the fitted cycle counts match measured behavior rather than being left unmodeled. The steady-state pipeline assumption behind the $\max\{\cdot\}$ formulation is itself derived from cycle-level traces of the double-buffered, ping-pong CUBE-core execution across the $\text{MTE1} \rightarrow \text{MTE2} \rightarrow \text{CUBE} \rightarrow \text{FixPipe}$ stage sequence, in which the four stages are observed to overlap rather than execute serially; their per-cycle occupancies are accordingly treated as concurrent, with the maximum taken rather than a serial sum. This cycle-level calibration is not uniformly exact, as cache hit rate at the L2 level is modeled more coarsely and is identified as a source of residual prediction error on some shapes, though the stage-cost functions themselves remain cycle-accurate constructions rather than opaque empirical latencies.

The solver output is therefore not merely a candidate tiling key, but also a diagnosis of the limiting factor: whether the operator is bottlenecked by movement from GM to L1, L2 reuse, CUBE utilization, writeback, or scalar overhead. This diagnosis is crucial for agentic optimization, because it prevents the agent from spending many iterations on plausible but irrelevant code edits.

The proposed Ascend solver is first validated on a common Matrix Multiplication (MatMul) kernel across frequent shapes in LLM models, with a 6.55% execution time reduction achieved by solely optimizing the tiling strategy. In other words, a solid starting point is created when the iterative kernel optimization process is initiated with optimized hyper-parameters, thereby avoiding the meaningless exploration and token waste that would otherwise occur.

Hardware-Grounded Harness and Loop Engineering. The effectiveness of a kernel optimization agent depends on three critical foundations: ① a scalable and verifiable execution environment, ② a robust sampling mechanism with cross-iteration memory management, and ③ a low-overhead harness that can orchestrate compilation, execution, verification, and profiling at scale. Without these components, the agent may either fail to validate candidate kernels reliably or spend most of its optimization budget on noisy measurements and redundant trials.

Considering these key factors, AuraKernel integrates an asynchronous sandbox system, cross-iteration optimization memory, and a dedicated orchestrator for parallel candidate sampling. The client side hosts the LLM decision loop, K-Search orchestrator, and persistent world model, while

the backend performs source modification, compilation, execution, correctness verification, profiling, and version management on real Ascend NPUs. All hardware-related actions are exposed through stable JSON-over-HTTP tool calls, allowing the search policy or LLM provider to be changed without modifying the backend execution system.

Compilation and execution harness with highly parallel sandbox services. The backend provides a unified interface for both AscendC and Triton-Ascend operators. For AscendC, it supports the joint optimization of host-side tiling logic and device-side kernel code. For Triton-Ascend, it supports meta-parameter tuning such as grid decomposition, block size, pipeline stages, and other backend-specific configuration choices for Ascend NPUs. Kernel development and evaluation are conducted in a highly parallel sandbox system, which provides lightweight isolation for individual Ascend NPU executions while preserving scalable parallel evaluation across devices.

In particular, each sandbox maintains an isolated execution context, including runtime configuration, operator dependencies, temporary build artifacts, and profiling outputs, to reduce cross-candidate interference and improve measurement reproducibility. Meanwhile, independent sandbox instances can compile, execute, and profile multiple candidates concurrently. This design allows AuraKernel to turn expensive hardware feedback into a scalable optimization signal, enabling the agent to explore loop transformations, tiling strategies, buffering choices, and pipeline schedules under real execution constraints. For each candidate kernel ($k \in \mathcal{K}$), the harness first compiles and executes the operator against a user-provided accuracy test that is read-only to the agent. Only implementations satisfying the functional-correctness predicate ($C(k) = 1$) enter performance evaluation, which defines the feasible set

$$\mathcal{F} = \{k \in \mathcal{K} \mid C(k) = 1\}. \quad (2)$$

Rather than using analytical estimates as the final selection criterion, AuraKernel optimizes the measured operator duration under representative application shapes. Specifically, the search process evaluates a sequence of candidate kernels $((k_i)_{i=1}^n)$ under a bounded profiling and LLM-call budget:

$$\sum_{i=1}^n b(k_i) \leq B, \quad (3)$$

and returns the best feasible candidate observed:

$$k^* \in \arg \min_{k_i \in \mathcal{F}} T(k_i), \quad (4)$$

where $T(k_i)$ denotes the measured end-to-end duration of one operator invocation and $b(k_i)$ denotes the evaluation cost of candidate (k_i) . Each profiling pass additionally returns a metric vector:

$$\mathbf{m}(k) = (m_{\text{vec}}, m_{\text{cube}}, m_{\text{scalar}}, m_{\text{mte2}}, m_{\text{mte3}}, \dots) \in \mathbb{R}^d, \quad (5)$$

which characterizes vector or CUBE computation, scalar execution, and MTE2/MTE3 data movement. The measured duration determines whether a candidate improves the incumbent, whereas the metric vector explains why its performance changes and helps the agent distinguish compute-bound, memory-bound, and scalar-bound behavior.

Because hardware measurements inevitably contain noise, a candidate (k') replaces the incumbent (k_b) only when it passes both an improvement threshold and a deduplication check:

$$T(k') \leq (1 - \varepsilon)T(k_b) \quad \text{and} \quad h(\text{canon}(k')) \notin \mathcal{H}, \quad \varepsilon = 0.02, \quad (6)$$

where $\text{canon}(\cdot)$ removes comments and whitespace, h is a content hash, and $\mathcal{H}$ stores hashes of previously evaluated candidates. The first condition rejects changes that are indistinguishable from profiling jitter, while the second avoids redundant evaluation of textually equivalent submissions. Accepted candidates are stored as versioned checkpoints together with their source code, correctness status, measured duration, and profiling metrics.

The harness also enforces the validity of the measured objective. A measurement-boundary rule forbids moving tensor computation from the measured kernel into the Python script or host launcher; an operator-kind lock prevents cross-paradigm replacement, such as rewriting a Triton-Ascend operator as AscendC when the task requires preserving the original implementation paradigm; These constraints prevent direct speedups obtained by changing the operator contract, bypassing unsupported hardware paths, or moving work outside the profiling boundary. Consequently, every

optimization hypothesis is evaluated through the same reproducible sequence: modify the code, precision verification, candidate profiling, and behavior recording.

Unlike recent CUDA-oriented kernel agents (Chen et al., 2026b; MIT HAN Lab, 2026), where the pre-defined knowledge bases play an important role in guiding optimization, our findings suggest that static knowledge bases and case-specific optimization skills can constrain the exploration space of LLM agents, especially on non-traditional computing platforms such as Ascend NPUs.

In contrast, AuraKernel relies on self-evolved memory and a self-maintained Skills that are continuously updated through hardware-verified feedback. During iterative search, successful transformations, failed edits, profiling diagnoses, and versioned performance records are accumulated into persistent memory, allowing the agent to refine its optimization strategy across candidates rather than repeatedly relying on fixed, case-by-case rules. This feedback-driven memory mechanism enables AuraKernel to achieve direct performance improvements on both complex AscendC kernels and Triton-Ascend operators, as shown in Section 4.

K-Search optimization-path exploration. On top of this harness, we adopt K-Search (Cao et al., 2026a) to organize candidate generation as a stateful, branchable, and rollbackable tree search, referred to hereafter as the world model, rather than a single linear “view–edit–profile” loop. Its root is the verified baseline, its internal nodes describe optimization directions such as tiling-granularity adjustment, MTE2 reduction, and pipeline interleaving, and its leaves are concrete code-change attempts. Each leaf is annotated with its predicted gain, confidence, implementation difficulty, visit count, correctness status, profiling result, and metric-vector transition.

At each iteration, K-Search selects a leaf from the current frontier $L(\mathcal{T})$ using an upper-confidence acquisition function that balances expected gain, exploration, and implementation difficulty:

$$\ell^* = \arg \max_{\ell \in L(\mathcal{T})} \left[\underbrace{c_\ell \hat{g}_\ell}_{\text{expected gain}} + \kappa \sqrt{\frac{\ln(1+N)}{1+n_\ell}} - \gamma d_\ell \right], \quad (7)$$

where ℓ^* denotes the leaf node from which further exploration is launched. $\hat{g}_\ell$ is the predicted speedup of leaf ℓ , $c_\ell \in [0, 1]$ is its confidence, d_ℓ is its estimated difficulty, n_ℓ is its visit count, $N = \sum_\ell n_\ell$, and $\kappa, \gamma > 0$ control the trade-off between exploration and implementation cost. Each search cycle follows a continuous optimization loop: it retrieves relevant optimization experience, formulates a hypothesis, modifies the AscendC or Triton code, invokes the harness for compilation and correctness verification, profiles each valid candidate, and updates the best-known checkpoint when the measured improvement satisfies the acceptance criterion. Let $(T(k_i))$ denote the measured execution time of the (i) -th profiled candidate. The best observed execution time after profiling (n) candidates is:

$$T_n^* = \min_{i \leq n} T(k_i), \quad (8)$$

which is non-increasing with n , even though candidates may regress or fail during exploration.

K-Search further calibrates its optimization priors through sibling-priority back-propagation. After a leaf ℓ is evaluated, its realized gain g_ℓ is compared with its predicted gain $\hat{g}_\ell$, and the priorities of related sibling directions are updated as

$$\text{score}(\ell') \leftarrow \text{score}(\ell') + \eta \left(\mathbb{1}[r_\ell > \tau_\uparrow] - \mathbb{1}[r_\ell < \tau_\downarrow] \right), \quad r_\ell = \frac{g_\ell}{\hat{g}_\ell}, \quad (9)$$

for every $\ell' \in \text{sib}(\ell)$, with $\eta = 0.1$, $\tau_\uparrow = 1.5$, and $\tau_\downarrow = 0.5$. Here, $\text{score}(\ell')$ instantiates the confidence weight $c_{\ell'}$ used in the acquisition function of Eq. (7), so promoting or demoting a sibling directly raises or lowers its $c_{\ell'} \hat{g}_{\ell'}$ term in subsequent applications of Eq. (7). Directions related to an unexpectedly effective candidate are promoted, whereas those associated with an underperforming candidate are demoted, and this adjustment is what carries the evaluated leaf’s outcome into the selection score of its still-unevaluated siblings. The WorldModel is thus continuously recalibrated by measurements from the actual NPU rather than by the LLM’s initial confidence alone.

When a path fails correctness checks, regresses in latency, or reaches a performance plateau, K-Search rolls back to a previously verified checkpoint and opens a new branch. This enables non-monotonic exploration of the coupled tiling, buffering, memory-layout, and pipeline-scheduling space while protecting the global incumbent. Each cycle is governed by limits on profiling calls,

tool calls, and consecutive non-improving attempts, together with a global cycle cap. For searches spanning dozens of cycles, a three-layer context-compression mechanism keeps the transcript tractable without discarding recoverable state: a failed profiling round, spanning every message since the last successful profile, is collapsed into a single summary recording the failure cause and the last verified checkpoint; repeated views of the same file are replaced by a compact marker referencing the most recent view, keyed by file and board; and repeated directory listings are deduplicated the same way, keyed by operator and board. The harness and K-Search therefore form complementary halves of the same loop: the former establishes whether a modification is correct and measurably faster, while the latter determines which optimization direction to attempt next and how hardware evidence should reshape the remaining search.

Skill distillation and continual improvement. For each checkpoint that achieves a verified gain, the agent distills a structured optimization skill from the metric-vector transition observed before and after the code change, represented as a triple

$$\sigma = (\mathbf{m}_{\text{before}}, \Delta \mathbf{m}, \pi), \quad \Delta \mathbf{m} = \mathbf{m}_{\text{after}} - \mathbf{m}_{\text{before}}, \quad (10)$$

where π is the applied code transformation. Given the current metric vector $\mathbf{m}$, the agent retrieves stored skills whose pre-image best matches it and conditions hypothesis generation on a temperature-controlled softmax over similarities,

$$w_j \propto \exp\left(\frac{1}{\beta} \text{sim}(\mathbf{m}, \mathbf{m}_{\text{before}}^{(j)})\right), \quad \text{sim}(\mathbf{a}, \mathbf{b}) = \frac{\langle \mathbf{a}, \mathbf{b} \rangle}{\|\mathbf{a}\| \|\mathbf{b}\|}, \quad (11)$$

with temperature $\beta > 0$. These skills are stored in an experience bank and retrieved to condition LLM hypothesis generation in subsequent episodes, so the agent progressively accumulates transferable AscendC optimization knowledge as the bank grows. All runs emit complete, replayable artifacts—including the serialized world model—so an optimization can be resumed from an intermediate cycle on another machine. In effect, AuraKernel recasts kernel optimization as an automatable search problem in which the LLM’s hypothesis generation and the NPU’s objective measurement continually calibrate each other, converging toward near-optimal AscendC implementations within a strict budget. Admittedly the

Applied to DeepSeek-V4-Pro training (Section 4.1), AuraKernel accelerates the shared-key-value sparse-attention backward step, the single largest device-time contributor, by up to 1.24× in its end-to-end training context, and in a single campaign optimizes 14 reduction-heavy framework operators to full correctness with a 2.06× average and an 11.90× best speedup. Consistent with our bottleneck analysis, these gains come from cutting redundant global-memory traffic rather than from raising raw compute utilization.

2.3.3 Ascend NPU-aware Kernel Fusion and Rewrite

The second optimization path addresses the three non-head patterns of the bottleneck analysis—the launch-fragmented eager tail, the memory-bound vector tail, and the generic Triton kernels. Whereas AuraKernel improves the body of already-large operators, this path re-expresses each target stage as one or a small number of hardware-affine AscendC kernels. The first two patterns reach the runtime as eager Python and torch chains and the third as generic Triton kernels, so all three collapse onto the two entries introduced above. For the eager chains, fusion recovers the high-level operation from the trace, classifies the emitted kernels into model arithmetic, shape bookkeeping, data movement, precision conversion, and autograd writeback, and collapses the chain. For the Triton kernels, the arithmetic is unchanged but the operator is rebuilt in AscendC with the on-chip residency and scheduling the generic lowering cannot express. The two entries overlap: the weight-free RMS normalization and the mHC pre-projection contraction are simultaneously fused and rewritten, because their Triton kernels also carried avoidable stage boundaries. The system-level communication and pipeline bottlenecks remain outside this scope. In both cases the arithmetic is unchanged, and only the substrate on which it runs changes.

On a high level, the mathematical contract records the formula that the fused implementation must preserve, including clamp subgradients, complex RoPE rotation, weighted expansion, reduction, and dtype-sensitive accumulation. The frontend-artifact record identifies operators that appear in the runtime trace because of the Python expression, view semantics, dtype promotion, or autograd expansion, such as casts, views, fills, tensor moves, concatenations, and scatter-style writebacks.

The hardware execution contract specifies the Ascend plan: which tensors are resident in UB, how GM-UB movement is scheduled through MTE2/MTE3, where reductions or broadcasts occur, and which dtype is carried at each boundary. This structure lets each fusion be reviewed as a mathematics-preserving rewrite rather than as a collection of local peephole optimizations.

The resulting kernels follow three hardware-facing design rules. First, intermediate tensors remain inside the Unified Buffer whenever their only role is to connect adjacent eager operators. Second, the kernel interface returns whole rows or whole stage outputs, so that slice-assignment and scatter-style autograd expansions are not generated in the first place. Third, dtype transitions follow the numerical contract of the stage rather than the conservative dtype inherited from a frontend expression. These rules do not remove model arithmetic. Instead, they change where the arithmetic is executed and how many times the surrounding memory traffic is paid. We apply this method to three representative targets, presented in the order used throughout the evaluation: the mHC compression suite, limited SwiGLU backward, and unified rotary embedding. A mixed-precision dataflow rewrite of the mHC pre-projection path is developed alongside the first, and appears as its own row in the atlas below. Figure 8 serves as the method overview for these rewrites. It separates each case into the observed trace, the semantic contract used to audit correctness, the frontend artifacts removed by fusion, and the AscendC boundary that implements the revised dataflow.

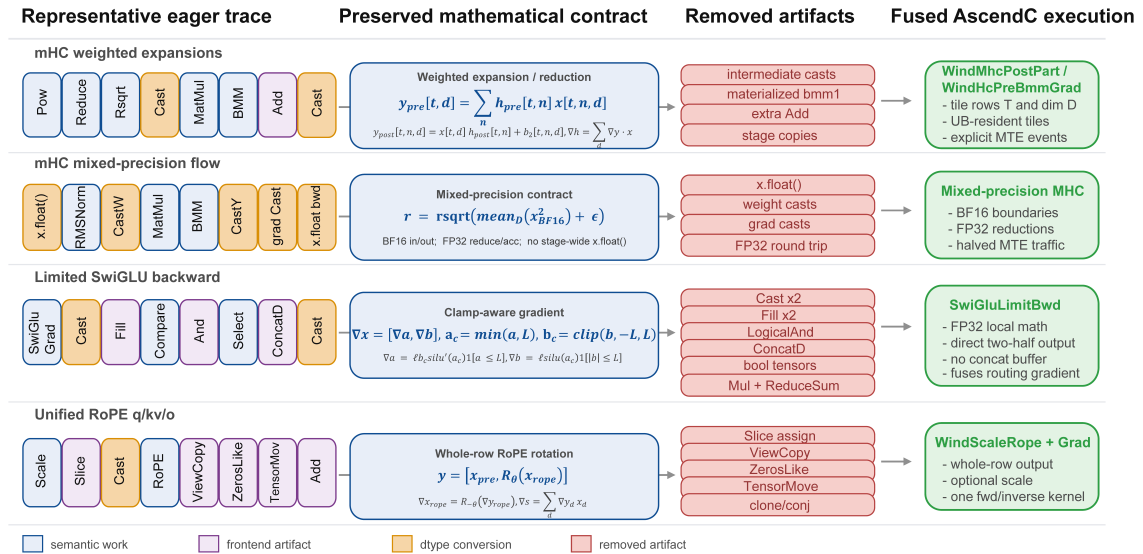


Figure 8: Operator-chain fusion atlas. Rows show four DeepSeek-V4-Pro rewrites: mHC weighted expansion, mHC mixed-precision dataflow, limited SwiGLU backward (with routing-weight gradient) and unified RoPE. Columns give the emitted trace, semantic invariant, removed frontend artifacts and fused AscendC boundary.

mHC fusion suite. The Manifold-Constrained Hyper-Connections (mHC) path (Xie et al., 2025) is a larger instance of the same problem, and it is not represented by a single fused operator. It also exercises both entries of this path at once: the weight-free RMS normalization and the pre-projection contraction originated as generic Triton kernels, so their rewrite into AscendC is a substrate change, while the surrounding casts and elementwise accumulation are eager fragments removed by a stage-boundary fusion. Its pre-projection stage combines weight-free RMS normalization, a small projection whose output is normalized by a Sinkhorn step into per-channel mixture weights, and a batched weighted expansion driven by those weights. Its post-projection stage combines another weighted expansion with an accumulated tensor and a final BF16 output. In the original trace, these semantics appear as Pows/ReduceMean/Add/Rsqrt, Cast, MatMul, custom BMM kernels, and elementwise accumulation. The cost is spread across normalization, conversion, and movement around the projection rather than concentrated in one matmul. Figure 9 summarizes the three mechanisms that make this path efficient on the NPU: the weighted expansion (a), the two-dimensional tiling and pipeline (b), and the mixed-precision dataflow (c).

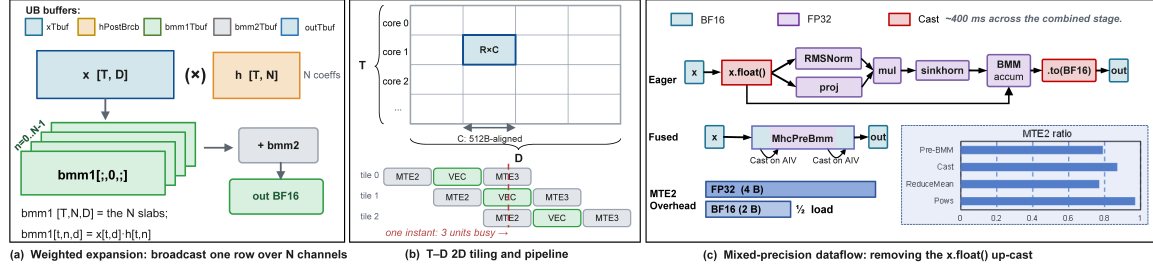


Figure 9: Three mechanisms of the mHC weighted-expansion fusion, detailed in the text. (a) The weighted expansion $bmm1[t, n, d] = x[t, d] h[t, n]$ over the N mixture channels, accumulated and emitted as BF16 in one UB-resident kernel. (b) The two-dimensional $[T, N, D]$ tiling and the three-stage MTE2/VEC/MTE3 pipeline, where the dashed line marks the instant at which all three engines are busy. (c) The pre-projection dataflow, in which $x.float()$ forks into RMS normalization and the projection, whose product feeds the Sinkhorn step and then the BMM. The inset gives the per-operator MTE2 ratios that make the chain memory-bound.

The purpose of the mHC suite is to expose the compression path as a small number of stable stage boundaries rather than as several unrelated peephole fusions. Table 3 lists the five kernels and the contract each one preserves.

Table 3: mHC fusion suite. Each row identifies a concrete AscendC kernel, the frontend-emitted chain it replaces, the mathematical contract preserved by the fused implementation, and the hardware-facing boundary used by the kernel.

Kernel	Replaced chain	Preserved mathematical contract	Hardware boundary
WindMhcPostPart	Cast + post-BMM1 + Add + Cast	$y_{\text{post}}[t, n, d] = x[t, d] h_{\text{post}}[t, n] + b_2[t, n, d]$	Tile over rows T and feature dim D ; UB-resident expansion and direct BF16 output
wind_rms_norm_without_weight	Pows + ReduceMean + Add + Rsqrt	$r[t] = \text{rsqrt}(\frac{1}{D} \sum_d x[t, d]^2 + \epsilon)$	Row-wise reduction in one local dataflow; FP32 reciprocal-scale output
WindHcPreBmmForward	pre-BMM forward casts + weighted-expansion contraction	$y[t, d] = \sum_{n=0}^{N-1} h_{\text{pre}}[t, n] x[t, n, d]$ ($N=4$)	Shared T - D tiling; Brcb-broadcast weights, FP32 h_{pre} / BF16 x , BF16 output
wind_rms_norm_without_weight_backward	Pows/Mul/Div RMS backward fragments	$\nabla x[t, d] = -\nabla r[t] r[t]^3 x[t, d] / D$	Compute row scalar once; expand across D inside the tile
WindHcPreBmmBackward	pre-BMM backward casts, zero/init, reduce fragments	$\nabla x[t, n, d] = \nabla y[t, d] h_{\text{pre}}[t, n]$ $\nabla h_{\text{pre}}[t, n] = \sum_d \nabla y[t, d] x[t, n, d]$	Shared T - D tiling for weighted expansion and reduction to ∇h_{pre}

WindMhcPostPart handles the post-projection stage. It flattens the batch and sequence dimensions into $T = BS$, computes the weighted expansion $x[t, d] h_{\text{post}}[t, n]$, accumulates the existing b_2 tensor, and emits BF16 output directly. The kernel tiles over rows T and hidden dimension D : rows are split across the cores to provide independent token-level work, while D is tiled into 512 B-aligned blocks that bound UB residency and MTE transfer granularity. This two-dimensional tiling is the common hardware contract across the mHC kernels. It lets the implementation reuse local buffers across expansion, accumulation, and output conversion, with MTE2/VEC/MTE3 movement scheduled inside the fused operator rather than by framework-level kernel boundaries.

The pre-projection side applies the same boundary principle to reductions and weighted expansion, and it is also where the substrate rewrite is most explicit: the weight-free RMSNorm and the pre-BMM contraction were previously generic Triton kernels, and rebuilding them in AscendC is what exposes the on-chip control the affinity argument relies on. The RMS-normalization forward and backward kernels fuse the square, reduction, epsilon addition, reciprocal square root, and gradient expansion so that the row scalar is computed once and reused locally. WindHcPreBmmForward carries the forward weighted expansion $y[t, d] = \sum_n h_{\text{pre}}[t, n] x[t, n, d]$ under a T - D tiling, broadcasting each FP32 h_{pre} coefficient with a Brcb instruction and consuming BF16 activations, so that the four-channel contraction is completed in one UB-resident pass instead of a matmul-plus-cast chain. A single vector multiply then applies one broadcast coefficient across a full register of feature elements, and a three-stage MTE2/VEC/MTE3 pipeline with ping-pong buffering overlaps the load of the next tile with the compute and store of the current one—the explicit residency, broadcast, and pipeline scheduling that a generic Triton lowering leaves to global-memory round trips. This substrate change moves the pre-BMM forward from movement-bound to compute-bound, and Section 4.2 reports the measured speedup and the corresponding rise in vector-compute occupancy.

WindHcPreBmmBackward then computes both outputs of the weighted expansion: ∇_x is formed by broadcasting h_{pre} across the feature dimension and multiplying by ∇_y , while ∇h_{pre} is obtained by reducing $\nabla y \cdot x$ across D . These kernels convert the mHC path from a sequence of frontend-generated fragments into a stage-level dataflow whose memory movement is scheduled once.

Backward of Limited SwiGLU. The limited SwiGLU activation adds a pre-SiLU clamp to the standard gated product: the gate half is clipped by an upper bound and the up half is clipped symmetrically before multiplication. In eager form, the backward pass expands this compact mathematical rule into a chain containing SwiGluGrad, two Cast kernels, scalar Fill kernels, three comparisons, LogicalAnd, two SelectV2 kernels, and ConcatD. The trace shows a roughly 401 μs memory-bound chain per invocation. Most of that chain does not introduce new model arithmetic, but instead materializes clamp constants, boolean masks, dtype round trips, and the concatenation needed to rebuild the gradient tensor.

The fused kernel implements the closed-form backward directly. Let $x = [a, b]$, where a is the gate half and b is the up half, and let ℓ be the incoming gradient. With $a_c = \min(a, \text{limit})$, $b_c = \text{clip}(b, -\text{limit}, \text{limit})$, and $\sigma = \sigma(a_c)$, the kernel computes

$$\nabla_a = \ell \cdot b_c \cdot (\sigma + a_c \sigma (1 - \sigma)) \cdot \mathbf{1}[a \leq \text{limit}], \quad (12)$$

$$\nabla_b = \ell \cdot a_c \sigma \cdot \mathbf{1}[-\text{limit} \leq b \leq \text{limit}]. \quad (13)$$

The fused boundary writes the two gradient halves directly to their output offsets, removing the concatenation and the frontend-visible boolean-composition chain. BF16/FP16 inputs are promoted once for the local derivative computation, while the sigmoid, clamp masks, and intermediate products remain in UB and the result is converted only at the output boundary. We verified the formula symbolically and against a PyTorch autograd golden reference, including clamp-boundary cases. Unsupported dtypes and non-positive limits fall back to the original path.

In the MoE setting the same kernel also fuses the routing-gradient reduction, which the eager path emits as its own chain. Each expert activation is scaled by a per-token routing weight p , so the layer computes $\text{out} = h \cdot p$ with h the limited-SwiGLU output, and the routing network requires $\nabla p = \sum_d \ell[t, d] h[t, d]$, a reduction over the hidden dimension. In eager form this reduction is three further kernels: a reload of h , a full-size elementwise product $\ell \cdot h$, and a ReduceSum, each paying its own global-memory round trip. The fused kernel instead forms ∇p inside the same pass that computes the activation gradient. The values it needs, $\sigma(a_c)$, a_c , and $\text{clip}(b)$, are already resident in UB, so the kernel reconstructs h locally and reduces it row by row into ∇p without materializing $\ell \cdot h$ or issuing any extra load. Because ∇p depends on the unweighted ℓ while ∇x depends on $\ell \cdot p$, the reduction is taken before the incoming gradient is scaled by the routing weight. The reconstructed h is kept in FP32 within the kernel, which matches the reference training path and avoids the BF16 reload that the eager chain reads back from global memory. When the hidden row is split across column tiles, each tile contributes a partial sum that is combined into the row total by a global-memory atomic accumulation. The routing gradient is validated against a FP32 autograd reference by relative L_2 error, appropriate for a reduction output whose per-element accumulation order differs from the reference. In the profiled step the eager routing-gradient chain is a full-size $\ell \cdot h$ product and a reduction that together cost about 82 ms. Both vanish once the reduction moves inside the fused kernel, and Section 4.2 reports the combined effect on the backward stage. Figure 10 shows the whole backward chain collapsing into this single kernel.

Unified rotary embedding. Rotary position embedding appears in three attention sites with different surrounding context. The query path first multiplies by the exposed RMS-normalization scale and then rotates the RoPE tail. The compressed key-value path applies RoPE without an external scale and uses local frequencies. The output-projection path applies the inverse rotation and previously relied on a clone plus conjugate construction. These sites differ in frontend expression, but they share the same stage boundary: preserve a full-row prefix, rotate the RoPE tail, and return the row without exposing slice assignment to autograd. In the backward pass, exposing that assignment produces ZerosLike, TensorMove, ViewCopy, Cast, and Add chains around the actual rotation gradient. Figure 11 summarizes the three mechanisms the unified operator applies.

We implement the three sites with one forward operator and one backward operator. The operator returns the pass-through prefix and the rotated tail as a whole-row output, so no slice assignment is exposed to autograd and the backward scatter chain is never generated (Figure 11a). Because the

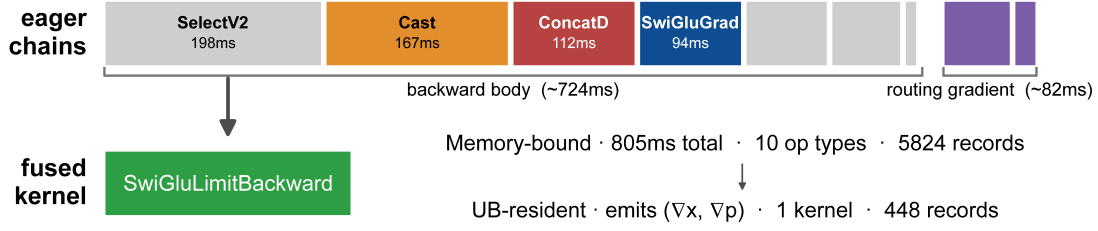


Figure 10: Schematic of the limited-SwiGLU backward chain collapsing into one fused kernel. The eager row shows the operator types emitted by autograd: SelectV2, Cast, ConcatD, SwiGluGrad, a trailing group of comparison, mask, and fill kernels (LessEqual, GreaterEqual, LogicalAnd, Fill), and, in the MoE path, the routing-weight-gradient reduction (Mul and ReduceSum), with tile widths proportional to their per-type trace time. Fusion replaces the entire chain with a single UB-resident SwiGluLimitBackwardV2 kernel that keeps the sigmoid, clamp masks, intermediate products, and routing-gradient reduction on chip and never materializes them to global memory.

target hardware has no native complex type, the operator reproduces the complex rotation in real arithmetic [Su et al. \(2024\)](#): it treats each adjacent element pair as one complex number and rotates it with a cosine multiply, an adjacent-pair swap, and a sign-flipped sine, so that a single vector code path realizes the complex product without a complex-construction chain (Figure 11b). Forward and inverse rotation share one kernel through the sign of the sine term, and the same kernel serves q, kv, and o because site-specific differences reduce to scale, frequency, and direction arguments rather than separate operator implementations.

The backward kernel follows the same boundary choice, keeping the rotation derivative and the scale-gradient reduction inside the fused operator. The query path still feeds the mathematically required RMS-normalization backward and preserves the final accumulation of the two gradient branches. In isolated per-site validation, the unified operator covers q, kv, and o with relative error within the accepted training tolerance.

The operator consumes real cosine and sine tensors, which are produced from the complex frequency table by a preparation chain of RepeatInterleave, StridedSlice, Neg, and Pack. Left in the graph, this chain re-executes every step, even though the frequencies depend only on position and do not change across steps. We therefore cache the converted tensors under a key derived from the frequency table, so that after the first step the cache hits and the entire preparation chain, together with its memory-movement traffic, is skipped (Figure 11c). The cache is keyed so that a stable frequency table across steps keeps the downstream lookup valid, which is what lets the preparation cost fall to zero in steady state.

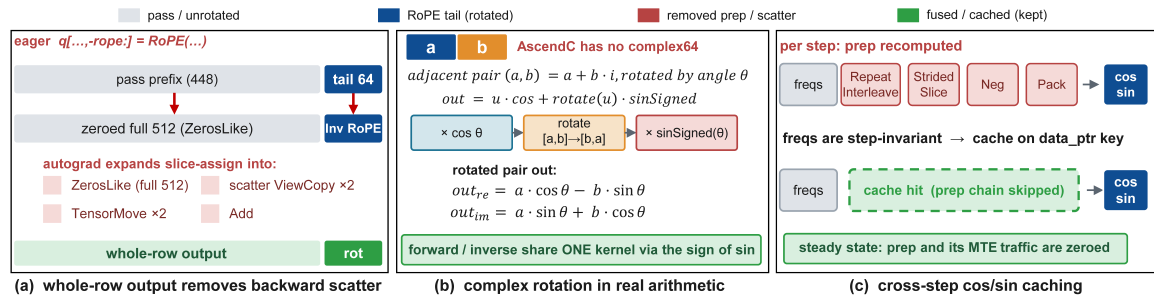


Figure 11: Three mechanisms of the unified rotary-embedding operator. (a) The eager in-place slice assignment and the full-width backward scatter it forces, contrasted with the whole-row output that avoids it. (b) The real-arithmetic realization of the complex rotation, showing the cosine multiply, adjacent-pair swap, and sign-flipped sine that produce the rotated pair. (c) The per-step cosine/sine preparation chain against its cached counterpart, where a step-invariant frequency table lets the chain be skipped after the first step.

Mixed-precision dataflow. The mHC pre-projection path also contained a stage-wide precision artifact. The Python implementation performed an early `x.float()` after flattening the input, which promoted the activation tensor from BF16 to FP32 before RMS normalization, the projection, and the BMM. Because the surrounding training path is BF16 mixed precision, this promotion forced a full-stage round trip: activations, gradients, and intermediate accumulations were repeatedly converted or moved at FP32 width, and the final outputs were converted back. In the profile, the mHC pre-projection stage is invoked hundreds of times per step (896 forward and 448 backward calls); the `x.float()` choice therefore amplifies into visible Cast kernels and doubled transfer volume across the forward and backward chains. Crucially, this stage is memory-bound rather than compute-bound: its dominant operators run at an AIV vector-compute ratio of only ≈ 0.06 against an MTE2 load ratio of ≈ 0.87 (Pows 0.97, ReduceMean 0.77, Cast 0.87; Figure 9c), so the wall time is set by how many bytes cross global memory, and promoting the whole chain to FP32 doubles exactly the quantity that bounds it. The single largest mover is the entry Cast from BF16 to FP32, at 222 ms per step.

We remove the up-cast by making the mixed-precision contract explicit, and the central move is to shift the dtype at which operands cross global memory. Because the chain is MTE2-bound, the kernels load activations from global memory in BF16 and convert them to FP32 inside the vector core, rather than loading an already-promoted FP32 tensor. This halves the load volume of exactly the pipe that bounds the stage while preserving accuracy, since the FP32 conversion still happens before any reduction. The RMS-normalization kernels accept BF16 activations while retaining FP32 reciprocal-square-root values and FP32 internal arithmetic where the reduction requires it. The projection computes BF16 $\times$ BF16 products with FP32 accumulation and emits the dtype required by the downstream stage. The pre-BMM forward consumes h_{pre} in FP32 and x in BF16, and returns BF16 output; its backward returns BF16 ∇_x and FP32 ∇h_{pre} . The weight-gradient path similarly returns gradient with the matched data type as the original BF16 weight, rather than retaining the FP32 tensors because the corresponding activation was upcast for computation.

This change is small in code but broad in effect because it changes the dtype carried by the whole mHC subgraph. The analysis identifies each removed or narrowed movement: the initial activation cast, weight casts inserted before matmul, forward and backward BMM input/output casts, gradient-add casts, and the final `x.float()` backward conversion. The scale of the opportunity is set by the entry cast alone, which the FP32 promotion inflates to 241 ms in the forward pass, so shifting the dtype at which operands cross global memory acts on the dominant term rather than a marginal one. The concrete before-and-after reduction across the combined stage, together with the end-to-end module speedups measured after fusion, is reported in Section 4.2. Correctness is argued operator by operator: reductions and accumulations that require FP32 retain FP32 internally, while tensor-wide BF16 values remain BF16 at the stage boundaries, matching the mixed-precision contract used by the surrounding training system.

Taken together, this fusion and rewrite turns the long-tail diagnosis into an implementation discipline. Across the four targets the recurring move is the same: recover the model-level operation from its frontend expansion, hold its mathematical contract fixed, and choose the granularity and substrate at which it meets the NPU, whether the starting point was an eager autograd chain or a generic Triton kernel. What changes is not the arithmetic but where it runs and how often its operands cross global memory. The module-level effect of these fusions is reported in Section 4.2, and their aggregate contribution to end-to-end training throughput is reported with the other operator-level optimizations in Section 2.

3 OR-Oriented Post-Training Workflow

Building on the system infrastructure established in Section 2, we further explore how to adapt the DeepSeek-V4 (DeepSeek-AI et al., 2026) family to the complex domain of Operations Research (OR). We use DeepSeek-V4-Flash as a validation model for the OR-oriented post-training pipeline. DeepSeek-V4-Flash retains the core structured reasoning capabilities of the V4 family, and the verified pipelines can directly serve as a blueprint for larger models such as DeepSeek-V4-Pro.

Operations Research (OR) modeling provides a demanding setting for evaluating domain-oriented post-training. A model must translate natural-language requirements into variables, objectives, constraints, data interfaces, and solver-executable programs, while preserving the mathematical

Table 4: Diagnostic performance of the original DeepSeek-V4-Flash checkpoint before our CPT and SFT stages. All benchmark entries are accuracies under the corresponding evaluation setting; the final column reports the total number of failed cases scanned under each setting.

Setting	NL4OPT	OptiBench	B4O-Feasible	B4O-ORGEval	Failures
Pass@1	84.08	63.33	60.47	34.26	661
Pass@16	93.77	78.33	80.52	60.66	370
5-shot	88.24	66.00	78.20	71.57	425

structure of the underlying optimization problem. Recent studies on LLMs for optimization have shown that this process involves more than answer generation: it requires semantic understanding, symbolic formulation, solver-aware implementation, and structure-sensitive verification (Xiao et al., 2025; Wang & Li, 2025; Mostajabdaveh et al., 2025; Wang et al., 2025b). These properties make OR a suitable domain for examining whether post-training can improve both executable behavior and structural modeling competence.

As a lighter-weight member of the DeepSeek-V4 family (DeepSeek-AI et al., 2026), DeepSeek-V4-Flash enables rapid iteration over data construction, continued pre-training, supervised fine-tuning, checkpoint selection, and benchmark evaluation, while retaining the modeling and reasoning characteristics needed for a realistic vertical-domain study. The original DeepSeek-V4-Flash checkpoint already exhibits non-trivial OR modeling ability, but its behavior varies across Pass@1, Pass@16, and 5-shot. This variation provides a useful starting point for analyzing which capabilities are already present, which capabilities are only activated by context, and which failures require parameter-level adaptation.

This section presents an OR-oriented post-training validation framework that connects original-checkpoint diagnosis, CPT-SFT design, and stage-wise evaluation. We first analyze the original checkpoint on OR benchmarks and summarize representative failure cases, focusing on protocol compliance, solver-facing implementation, mathematical formulation, and structural equivalence. We then connect these observations to the training design: CPT is used to broaden OR-domain knowledge and modeling priors, while SFT is used to align the checkpoint with verified Gurobi code, task-specific output contracts, and canonical formulation styles. Finally, we validate the workflow through stage-wise experiments, including CPT training stability, SFT data scaling and cleaning, and matched CPT-to-SFT transfer.

3.1 Capability Boundary Analysis of the Original Checkpoint

Before introducing the post-training recipe, we diagnose the original DeepSeek-V4-Flash checkpoint on the same OR evaluation suite used throughout this work. The base checkpoint is not a blank slate: it already writes many valid Gurobi programs and solves a substantial fraction of natural-language optimization problems. The practical question is more specific: which parts of the OR modeling pipeline are already latent in the model, which parts can be activated by prompting or sampling, and which parts require parameter updates through CPT or SFT.

3.1.1 Benchmark-Level Signal

Table 4 summarizes the original checkpoint under Pass@1, Pass@16, and 5-shot. For NL4OPT, OptiBench, and B4O-Feasible, a sample is counted as correct when the generated solver program returns the reference objective value. For B4O-ORGEval, correctness requires structural equivalence to the reference optimization model, measured by the ORGEval WL reward. This distinction is central to our diagnosis: executable code and even plausible objective values do not guarantee the correct variable family, constraint family, or constraint-variable graph.

The results reveal three diagnostic signals. First, B4O exposes a clear structural bottleneck. Under Pass@1, B4O-Feasible reaches 60.47%, while B4O ORGEval reaches only 34.26%. Meanwhile, the code/build pass rate for B4O ORGEval is 79.19%, indicating that many generated programs are executable but do not preserve the intended mathematical structure. This gap shows that solver executability and structural equivalence are only weakly aligned in the original checkpoint.

Table 5: Error-family distribution of the original DeepSeek-V4-Flash checkpoint. Entries are shares among failed cases in each setting (Overall: 1,456; Pass@1: 661; Pass@16: 370; 5-shot: 425).

Error family	Overall	Pass@1	Pass@16	5-shot
Structural equivalence	28.3	26.8	34.3	25.4
Protocol / API / schema	23.9	30.1	24.3	13.9
Ratio / nonlinear modeling	13.8	11.6	13.0	17.9
Discrete-variable semantics	12.5	13.0	8.6	15.1
Complex constraints / objectives	10.8	9.1	9.5	14.6
Objective / units / extraction	7.6	7.0	6.2	9.6
Numerical tolerance / rounding	3.2	2.4	4.1	3.5

Second, Pass@16 improves NL4OPT, OptiBench, and B4O-Feasible compared with Pass@1, showing that additional sampling can recover more valid solver-value solutions. However, Pass@16 still remains below 5-shot prompting on B4O ORGEval, suggesting that sampling alone does not reliably select canonical optimization formulations.

Third, the 5-shot performance of B4O-ORGEval increases from 34.26% to 71.57%, which demonstrates that the contextual examples can activate solver-facing protocols, Gurobi implementation patterns, and formulation templates that are already partially present in the checkpoint. These observations motivate the following CPT-SFT design: CPT strengthens OR-domain modeling priors and structural formulation knowledge, while SFT consolidates prompt-activated behaviors into stable one-pass generation.

3.1.2 Error Spectrum Across Settings

To make the diagnosis actionable, we conducted a comprehensive error analysis on the original checkpoint across the three inference settings and four benchmark cells, covering 1456 failed cases in total. Table 5 and Figure 12 group these failures into broad error families to show how the distribution changes from Pass@1 to Pass@16 and 5-shot. Overall, structural equivalence errors form the largest family: WL graph mismatch, variable-count mismatch, and constraint-count mismatch together account for a large fraction of B4O-ORGEval failures. Under Pass@1, the most visible failure surface is protocol, API, and schema compliance, including markdown fences in code-only tasks, model extraction failures, and malformed Python. The remaining high-frequency errors reflect missing OR modeling priors: discrete entities modeled as continuous variables, incomplete constraint systems, objective-scale mistakes, ratio or efficiency modeling errors, and nonlinear forms that are not compatible with the solver.

3.1.3 Inference-Setting Shift

The distribution shift across inference settings separates promptable behavioral failures from deeper modeling failures. Under Pass@1, the checkpoint produces 661 failures, with protocol/API/schema errors forming the largest family: 199 cases, or 30.1%. This is the raw evaluator-facing failure surface of the original checkpoint. Moving to 5-shot reduces failures to 425 and cuts protocol/API/schema errors to 59 cases, showing that many code-only contracts, Gurobi idioms, and schema-handling habits are already latent and can be activated by demonstrations. Moving to Pass@16 further reduces total failures to 370 and improves value-based benchmarks, but structural ORGEval errors become the largest remaining family, 127 cases or 34.3%. Thus, additional sampling can retrieve more numerically correct solutions, but it does not reliably choose the canonical variable-constraint graph.

The same observations can be read as a training-oriented failure taxonomy. Below, the failure types are ordered by their overall share among the 1,456 scanned failed cases. This paragraph-level view separates failures that mainly block the evaluation interface from failures that expose missing OR modeling priors, while keeping the representative cases and training implications tied to concrete samples.

Canonical structure mismatch under ORGEval (28.3%). This is the largest failure type in the scan and the most important structural problem. In the pharmaceutical blending cases BENCH4OPT_0/1, the generated LP can be executable, but it fails variable-count or constraint-count checks because

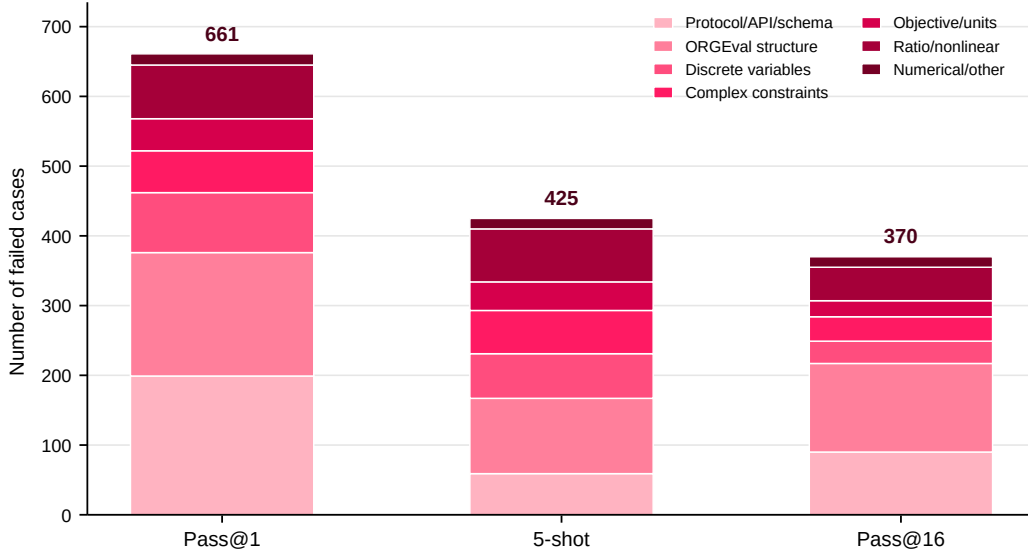


Figure 12: Stacked error-family counts of the original checkpoint under Pass@1, 5-shot, and Pass@16. The bar heights show total failed cases, while the colored segments show how the failure composition changes. 5-shot sharply reduces protocol/API/schema failures; Pass@16 reduces total failures but leaves a large structural ORGEval segment.

the model uses a non-reference variable or constraint family. BENCH4OPT_12 further shows that even when variable and constraint counts match, the WL graph can still be wrong because the constraint-variable connections differ. These failures are important because solver success or a plausible objective value is not enough for ORGEval: the model must produce the canonical LP graph. CPT is needed for domain templates such as blending, scheduling, and flow; SFT is needed to align these templates with benchmark-facing variable and constraint families.

Output protocol, syntax, API, and schema failures (23.9%). This is the most fatal evaluation-interface problem. In BENCH4OPT_4, a capital-budgeting problem, the Pass@1 response adds a markdown code fence in a code-only target, causing a syntax failure before model construction. In BENCH4OPT_176, a nurse-scheduling problem, a string shift label is used to index a list of requirements, triggering a schema/indexing runtime error. These cases show that the checkpoint may have usable OR reasoning internally but fail to expose it through the required program interface. Non-executable code is scored as zero regardless of whether the mathematical idea is correct, so this family can severely mask the checkpoint’s real OR capability. It primarily motivates SFT: code-only contracts, verified Gurobi APIs, and schema-safe templates must become stable Pass@1 behavior.

Discrete variable semantics (12.5%). NL4OPT 15 is a staffing problem that asks for numbers of senior and young workers. The model encodes the worker counts as GRB.CONTINUOUS and returns the LP relaxation value 28125.0 instead of the integer optimum 28250.0. The same relaxation persists under 5-shot and Pass@16, showing that prompting and sampling do not reliably recover the missing integrality prior. This is a core OR semantics problem: the model understands many constraints, but fails to map countable business entities to integer or binary variables. CPT should strengthen countable-entity priors, while SFT should provide explicit integer/binary Gurobi demonstrations for workers, trips, batches, vehicles, assignments, and selections.

Loss/yield-driven conservation constraints (10.8%). BENCH4OPT_2 is a water-distribution network-flow problem with capacity, cost, evaporation rate, and node demand. The Pass@1 model solves an LP but mixes evaporation into both cost and conservation in a way that returns 265.58 instead of 531.15. 5-shot and Pass@16 recover the correct value, showing that the template is promptable but not stable. This represents complex constraint semantics rather than mere code generation. Loss, yield, shrinkage, and evaporation determine where conservation equations change

and whether costs or delivered quantities change. CPT should teach these domain patterns, while SFT should distill promptable templates into stable data-separated code.

Ratio, unit, and total-quantity coupling (7.4%). OptiBench 6 is a logistics investment problem where route-optimization investment reduces per-truck delivery cost and total investment must not exceed total savings. Pass@1 and Pass@16 recognize the bilinear relation but mishandle investment units and total savings; 5-shot removes the truck-count multiplier from the savings constraint. This failure corrupts the economic meaning of an optimization model while still producing plausible algebra. Ratio, ROI, average-cost, percentage, and efficiency expressions require the model to distinguish per-unit quantities, totals, denominators, and scaling units. CPT should broaden this semantic prior, while SFT can teach verified formulations.

Solver-compatible nonlinear reformulation (6.4%). OptiBench 13 asks for the closest feasible rescue point on a parabolic boundary. The Pass@1 solution writes a fourth-order distance objective directly into Gurobi and triggers an “objective must be linear or quadratic” error. 5-shot and Pass@16 avoid the build error but still return the wrong distance, which means the missing piece is not just API syntax but nonlinear reformulation knowledge. This blocks a whole class of mathematically valid OR tasks. The model must know when to introduce auxiliary variables, when to use quadratic/nonconvex/general constraints, and when analytic simplification is required. CPT is the main lever; SFT hardens the verified Gurobi patterns.

This taxonomy explains why the setting-wise distribution motivates both training stages. The Pass@1 distribution is dominated by protocol/API/schema and structural failures, so SFT is needed to make code-only generation, Gurobi API usage, objective extraction, and JSON-style data access stable without in-context demonstrations. The 5-shot distribution shows which failures are distillable: markdown fences, several API mistakes, and some common formulation templates improve sharply once examples are present. This directly motivates verified SFT and few-shot distillation. However, the residual 5-shot errors concentrate in integer semantics, complex constraints, ratio/nonlinear modeling, and ORGEval structure, which are not just output habits. They require CPT to broaden the model’s OR formulation priors before SFT can reliably elicit them.

The Pass@16 distribution adds a different lesson. Because total failures fall from 425 to 370 and value-based accuracies improve, the original sampling distribution already contains many recoverable correct programs; this supports verifier-guided SFT and rejection-sampled distillation from Pass@ N outputs. But Pass@16 leaves more structural ORGEval failures than 5-shot, both in absolute count (127 vs. 108 grouped structural errors) and in share (34.3% vs. 25.4%). Sampling therefore improves search over solutions without enforcing the canonical LP graph. This is the key bridge to CPT plus SFT: CPT shifts the model toward OR-native structural priors, while SFT turns those priors into benchmark-facing, executable, and aligned outputs.

3.1.4 Two-Stage Training Adaptation: Why Both CPT and SFT

In large language model pipelines, continual pre-training serves as a knowledge-infusion stage—adapting the model to a domain through self-supervised learning on unlabeled corpora (Gururangan et al., 2020; Ke et al., 2025; Siriwardhana et al., 2024)—while instruction tuning acts as a behavior-shaping stage (Ouyang et al., 2022; Wei et al., 2022; Sanh et al., 2021) that aligns the model with human intent via supervised learning on instruction-response pairs. The core distinction is that continual pre-training determines what the model knows, and instruction tuning controls how it responds. Their relationship is one of foundation and activation: continual pre-training first builds a domain-informed knowledge base, and subsequent instruction tuning elicits it as coherent, instruction-following behavior (Yang et al., 2025a; Jindal et al., 2024; Jiang et al., 2025; Zhang et al., 2025a). Only this cascaded synergy yields models that are both domain-proficient and reliably usable.

The setting-wise diagnosis gives a clean division of labor. CPT targets knowledge acquisition: OR terminology, standard formulation patterns, solver-aware nonlinear reasoning, fractional and efficiency modeling, data-structure grounding, and structural modeling priors. These are the failure modes that persist even when examples or multiple samples are added: integer-domain semantics, nonlinear reformulation, ratio modeling, missing constraint families, and canonical LP graph structure.

SFT targets solver-oriented behavioral alignment. It should teach the model to obey code-only and LP-writing contracts, avoid markdown fences, use verified Gurobi APIs, map JSON/list/dict data safely into index sets, extract objective values with the right units, and reproduce benchmark-facing canonical formulations. The 5-shot improvements show that many of these habits are already promptable; SFT converts that prompt dependence into stable Pass@1 behavior. The Pass@16 improvements show that correct solutions often exist in the sampling distribution; verifier-guided distillation can convert those retrieved samples into parameterized competence. The remaining structural and modeling failures explain why this SFT stage should be built on top of OR-oriented CPT rather than replacing it.

3.2 OR-Oriented Continued Pre-Training

The original-checkpoint analysis shows that DeepSeek-V4-Flash already contains a useful OR modeling core, especially on standard LP and MILP templates, but its remaining errors concentrate on variable-domain semantics, solver-compatible nonlinear reformulation, ratio- and efficiency-based modeling, complex constraint construction, and structural equivalence. These failure modes indicate that the next stage should not only teach the model to follow benchmark protocols, but also strengthen its prior over OR-domain concepts, mathematical-programming structures, and solver-aware modeling patterns. We therefore use CPT as the knowledge-acquisition stage in the OR-oriented post-training workflow, following the domain-adaptive pre-training paradigm for adapting language models to specialized distributions (Gururangan et al., 2020; Chen et al., 2026a).

In this stage, the model is further trained on a domain-dominant mixture built around solver-verified OR documents. The design is guided by the capability gaps observed in the original checkpoint: integer and binary variable semantics motivate broader exposure to countable-entity formulations; nonlinear and ratio-based failures motivate solver-aware reformulation data; and ORGEval structural mismatches motivate data that preserves canonical variable families, constraint families, and constraint-variable connections. Following this principle, our CPT workflow centers on an OR-CPT data engine that converts parameterized optimization generators into traceable, business-oriented, and executable modeling documents. The resulting CPT stage serves as a bridge between data construction and SFT: it expands the model’s OR-domain modeling distribution before SFT aligns the checkpoint with verified Gurobi code, strict output contracts, and benchmark-specific formulation styles (Jiang et al., 2025).

3.2.1 OR-CPT Data Construction and Domain-Dominant Mixture

The core of our CPT data construction is the OR-CPT Data Engine, a solver-verified bidirectional synthesis workflow that converts structured optimization instances into natural-language modeling documents, as illustrated in Figure 13. The workflow targets the capability gaps observed in the original-checkpoint analysis, including variable-domain ambiguity, weak parameter grounding, incomplete constraint construction, nonlinear or ratio-based modeling failures, and structural mismatch under ORGEval-style evaluation. Each accepted CPT document is traceable to a generated optimization instance, an independently verified solver reference, and a checked forward modeling result.

The workflow starts from parameterized optimization instance generators. Each generator specifies the decision variables, objective, constraints, numerical parameters, and task-specific structure of an optimization family. The current generator registry covers assignment, scheduling, facility location, network flow, production planning, lot sizing, routing, covering, transportation, portfolio optimization, and revenue management. For each instance, the engine records structured metadata, including task family, generator identity, concept tags, difficulty label, generation parameters, and canonical mathematical signature. This metadata is used for corpus sampling, auditing, and concept-level analysis, but is not exposed as model-facing CPT text. We also maintain separation between the data-generation assets and the ORGEval evaluation assets: no ORGEval reference formulation, canonical LP graph, or evaluator-side template is reused when rendering CPT documents. The intended overlap is at the level of broad OR problem families and modeling concepts, rather than at the level of shared canonical templates or instance provenance.

Each generated instance is independently executed with Gurobi to obtain a reference solver status, objective value, variable assignment, and constraint diagnostics. The engine then applies instance-level quality filters to remove unstable or low-value samples, including infeasible instances,

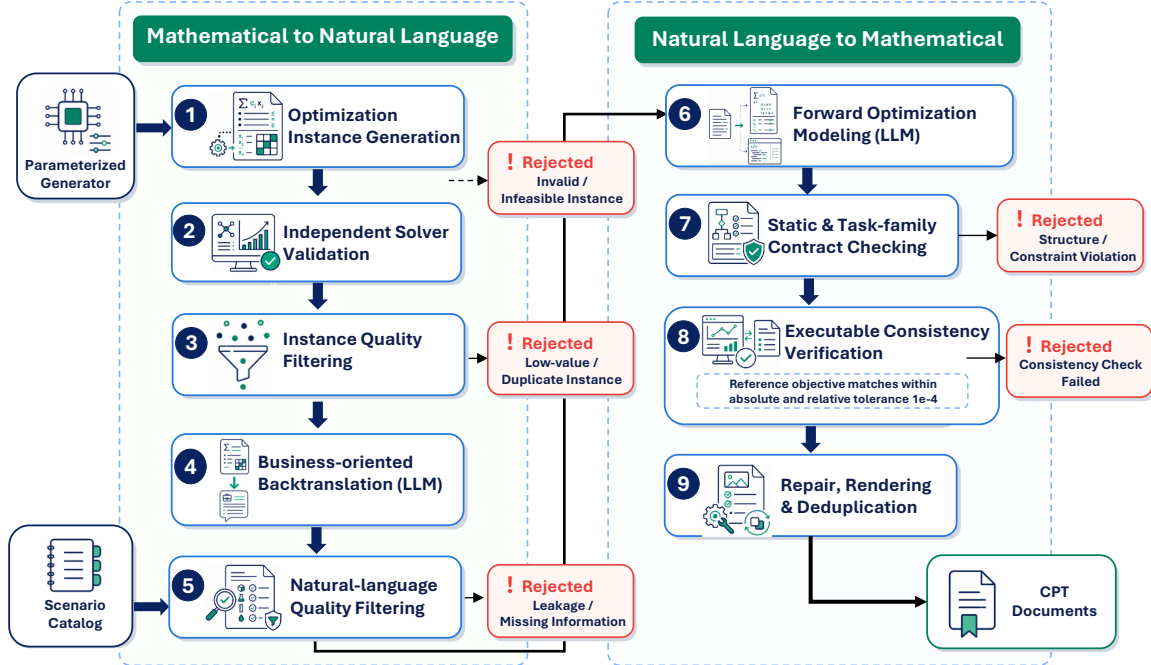


Figure 13: Pipeline of OR-CPT data construction. Parameterized optimization generators produce structured instances, which are verified with Gurobi, rendered into business-oriented problem statements, reconstructed into executable formulations, and filtered by contract checks, solver execution, and objective-value agreement before export.

degenerate all-zero solutions, duplicated parameter configurations, duplicated solution signatures, and weak optimization trade-offs. This solver-grounded filtering step provides the mathematical anchor for the synthetic corpus and improves the quality of the subsequent natural-language modeling documents.

After solver validation, the structured instance is converted into a business-oriented natural-language problem. A scenario catalog controls the industry background, organization type, planning horizon, entity naming style, units, and narrative perspective. Deterministic leakage checks prevent the problem statement from exposing the optimal objective value, solver status, implementation code, or other answer-related artifacts. This step turns a mathematical instance into a realistic modeling prompt while preserving the underlying optimization structure.

The second direction of the workflow performs forward modeling from the generated natural-language problem. Given only the problem statement, a model produces decision variables, objective functions, constraints, modeling explanations, and executable Gurobi code. The generated answer is checked by static rules and task-family contracts. For example, facility-location samples are expected to connect facility-opening variables with assignment variables, network-flow samples should contain flow-conservation constraints, and scheduling samples should preserve assignment, coverage, and capacity structures. The reconstructed Gurobi program is then executed independently and compared with the solver reference from the original structured instance. A sample is accepted only when the program reaches a valid solver status and its objective value matches the reference solution within the verification tolerance. Recoverable failures are repaired with structured solver feedback, and accepted documents are rendered and deduplicated before export.

This bidirectional design gives the synthetic CPT corpus three useful properties. It is scalable because new data can be produced from parameterized OR generators across many task families. It is verifiable because each accepted document is tied to solver execution and reference-objective agreement. It is structurally informative because the workflow preserves the connection among natural-language requirements, variable families, constraint families, solver code, and objective values. These properties make the OR-CPT Data Engine the main source of solver-grounded modeling signal in our CPT stage.

The complete CPT corpus combines these solver-verified synthetic documents with collected OR-domain resources. The collected portion covers optimization modeling, mathematical reasoning, and solver-oriented problem solving, including OptMATH (Lu et al., 2025), ReSocratic and OptiBench (Yang et al., 2025c), Text2Opt-Bench (Gao et al., 2026), OR-Instruct (Huang et al., 2025), NLP4LP (Ahmaditeshnizi et al., 2024), ORQA-derived data (Mostajabdaveh et al., 2025; Ge, 2025), and MAMO (Huang et al., 2024). We also include OR research papers, official Gurobi modeling examples, and textbook-style materials to increase coverage of formal modeling conventions, solver usage, and domain terminology (Kong et al., 2026; Gurobi Optimization, LLC, 2026; Lownes, 2024). All collected resources are converted into a unified CPT document format and processed through source-aware cleaning, formatting, and deduplication. At the current stage, the OR data pool contains approximately 100K samples, which serve as the corpus inventory for CPT sampling.

We adopt a domain-dominant mixture strategy in which OR documents provide the main adaptation signal. Mathematical, code, and general English corpora are included as supporting components. The mathematical component is drawn from selected AutoMathText-V2 subsets (Li et al., 2025), following the AutoMathText data-selection methodology (Zhang et al., 2025c), and supports symbolic formulation, derivation, and multi-step reasoning. The code component is drawn from NVIDIA Nemotron pretraining data releases, including scientific-coding data associated with the Nemotron Nano series and code-concept data associated with Nemotron 3 Super (NVIDIA et al., 2025, 2026), and supports procedural reasoning, solver-oriented implementation, and robust code generation. The general English component uses Ultra-FineWeb-L3, derived from Ultra-FineWeb under the UltraData tiered data management framework (Wang et al., 2025a, 2026), to maintain broad language comprehension and generation quality. In this mixture, OR data strengthens formulation semantics and structural modeling priors, mathematical data supports reformulation and symbolic reasoning, code data supports executable solver implementation, and general English data stabilizes broad language capability. The exact sampling allocation is treated as a recipe-level parameter and reported with the training configuration.

3.2.2 Training Execution in the Ascend Ecosystem

We execute OR-oriented CPT within the Ascend software ecosystem, using MindSpeed and TorchTitan-NPU as two implementation routes. Both routes follow the same autoregressive training objective, OR-oriented data interface, monitoring protocol, and checkpoint-evaluation workflow. This design keeps the CPT recipe consistent across different Ascend training abstractions, so that data construction, optimization behavior, and downstream validation remain comparable.

The execution setup is configured around the main requirements of DeepSeek-V4-Flash CPT. Long-context training is supported through context parallelism, which is important for OR documents that contain business descriptions, mathematical formulations, solver traces, and multi-step modeling explanations. Sparse MoE execution is supported through expert parallelism, token dispatch, and grouped expert computation. The training stack also supports model-specific attention and MoE operator paths, MTP training, activation recomputation, and memory-aware optimizer execution. These capabilities provide the execution basis for full-parameter CPT while keeping the model architecture and language-modeling objective unchanged.

The broader Ascend system design, including cluster organization, collective communication, operator optimization, runtime monitoring, and fault recovery, is reported in Section 2. Here, the training execution serves as the substrate for the OR-oriented CPT recipe. During CPT, we monitor loss dynamics, learning-rate progression, gradient norms, numerical stability, throughput, memory behavior, and iteration stability. These signals are used to identify stable candidate checkpoints for intermediate evaluation and subsequent SFT. Recipe-level settings, including sequence length, batch size, learning-rate schedule, parallelism degrees, optimizer configuration, and memory-saving mechanisms, are summarized in Appendix B.

3.2.3 Checkpoint Selection and CPT-to-SFT Transfer Validation

CPT checkpoints are selected according to their value as SFT initializations. We evaluate intermediate checkpoints with three complementary signals: OR-domain likelihood, general-capability retention, and downstream OR transfer. The OR likelihood signal is measured on a held-out OR validation set that is disjoint from the CPT training mixture. It provides an early indication of whether the checkpoint better models OR terminology, formulation patterns, and solver-oriented documents.

General validation loss and representative general-capability benchmarks are tracked in parallel to monitor whether the checkpoint preserves the mathematical, coding, and language abilities needed for later instruction tuning.

We further evaluate selected checkpoints on OR downstream benchmarks. These benchmarks cover different aspects of optimization modeling: NL4OPT emphasizes natural-language-to-formulation ability, OptiBench evaluates broader optimization problem understanding, B4O-Feasible measures solver-facing feasibility behavior, and B4O-ORGEval tests structural modeling quality through a stricter formulation-level evaluation. This benchmark set allows us to check whether improvements in OR-domain likelihood translate into measurable gains in modeling, implementation, and structural-equivalence behavior.

Since CPT is designed as a preparation stage for SFT, its most important evaluation is matched transfer validation. We compare two controlled training routes:

Route A: Original Checkpoint $\rightarrow$ SFT, (14)

Route B: Original Checkpoint $\rightarrow$ CPT $\rightarrow$ SFT. (15)

The SFT dataset, optimizer configuration, update steps, and evaluation protocol are kept the same between the two routes. As a result, the difference between the trained models reflects the contribution of CPT as an initialization stage. For a downstream benchmark b , we define the transfer gain as:

$$\Delta_{\text{transfer}}^{(b)} = S_{\text{CPT+SFT}}^{(b)} - S_{\text{SFT}}^{(b)} \quad (16)$$

where $S_{\text{CPT+SFT}}^{(b)}$ is the score of the model trained through Route B and $S_{\text{SFT}}^{(b)}$ is the score of the matched SFT-only route.

The final checkpoint is selected with a multi-objective criterion. A preferred checkpoint should reduce OR validation loss, maintain stable optimization signals during training, preserve general capabilities within the expected range, and produce positive CPT-to-SFT transfer on the primary OR benchmarks. This selection procedure connects the CPT stage directly to the later SFT and experimental analysis: CPT is not only evaluated as a language-modeling continuation, but as a domain-adaptive initialization whose value is determined by downstream OR modeling gains under matched training conditions.

3.3 Data-Efficient Supervised Fine-Tuning for Operations Research

We conduct supervised fine-tuning for DeepSeek-V4-Flash on operations research modeling tasks. The goal is to adapt the model from general instruction following to solver-oriented mathematical programming, with emphasis on benchmarks such as NL4OPT, OptiBench, and Bench4Opt. Unlike general code-generation tasks, OR modeling evaluation does not only check executability. It also evaluates natural-language semantic understanding, variable-type selection, objective and constraint construction, Gurobi-style API usage, external data loading, answer-format contracts, and structural equivalence of mathematical formulations.

Since large-scale SFT data in the OR domain is scarce, heterogeneous, and uncertain in quality, we adopt a DeepSeek-V4-Flash cyclic self-distillation method inspired by iterative data-construction frameworks such as ReSocratic-style synthesis (Yang et al., 2025c; Zhang et al., 2025b). More broadly, self-distillation has been shown to be effective for bootstrapping high-quality synthetic data (Lee & Liang, 2026), especially when combined with structured validation for optimization modeling tasks (Wu et al., 2025b). We construct three training-data scales, namely 3K, 10K, and 50K. Based on the comparative experiments reported in Section 4.4, we choose the 10K-scale data as the basis for further cleaning and enhancement. The final SFT design therefore focuses not only on scaling synthetic data, but also on improving the correctness, contract awareness, and learnability of the distilled samples.

3.3.1 Solver-Oriented SFT Objective

The SFT stage is designed to train the model to form stable optimization-modeling habits. The model should identify sets, parameters, variables, objectives, and constraints; correctly read structured data files; use Gurobi-style Python APIs to build and solve models; and, when required, write LP files without solving them. The target response is therefore not a simple numerical answer,

Table 6: Problem modes used in self-distilled OR modeling data.

Mode	Name	Description
DP	Data in Problem	Data are directly written in the problem statement.
DT	Data in Table	Data are provided in a structured table, such as a Markdown table.
DPS	Data-Problem Separate	The problem statement and data files are separated, such as <code>data.json</code> , <code>instance.json</code> , or CSV files.

but a structured modeling trajectory that connects natural-language understanding, mathematical formulation, and solver-facing implementation. This follows recent findings that supervised fine-tuning is an important foundation for equipping LLMs with optimization reasoning and modeling capabilities (Ding et al., 2026; Xiao et al., 2026).

Given an instruction prompt x and a target response $y = (y_1, \dots, y_T)$, the SFT stage optimizes the standard autoregressive cross-entropy loss over the response tokens:

$$\mathcal{L}_{\text{SFT}} = -\frac{1}{T} \sum_{t=1}^T \log p_{\theta}(y_t | x, y_{<t}). \quad (17)$$

The loss on prompt, system, and formatting tokens is masked, so that gradient updates are applied only to the target response. This encourages the model to learn the desired reasoning trajectory, mathematical formulation, and solver implementation, while avoiding unnecessary optimization on fixed prompt templates.

3.3.2 Self-Distilled OR Modeling Data Flywheel

We construct a self-distillation data-production pipeline for operations research and mathematical programming tasks. The input is a set of seed samples, including original problems, reference answers, reference code, data files, or LP files from high-quality modeling QA data. These seed samples are not directly exported as final training data. Instead, they serve as references for problem types, modeling structures, and target response specifications.

The seed samples are organized into three input modes, as summarized in Table 6.

To make the self-distillation process controllable and auditable, we introduce three levels of intermediate representations. This layered design follows progressive synthesis strategies in optimization data generation, where intermediate representations help control problem complexity and structural diversity (Yang et al., 2025c; Wu et al., 2025b).

- **L1 Canonical IR** normalizes heterogeneous samples and records the source path, target contract, data interface, reference-file status, static code features, risk labels, and objective-function hints.
- **L2 Semantic IR** extracts the mathematical-programming semantics of the seed sample, including the objective function, sets, parameters, variables, constraints, data interface, and potential modeling risks.
- **L3 Synthetic IR** generates a new modeling-task abstraction based on the L2 seed and the historical accepted synthetic pool. It must not copy the seed problem statement, answer, code, LP file, file name, path, exact numbers, or formulation. Instead, it varies the domain, entities, objective semantics, variable families, constraint families, parameter schema, numerical range, and data shape.

To further reduce the risk of data leakage, we apply leakage-control checks throughout the distillation process. Candidate synthetic samples are compared against held-out evaluation samples at multiple levels, including surface-level problem wording and abstract modeling structure. Semantic-similarity filtering is used to reject near-duplicate problem statements, while IR-level constraints prevent the reuse of the same objective semantics, constraint patterns, parameter schema, or numerical values. These controls ensure that the accepted SFT samples differ from the test set not only in textual

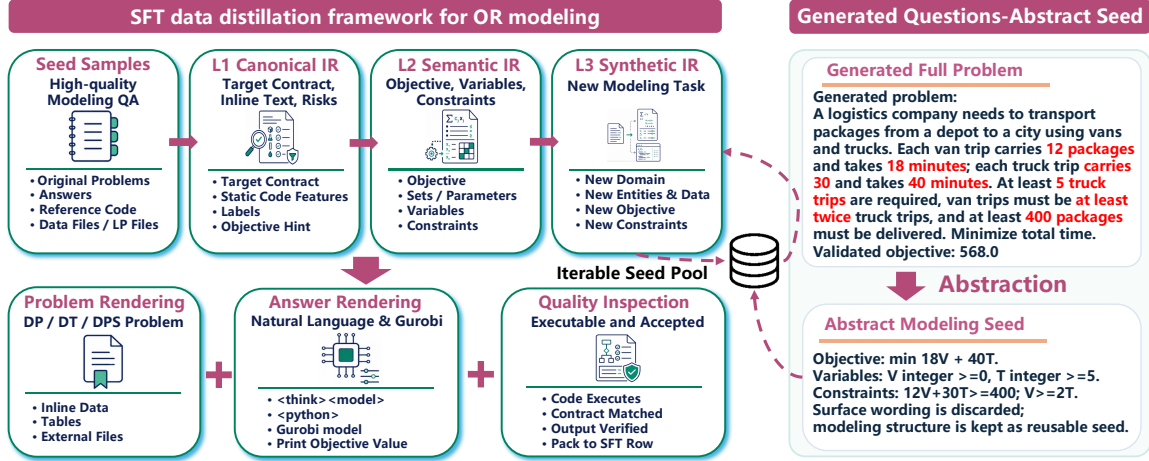


Figure 14: Overview of the SFT data distillation framework for OR modeling. The pipeline converts high-quality seed samples into canonical and semantic intermediate representations, generates novel synthetic IRs, renders them into multiple problem formats, validates the corresponding answers, and feeds accepted synthetic IRs back into the next self-distillation round.

expression, but also in data form and modeling structure, thereby reducing the possibility that the model learns leaked benchmark instances rather than generalizable OR modeling behaviors.

After L3 generation, the pipeline renders the synthetic IR into DP, DT, or DPS problems. DP samples embed data directly into the statement, DT samples contain structured Markdown tables, and DPS samples generate external files such as `data.json`, `instance.json`, or CSV files. The answer rendering stage then produces target responses according to the task-specific output contract. For Markdown-style answers, the response contains `<think>`, `<model>`, and `<python>` tags, while the Python code uses Gurobi to build and solve the model and output the objective value. In this way, the model learns not a single code template, but a family of modeling-output protocols under different task requirements.

The pipeline further adopts a flywheel-style iteration. Samples accepted into SFT in each round have their corresponding Synthetic IRs accumulated into the parent pool for the next generation round. The model therefore refers not only to the original Semantic IR, but also to previously accepted synthetic structures. This gradually reduces direct dependence on the original seeds and expands the modeling-structure space, while still using the seed data as anchors to prevent distributional drift. As illustrated in Figure 14, the overall framework connects seed samples, intermediate representations, problem rendering, answer rendering, automatic validation, and accepted-pool feedback.

3.3.3 Contract-Aware Cleaning and CoT Enhancement

The observations from preliminary SFT runs motivate a shift from merely enlarging the self-distilled dataset to improving the learnability of already distilled samples. In the comparison among 3K, 10K, and 50K reported in Section 4.4, the 10K dataset already significantly improves Bench4Opt protocol-oriented tasks, but it remains below the baseline on NL4OPT and OptiBench. Further scaling to 50K does not recover this ability. Therefore, instead of continuing to expand the same pipeline to 100K, we perform fine-grained cleaning of the existing 10K data, focusing on modeling errors and chain-of-thought expression.

The cleaning process targets concrete error types exposed in the previous stage. In optimization modeling tasks, structural errors that directly damage the training signal include incorrect variable types, wrong objective direction, wrong constraint direction, missing capacity or demand constraints, improper handling of ratios or averages, and incorrect use of the Gurobi API. For example, objects such as counts, vehicles, routes, assignments, and selections are usually modeled as integer or binary variables, whereas continuous quantities such as tons, liters, flow rates, and working hours should not be over-integerized.

We design two cleaning branches. The first branch is **clean-only**, which repairs real modeling or implementation errors while preserving the original reasoning style as much as possible. It does not actively add new explanatory content. The second branch is **Clean-CoT**, which repairs errors and adds a concise modeling checklist to the target response. This checklist makes the `<think>` block explicitly cover variable types, objective direction, key constraints, ratio or nonlinear forms, and Gurobi execution actions. The goal is not to generate long free-form reasoning, but to expose the model repeatedly to a stable modeling pattern: determine variable domains, determine the objective, check constraints, and implement the formulation in Gurobi. This design is consistent with recent findings that CoT-style SFT can transfer reasoning ability more effectively when the reasoning traces are structured and task-relevant (Yu et al., 2025; Lin et al., 2025; Yeo et al., 2025).

The cleaning workflow uses four levels of gating: cleaner, reviewer, local validation, and export checking. The cleaner generates repaired candidate answers. The reviewer independently determines whether a candidate is suitable for SFT and blocks over-repair, such as incorrectly changing continuous variables into integers, deleting necessary constraints, or mishandling ratio and big-M formulations. Local validation then checks code executability, Gurobi calls, output contracts, and LP writing paths. In the full 10K run, clean-only passes reviewer filtering for 9,043 out of 10,000 samples, and 8,731 samples finally pass validation and are exported. Clean-CoT passes reviewer filtering for 9,174 out of 10,000 samples, and 8,881 samples are finally exported.

Overall, this cleaning stage makes self-distilled SFT data more contract-aware and more learnable. For tagged Markdown targets, short structured `<think>` blocks inject the most error-prone optimization-modeling checkpoints into the training distribution. For code-only and LP-structure-equivalence tasks, the output contract remains strict.

3.3.4 Progressive SFT with General Capability Anchoring

Our scaling experiments reveal a conflict between two capabilities that SFT must instill simultaneously: syntactic precision in solver-API usage and semantic reasoning in natural-language modeling. In a single-stage setup, gradient updates from pure code targets such as Bench4Opt and reasoning-rich targets such as NL4OPT may interfere destructively, leading to the non-monotonic performance observed when scaling from 10K to 50K samples. To reduce this conflict, we decouple these capability dimensions into a two-stage progressive training strategy. This follows the general idea of progressive domain adaptation, where broad task competence and specialized tuning are separated into different training stages (Zhang et al., 2025a).

Stage I, **Solver-API Alignment**, trains exclusively on code-only OR targets, including LP-writing and feasibility tasks. It uses a conservative learning rate of 5×10^{-7} to establish a stable syntactic backbone for Gurobi invocations, data-loading patterns, and output contracts. Stage II, **Full OR Modeling SFT**, continues from this checkpoint on the complete Clean-CoT 10K dataset with a learning rate of 1×10^{-6} . This allows the model to learn formulation and reasoning while preserving low-level solver syntax. By separating solver-API alignment from full modeling adaptation, the training process reduces the destructive interference observed in monolithic SFT (Yıldız et al., 2024).

However, OR-specific SFT can still cause degradation in general language understanding and code syntax health, as observed in curriculum monitoring and the NL4OPT decline in Table 13. This drift toward domain specialization may erode the semantic representations needed for natural-language modeling and agentic diversity. To counteract this forgetting, we augment the Stage II mixture with 10% – 15% general-purpose instruction data, including open-domain QA, multi-turn dialogue, and creative writing. This general-capability anchoring maintains gradient updates from diverse linguistic tasks, helps preserve NL4OPT performance, and mitigates the parametric homogenization observed in agentic rollout experiments. As a result, the SFT checkpoint provides a more stable and explorable initialization for the downstream agentic pipeline.

3.4 AI-Assisted Data Construction and Refinement

To scale up the data construction process described above, we employ AI-assisted tools to automate candidate generation, repair, and validation. As illustrated in Figure 15, the workflow combines an AI-based Cleaner and independent Reviewer with executable validation gates. Their tasks are inspired by multi-agent validation architectures (Vallabhaneni et al., 2026). In our workflow they operate as **prompt-and-rule-driven functions** under human-defined policies; no automatic rule

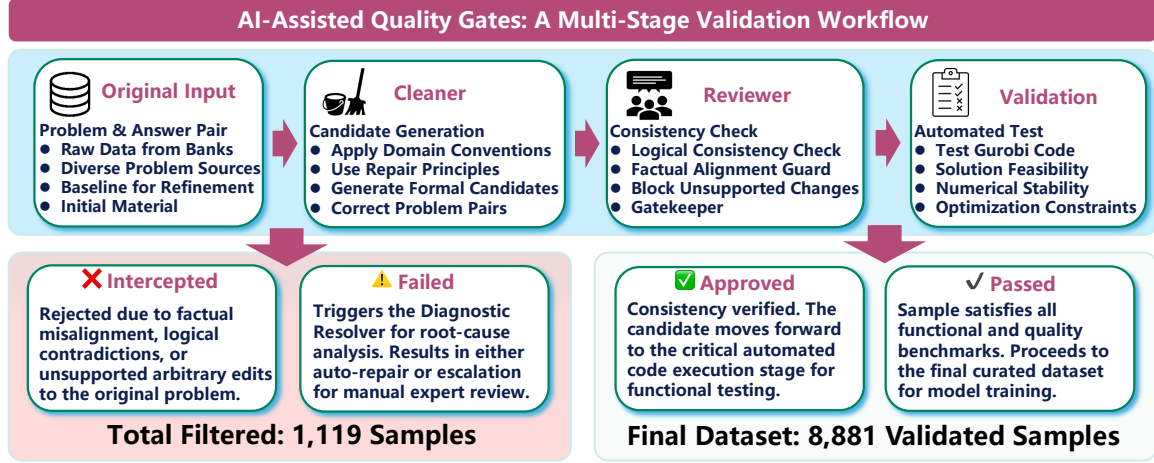


Figure 15: The AI-Assisted Quality Gates workflow. This multi-stage validation process uses the Cleaner, Reviewer, and code execution Validation to ensure the quality and auditability of the final dataset.

invention is performed by the agents to ensure reproducibility. This section describes what these tools do and how they are orchestrated in our pipeline.

Cleaner. Given an original problem and answer, the Cleaner generates a repaired candidate. Its system prompt encodes the domain conventions, target contracts, and evidence-based repair principles described in the case studies. It is proactive in fixing clear errors (e.g., variable domain mismatches, missing constraints, obvious Gurobi API misuse) but can sometimes introduce new inconsistencies.

Reviewer. The Reviewer evaluates each cleaned candidate independently, without access to the Cleaner’s reasoning. It sees only the original problem, the original answer, and the cleaned answer. Its task is to check internal consistency and factual alignment with the prompt, and to block changes that are unsupported or introduce new errors. This adversarial check is essential to prevent over-repair.

Diagnostic Resolver. When local validation (code execution against Gurobi) fails, the Resolver ingests the execution report, the problem statement, the current formulation, and the generated code. It produces a structured diagnosis attributing the failure to one or more categories: semantic modeling error, API usage error, data schema mismatch, or solver-incompatible expression. This diagnosis can trigger a second-stage repair or flag the sample for manual review.

The importance of the independent Reviewer is illustrated by an incident during development. For a transportation problem, the Cleaner accepted the sample with high confidence (`confidence = 0.95`) and reported only formatting changes. However, inspection of the cleaned output revealed that the cost from Warehouse 2 to Store 3 had been inadvertently changed from 2 to 3 in the `<model>` description, while the Python code correctly used 2. The Reviewer detected this inconsistency (`decision = needs_repair, confidence = 0.95`) with the diagnosis: `issues = ["data_inconsistency"]`. Without this gate, the conflicting sample would have entered SFT.

Across the full Clean-CoT run of 10,000 samples, the Reviewer intercepted 826 samples that the Cleaner had accepted; subsequent local validation caught an additional 293 samples. The final exported 8,881 samples are thus the intersection of three independent quality checks: Cleaner generation, Reviewer approval, and Validator execution. This multi-stage quality architecture is purely rule-driven and audit-trailed: every sample in the final SFT set carries verifiable history.

4 Experimental Results

Our evaluation covers both halves of the full-stack effort in Section 3: the operator-level optimizations that raise device efficiency, and the post-training pipeline that turns the stabilized stack into downstream capability. We begin at the operator level and validate the two kernel-optimization strategies against the same DeepSeek-V4-Pro training workload whose bottlenecks motivated them (Section 2.3.1). Section 4.1 evaluates AuraKernel on the heavy head of architecture-intrinsic kernels, where profiling-in-the-loop tuning reduces the per-invocation duration of the dominant sparse-attention and normalization kernels. Section 4.2 evaluates the proactive operator-chain fusion on the fragmented, memory-bound tail, where collapsing framework-emitted eager chains into fused AscendC kernels removes redundant launches, intermediate tensors, and global-memory traffic. Because both are measured on the profiled training workload rather than on isolated microbenchmarks, the reported speedups attach directly to the device-time regimes identified in Section 2.1.

We then evaluate the post-training pipeline on Ascend 910C using DeepSeek-V4-Flash as the experimental platform. We organize the experiments from infrastructure optimization to OR-specific model adaptation. We first verify that the training stack is stable on Ascend NPUs, retaining only the larger SFT stability run as the representative systems evidence. We then analyze the SFT scale and cleaning experiments, followed by the matched CPT→SFT transfer experiment. The final end-to-end result section is left as the summary comparison across Base, CPT, SFT, and CPT+SFT.

For clarity, the SFT labels in this section refer to two related but distinct points in the training trajectory. SFT-10K is the uncleaned 10K run used for scale analysis (Table 10) and as the pre-cleaning baseline in Table 11. Clean-CoT is the cleaned, CoT-enhanced checkpoint; the SFT row in the transfer and end-to-end tables (Tables 12 and 13) uses this matched-condition cleaned SFT checkpoint, so its 0-shot Pass@1 values align with the Clean-CoT row.

4.1 Efficient Kernels via AuraKernel

Optimization targets. AuraKernel is applied along the two axes identified in Section 2.3.1: the dominant architecture-intrinsic kernels and the reduction-heavy framework operators that dominate the long tail. Table 7 reports representative speedups on both axes, while Figure 16 and Figure 18 visualize the task-duration and pipeline-metric changes for sparse attention and the lightning indexer, respectively. To keep the numbers hardware-agnostic, we describe the gains through speedups and pipeline ratios rather than absolute latencies and analyze each axis in turn below.

Table 7: Representative operators optimized by AuraKernel on Ascend 910C, reported as single-operator task duration. The sparse-attention forward and backward and the sparse lightning-indexer gradient are per-invocation durations measured in the end-to-end training context at two compression ratios (for the indexer, cmp. ratio 0 is the full CFA path with RoPE and softmax, cmp. ratio 4 is the SCFA path); the RMS-normalization kernels are measured at the operator level ($tp=2$, input $[B:S=8192, H=64, D=512]$, BF16). Speedup is baseline/optimized.

Operator	Context	Baseline (ms)	Optimized (ms)	Speedup
SparseAttnSharedkv	cmp. ratio 128	0.88	0.72	1.23×
SparseAttnSharedkv	cmp. ratio 4	4.17	3.83	1.09×
SparseAttnSharedkvGrad	cmp. ratio 128	32.9	26.5	1.24×
SparseAttnSharedkvGrad	cmp. ratio 4	24.3	20.1	1.21×
SparseLightningIndexerGradKLLoss	cmp. ratio 0 (CFA)	20.7	17.9	1.16×
SparseLightningIndexerGradKLLoss	cmp. ratio 4 (SCFA)	7.22	5.97	1.21×
RMSNormWithoutWeightFwd	operator, $tp=2$	20.9	1.76	11.90×
RMSNormWithoutWeightBwd	operator, $tp=2$	27.7	8.09	3.42×

Top hot-spot operators. The heavy head is formed by the architecture-intrinsic kernels of the DeepSeek-V4 hierarchical sparse attention, which Section 2.3.1 identified as the largest single consumers of device time, with the shared-key-value sparse-attention backward alone at $\approx 16.33\%$ of per-step compute. Their metric-vector fingerprint is not compute-bound but movement- and scalar-bound: for SparseAttnSharedkvGrad the AIV load/store ratios (MTE2+MTE3) reach ≈ 0.559

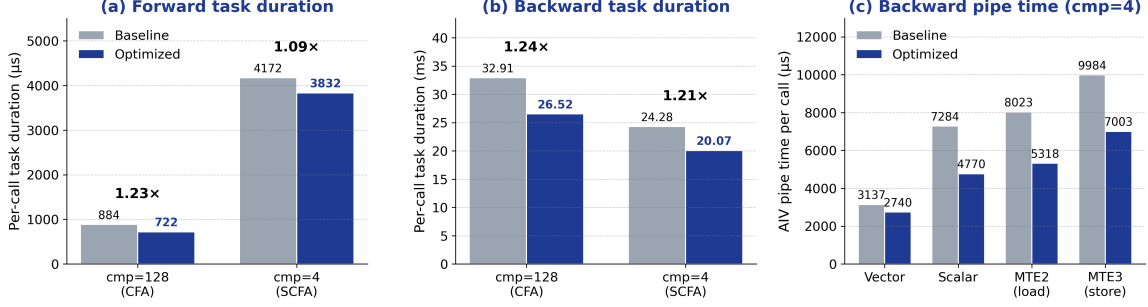


Figure 16: AuraKernel optimization of the shared-key-value sparse-attention kernels on Ascend 910C. (a) Per-call forward task duration (1.23 $\times$ at compression ratio 128, 1.09 $\times$ at compression ratio 4). (b) Per-call backward task duration of SparseAttnSharedkvGrad (1.24 $\times$ and 1.21 $\times$). (c) AIV pipeline-time decomposition of the backward kernel at compression ratio 4: the reductions concentrate in the memory-movement pipes (MTE2 load and MTE3 store) rather than in vector compute, confirming that the gain comes from cutting global-memory traffic.

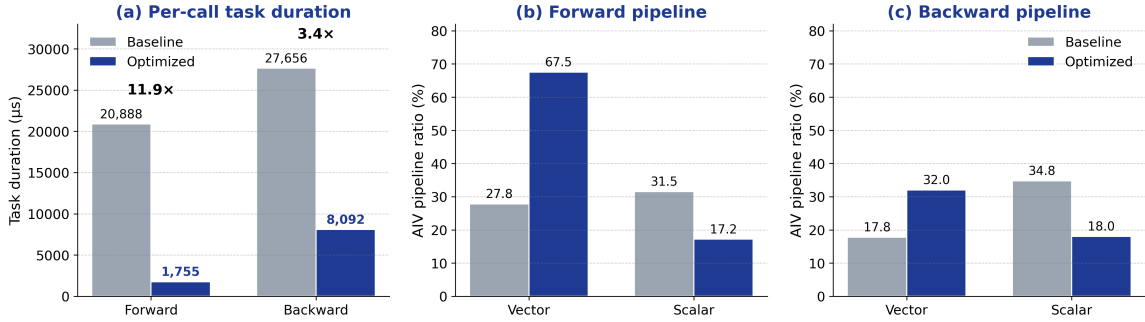


Figure 17: AuraKernel optimization of the weight-free RMS-normalization kernels ($tp=2$). (a) Per-call task duration speedup: forward 11.9 $\times$ and backward 3.4 $\times$. (b,c) Duration-weighted AIV pipeline ratios before and after optimization: the vector-compute ratio rises (forward 27.8% $\rightarrow$ 67.5%, backward 17.8% $\rightarrow$ 32.0%) while the scalar ratio falls (forward 31.5% $\rightarrow$ 17.2%, backward 34.8% $\rightarrow$ 18.0%), showing that 2D head blocking, multi-row processing, and explicit FP32 accumulation turn a scalar/address-bound reduction loop into a vector-compute-dominated kernel.

against a MAC ratio of only ≈ 0.10 , and the sparse lightning-indexer gradient is scalar- and load-bound (AIV scalar ≈ 0.81 , MTE2 load ≈ 0.66 on the CFA path), so the duration floor is set by global-memory (GM) traffic and address computation rather than arithmetic. A top- k locality analysis on real routing indices explains where this traffic originates: adjacent queries select almost identical compressed key-value blocks (adjacent-query Jaccard ≈ 0.998 at $K=1024$), yet the baseline processes one query per task and re-gathers and re-scatters the same shared blocks once per query. AuraKernel maps this to two hardware-grounded rewrites: (i) it batches consecutive queries per task and accesses the compressed side densely over the causal range instead of per-query sparse gather, cutting compressed-side GM traffic by roughly the batch factor; and (ii) once batching shifts the floor onto the cube engine, it stores the heaviest FP32 cube $\rightarrow$ vector hand-offs in BF16 to narrow the remaining GM round-trips, under an exact top- k membership mask and gradient-magnitude and loss-stability gates that keep the rewrite numerically faithful. The gain lands on the modeled bottleneck: the sparse-attention backward improves by 1.24 $\times$ and 1.21 $\times$ at compression ratios 128 and 4, its forward by 1.23 $\times$ and 1.09 $\times$, and the lightning-indexer gradient by 1.16 $\times$ on the CFA path and 1.21 $\times$ on the SCFA path (Table 7, Figures 16 and 18); in every case the per-call reduction concentrates in the MTE2/MTE3 load-store pipes, and additionally the scalar pipe for the indexer, rather than in vector compute, confirming that the improvement comes from removing GM traffic and address overhead rather than from raising raw compute utilization.

Triton-optimized operators. The long tail is carried by reduction-heavy framework operators expressed as Triton-Ascend kernels, which Section 2.3.1 flagged as launch-fragmented and memory-

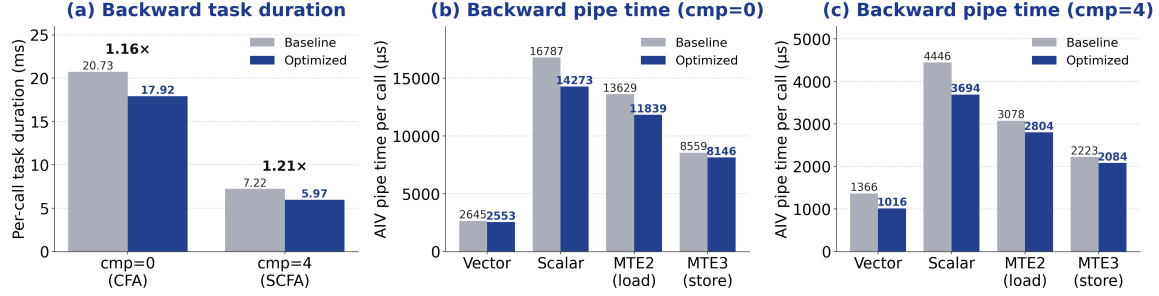


Figure 18: AuraKernel optimization of the sparse lightning-indexer gradient on Ascend 910C. (a) Per-call backward task duration speedup, 1.16× on the full CFA path (cmp. ratio 0) and 1.21× on the SCFA path (cmp. ratio 4). (b,c) AIV pipeline-time decomposition before and after optimization at the two ratios: the kernel is scalar- and load-bound, and the reductions concentrate in the scalar and MTE2-pipes rather than in vector compute, consistent with cutting address-computation and global-memory-load traffic.

and scalar-bound rather than compute-bound. Profiling localizes the cause to structural inefficiency inside these kernels: the weight-free RMS-normalization forward runs a serial per-head loop that leaves the vector engine idle behind a high scalar ratio, its backward emits poor scalar code because the reductions are not explicitly promoted to FP32, and several MHC operators accumulate through atomic-add or precede the main kernel with a standalone zero-init launch. Because these are Triton operators, AuraKernel restructures them within the Triton programming model, adjusting only host-side tiling and pipeline parameters while preserving the kernel mathematics: 2D head blocking and multi-row processing to replace the serial head loop, explicit FP32 accumulation and head-dimension unrolling, three-phase load-compute-store software pipelining to overlap the MTE2, vector, and MTE3 pipes, and elimination of atomic-add and standalone zero-init launches. A single campaign drives all 14 Triton-Ascend operators (the RMS-normalization pair and the DeepSeek-V4 MHC family) to 100% completion at a 2.06× average and an 11.90× best speedup, the backward RMS-normalization improving by 3.42× (Table 7, Figure 17). The gain shows up as a pipeline shift: the AIV vector-compute ratio rises sharply while the scalar ratio falls (RMS-normalization forward 27.8% → 67.5% vector and 31.5% → 17.2% scalar; backward 17.8% → 32.0% vector and 34.8% → ≈ 18% scalar), turning scalar- and address-bound reduction loops into vector-compute-dominated kernels. The largest wins concentrate on reduction-class operators with structural inefficiency, whereas operators already at the bandwidth limit gain only a few percent.

4.2 Performance Improvements from Kernel Fusion

We next evaluate the operator fusion and rewrite path in Section 2.3.3. The measurement unit is different from the AuraKernel results above. Whereas AuraKernel improves kernels that already appear as large execution units in the trace, this path changes the unit of execution itself. The baseline is therefore the complete eager chain emitted by Python, torch, and autograd for a model-level operation, or the generic Triton kernel that a stage was previously lowered into, not a single primitive operator selected from that chain. This distinction is important for interpreting the result: a fused or rewritten implementation may preserve the same mathematical work while removing the launches, intermediate tensors, dtype round trips, and scatter-style writebacks that surrounded it in the original trace, and while relocating the remaining arithmetic onto a substrate with explicit on-chip residency and pipeline scheduling. We present the targets in the order of Section 2.3.3: the mHC suite first, then limited SwiGLU and unified RoPE.

The mHC results exercise the two entries of this path together, so Table 8 reports each stage separately. Pre-projection absorbs the RMS-normalization fragments and the original pre-BMM kernel; its backward absorbs the pre-BMM backward path, RMS backward fragments, casts, and element-wise reconstruction; and post-projection replaces the add/post-BMM1/cast portion. The three stages reach 3.03×, 1.88×, and 2.74× at the module level (Table 8). This attribution is corroborated independently of the record counts: summing the inserted mHC fused kernels directly from the post-optimization trace reproduces the same stage totals, with WindMhcPostPart the single largest contributor at 0.313 s across all matched calls. The post-projection backward is left on its original

Table 8: Operator-chain fusion results from the pre-optimization (fusion off) and post-optimization (fusion on) profiling traces, measured at global batch size 1024. Operator counts are trace records attributed to the target module. “Records” gives the module-level count before and after fusion and its net change Δ . “Fused calls” is the count of the inserted AscendC kernel, cross-checked against the post-optimization trace by matching operator name and type. “Module time” is the summed device task duration of the module before and after fusion, under the same submodule attribution. “Per call” divides each module time by the number of semantic invocations, so it reports how heavy one invocation of the eager chain is against its fused replacement. The speedup is identical whether read at the module or per-call level, because both share the same invocation count.

Optimization target	Fused kernel	Records (Δ)	Fused calls	Module time (before $\rightarrow$ after)	Per call (μ s)
mHC pre-projection	WindHcPreBmmForward	14336 $\rightarrow$ 8064 (-6272)	896	1.287 $\rightarrow$ 0.425 s 3.03 $\times$	1437 $\rightarrow$ 474
mHC pre-projection backward	WindHcPreBmmBackward	21504 $\rightarrow$ 17920 (-3584)	448	1.913 $\rightarrow$ 1.019 s 1.88 $\times$	4270 $\rightarrow$ 2274
mHC post-projection	WindMhcPostPart	7168 $\rightarrow$ 3584 (-3584)	896	1.710 $\rightarrow$ 0.624 s 2.74 $\times$	1908 $\rightarrow$ 697
Limited SwiGLU forward	SwiGluLimitV2	5376 $\rightarrow$ 896 (-4480)	896	1.480 $\rightarrow$ 0.167 s 8.86 $\times$	1652 $\rightarrow$ 186
Limited SwiGLU backward	SwiGluLimitBackwardV2	5824 $\rightarrow$ 448 (-5376)	448	0.805 $\rightarrow$ 0.246 s 3.27 $\times$	1797 $\rightarrow$ 549
RoPE forward sites	WindScaleRope	14208 $\rightarrow$ 2176 (-12032)	2176	1.421 $\rightarrow$ 0.611 s 2.33 $\times$	653 $\rightarrow$ 281
RoPE backward sites	WindScaleRopeGrad	15744 $\rightarrow$ 2176 (-13568)	1088	1.599 $\rightarrow$ 0.451 s 3.54 $\times$	1470 $\rightarrow$ 415

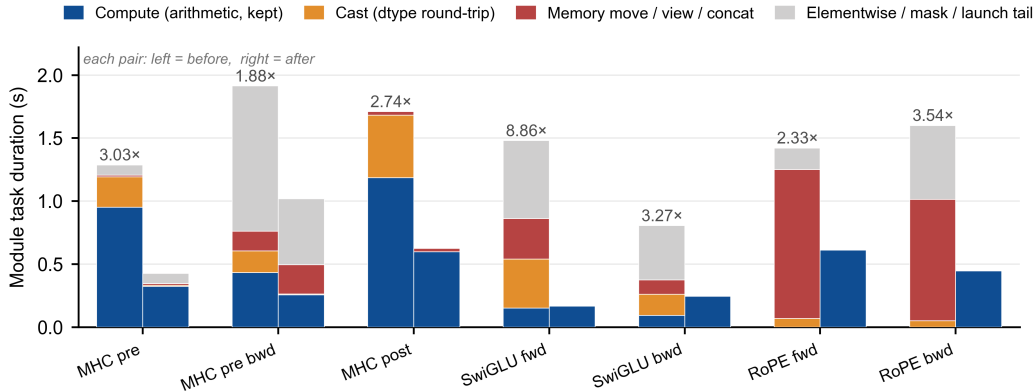


Figure 19: Module task duration before (left bar) and after (right bar) operator fusion and rewrite for the seven targets, decomposed by operator category. Category durations are re-aggregated from the per-operator trace times of the pre- and post-optimization profiles. Each bar pair represents kernel-module durations (Table 8), with the module speedup annotated above. Across targets, the baseline implementations are dominated by memory movement, dtype casts, and small elementwise/mask fragments, whereas the proposed fusion retain achieves significant efficiency improvements.

path in this profile and is therefore not reported as a fused target, and folding it into the same boundary is left to future work.

Four of these kernels—the weight-free RMS normalization forward and backward and the pre-BMM contraction forward and backward—reach the AscendC substrate through the Triton rewrite rather than through eager-chain fusion, and comparing the original Triton kernel against its AscendC replacement at the single-operator level isolates what the substrate change alone contributes. Table 9 reports this comparison. The RMS-normalization pair, measured at an identical BF16 shape, drops from 2307 μ s to 309 μ s forward (7.48 $\times$) and from 3318 μ s to 439 μ s backward (7.55 $\times$). Much of this comes from the tiling change, as the generic Triton kernel launches across 4096 blocks whereas

the AscendC kernel processes multiple rows per core across 48 blocks. These ratios measure the Triton-to-AscendC rewrite against the original Triton kernel, and are therefore distinct from the AuraKernel figures in Section 4.1 ($11.90\times/3.42\times$), which tune the resulting AscendC kernel further at a different shape. The pre-BMM contraction, whose rewrite also folds in the mixed-precision contract (FLOAT inputs become BF16), improves by $2.10\times$ forward and $1.89\times$ backward. The AIV pipeline decomposition confirms the affinity mechanism of Section 2.3.3 rather than a mere clock reduction: on the pre-BMM forward the vector-compute ratio rises from 0.17 to 0.86 while the MTE2 load ratio falls from 0.59 to 0.35, so the BrCb broadcast and UB-resident contraction turn a movement-bound generic kernel into a vector-compute-dominated one. The same shift, in weaker form, appears on the backward kernels (pre-BMM backward vector ratio $0.14 \rightarrow 0.47$, RMS backward $0.28 \rightarrow 0.36$). Beyond these single-operator gains, the rewrite carries a stage-wide precision benefit that a generic Triton lowering left on the table: making the mixed-precision contract explicit removes a full-stage FP32 up-cast across the mHC pre-projection calls. Measured on the post-optimization trace, the Cast kernels of the stage fall from 241 ms to 8 ms in the forward path and from 172 ms to 6 ms in the backward path—about 400 ms of cast traffic removed across the combined stage, whose pre-optimization cost is 1.29 s forward plus 1.91 s backward—with a further 157 ms recovered from a backward Mul that the promotion had widened to FP32.

Table 9: Triton-to-AscendC operator rewrite for the four mHC pre-projection kernels, measured as single-operator task duration at the profiled training shape. The RMS-normalization pair is compared at an identical BF16 input, while the pre-BMM pair additionally narrows its I/O dtype from FLOAT to BF16 as part of the rewrite. “Vec. ratio” is the duration-weighted AIV vector-compute ratio, whose rise on the AscendC side indicates a shift from movement-bound to compute-bound execution. Speedup is Triton/AscendC.

Kernel	Task duration (μ s)		Speedup	Vec. ratio	
	Triton	AscendC		Triton	AscendC
RMS-norm forward	2307	309	$7.48\times$	0.30	0.32
RMS-norm backward	3318	439	$7.55\times$	0.28	0.36
Pre-BMM forward	160	76	$2.10\times$	0.17	0.86
Pre-BMM backward	510	270	$1.89\times$	0.14	0.47

The remaining targets isolate the boundary-fusion effect, where autograd-visible tensor boundaries disappear. Limited SwiGLU forward collapses its eager clamp-and-gate chain into a single SwiGLULimitV2 call, cutting the module from 1.480 s to 0.167 s ($8.86\times$), because the eager forward materialized clamp constants and a concatenation that the fused kernel keeps on chip. The backward path is more launch-dense still—fills, boolean materialization, selections, casts, SwiGLUGrad, and concatenation—and there the fused SwiGLULimitBackwardV2 operator removes almost the entire chain (Figure 10). In the MoE path the same kernel also absorbs the routing-weight gradient, so the eager reduction that formed it (a full-size $\ell \cdot h$ product of 61 ms plus a ReduceSum of 20 ms per step, about 82 ms together) disappears into the fused pass at no added kernel cost. Counting the routing-gradient chain as part of the backward, the module falls from 805 ms to 246 ms ($3.27\times$). RoPE retains a larger optimized residual because the rewritten path still contains shape and indexing support operators, but the count accounting in Table 8 exposes the same mechanism, yielding $2.33\times$ (forward) and $3.54\times$ (backward). The backward gain is the larger of the two because the fused kernel removes the eager slice-assignment scatter chain at the graph level. These module-level figures aggregate the query, key-value, and output sites.

These results close the loop with the bottleneck anatomy in Section 2.3.1. The gains are not explained by faster arithmetic alone, as Figure 19 makes explicit: they come from changing which tensors are materialized, which boundaries are visible to autograd, which dtype crosses global memory, and—for the Triton-originated stages—which substrate the arithmetic runs on. This is why the strongest effects appear on chains dominated by movement, conversion, and writeback rather than on compute-dense cube kernels, why the resource-utilization shift on the rewritten kernels turns scalar- and movement-bound loops into vector-compute-dominated ones, and why the appropriate baseline is the emitted trace slice, stage subgraph, or generic Triton kernel rather than an isolated primitive operator.

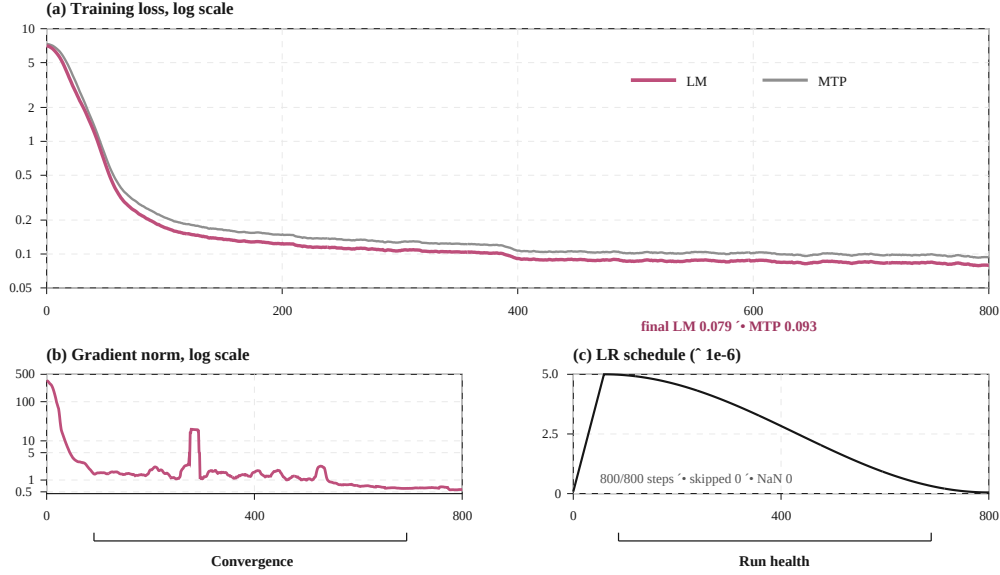


Figure 20: Training stability of the 50K self-distilled SFT run on Ascend 910C NPUs. The model is trained for 800 steps with GBS=128, MBS=1, and SEQ_LEN=8192, and the training process reaches the final iteration without skipped or NaN iterations.

4.3 Long-term Training Stability with Ascend SuperPOD

Stable training is a prerequisite for meaningful downstream evaluation. If the training process is unstable, subsequent benchmark differences become difficult to interpret. In this work, we implement stable SFT training of DeepSeek-V4-Flash in a Ascend-based environment with 8 nodes and 128 Ascend 910C NPUs in total.

Using the 50K self-distilled dataset as an example, we complete a relatively full-scale SFT verification run on DeepSeek-V4-Flash. The training configuration uses GBS=128, MBS=1, and SEQ_LEN=8192. The model is trained for 800 steps, and checkpoints are saved at 200, 400, 600, and 800 steps. The training log shows that the job successfully reaches iteration 800/800, without skipped iterations or NaN iterations. At the final iteration, the `lm_loss` converges to approximately 0.079, the `mtp_loss` is approximately 0.093, and the gradient norm remains within a normal range. The overall training curve decreases continuously and then gradually becomes stable.

As shown in Figure 20, these observations indicate that the current data preprocessing pipeline, MindSpeed-LLM (Huawei Ascend, 2024) training script, DeepSeek-V4-Flash base model, and 8-node 910C training environment form a reproducible and sustainable full-parameter SFT workflow. This provides a stable foundation for subsequent experiments on different data scales, data mixtures, and task-oriented data-distillation strategies.

4.4 SFT Scaling and Cleaning Analysis

The first systematic experiments on 3K, 10K, and 50K self-distilled data show that SFT brings two opposite effects. On the one hand, SFT helps the model learn code-protocol tasks such as B4O-Feasible. Using DeepSeek-V4-Flash as the baseline, we find that B4O-Feasible improves from 60.47 to 65.07 with SFT-10K, and B4O-ORGEval improves from 34.26 to 47.21. These gains indicate that SFT makes the model more familiar with code-only output, `data.json` loading, Gurobi model construction, LP writing paths, and other benchmark-specific protocols, with the strongest relative improvement concentrated on structural-equivalence evaluation.

On the other hand, SFT also weakens part of the model’s original ability on natural-language modeling tasks. The baseline obtains 84.08 on NL4OPT, while SFT-10K obtains 81.66. Similarly, the baseline obtains 63.33 on OptiBench, while SFT-10K obtains 62.67. Further scaling to 50K does

Table 10: Systematic comparison of SFT data scales on OR modeling benchmarks.

Model	NL4OPT	OptiBench	B4O-Feasible	B4O-ORGEval
DeepSeek-V4-Flash	84.08	63.33	60.47	34.26
SFT-3K	80.62	60.66	64.51	43.65
SFT-10K	81.66	62.67	65.07	47.21
SFT-50K	82.01	58.68	64.97	45.94

Table 11: Effect of data cleaning and CoT enhancement on OR modeling benchmarks.

Model	NL4OPT	OptiBench	B4O-Feasible	B4O-ORGEval
DeepSeek-V4-Flash	84.08	63.33	60.47	34.26
SFT-10K	81.66	62.67	65.07	47.21
SFT-Clean-only	84.23	63.08	64.31	48.28
SFT-Clean-CoT	86.93	64.17	65.93	48.73

not solve this issue; instead, OptiBench decreases to 58.68. Since the 3K, 10K, and 50K settings follow almost the same construction design, the degradation of 50K is difficult to explain as a data-mixture change. A more plausible explanation is that continuous iterative distillation introduces lower-quality samples and amplifies the model’s own noise.

Table 10 supports two observations. First, supervised fine-tuning provides a clear benefit for protocol-oriented OR modeling tasks: all SFT variants improve B4O-Feasible and, more prominently, B4O-ORGEval relative to the base model, indicating that the self-distilled data contains effective supervision for solver-facing output formats and benchmark-specific conventions. Second, the gains are not monotonic with data scale. If additional self-distilled samples were uniformly beneficial, SFT-50K would be expected to dominate SFT-10K, and the uncleaned SFT-10K checkpoint would not fall below the base model on NL4OPT and OptiBench. The observed regressions suggest that scaling the same self-distillation pipeline also increases the weight of weakly verified or biased samples. Such samples can reinforce useful modeling patterns, but they can also amplify systematic errors inherited from the base model. Therefore, the 50K result should be interpreted less as evidence against SFT scale itself and more as evidence that scale must be coupled with stronger verification and data-quality control. This motivates returning to the 10K setting for fine-grained cleaning rather than further expanding the unfiltered pipeline to 100K.

The SFT results after cleaning validate the effectiveness of the contract-aware cleaning and CoT enhancement framework introduced in Section 3.3.3. Compared with the uncleaned SFT-10K, Clean-CoT improves NL4OPT by 5.27 percentage points, OptiBench by 1.50 percentage points, B4O-Feasible by 0.86 percentage points, and B4O-ORGEval by 1.52 percentage points. This result suggests that the main issue of the 10K data is not insufficient scale, but semantic noise that is strong enough to damage natural-language modeling ability. Once such noise is systematically filtered and repaired, SFT no longer harms NL4OPT and OptiBench, but instead outperforms both the baseline and the original SFT-10K while further improving both B4O metrics.

However, the gains of Clean-CoT are not uniformly distributed across all tasks. It is most effective on NL4OPT and OptiBench because these targets allow `<think>`, `<model>`, and `<python>` style answers. The structured modeling checklist can directly enter the training response and help the model learn explicit judgments about variable domains, objective direction, and constraint direction. By contrast, B4O-Feasible and B4O-ORGEval are code-only tasks, where explanatory chain-of-thought cannot appear in the output. Therefore, the advantage of Clean-CoT is less directly expressed in the response format. In the updated results, Clean-CoT also outperforms clean-only on B4O-Feasible (65.93 vs. 64.31) and B4O-ORGEval (48.73 vs. 48.28), but the margins are smaller than on NL4OPT. This suggests that CoT-oriented enhancement is beneficial for code-only tasks, yet these tasks still require more specialized training on schema reading, index alignment, variable-family design, `model.write()` paths, and LP-structure consistency.

Error analysis further reveals the boundary of cleaning-based enhancement. On NL4OPT, Clean-CoT reduces `wrong_objective_or_model` errors from 52 cases before cleaning to 31 cases. On

Table 12: CPT-to-SFT transfer results on four OR modeling benchmarks.

Model	NL4OPT	OptiBench	B4O-Feasible	B4O-ORGEval
DeepSeek-V4-Flash	84.08	63.33	60.47	34.26
SFT-Clean-CoT	86.93	64.17	65.93	48.73
CPT+SFT-Clean-CoT	89.52	67.12	71.22	59.39

OptiBench, the same error type decreases from 154 to 128 cases, indicating that modeling checklists indeed reduce coarse errors at the objective and constraint levels. However, `nonlinear_or_bad_gurobi_form` errors on OptiBench do not decrease; instead, they increase from 49 to 56 cases. This shows that ordinary cleaning and short chain-of-thought enhancement can improve the question of what model should be built, but are insufficient to stably solve the question of how to correctly express nonlinear terms, ratios, averages, and quadratic terms in Gurobi. This observation motivates the subsequent design of nonlinear repair and specialized nonlinear/Gurobi synthetic data, rather than forcing all issues into a single Clean-CoT path.

Therefore, the main conclusion of this SFT experiment is that chain-of-thought-oriented data enhancement is effective, but it must be constrained, contract-aware, and gated by reviewer and validation checks. Its value is not to make the model generate longer reasoning, but to inject the most error-prone optimization-modeling checkpoints into the training distribution in a stable form.

4.5 Effect of CPT Initialization under Matched SFT Conditions

Following the evaluation protocol in Section 3.2.3, we evaluate whether the CPT checkpoint improves downstream SFT performance under matched SFT conditions. Both the direct SFT route and the CPT-initialized SFT route use the same 10K chain-of-thought OR modeling samples, 220 SFT iterations, peak learning rate 5×10^{-6} with cosine decay, global batch size 128, and sequence length 8192. Therefore, performance differences between the two routes can be attributed to CPT initialization rather than different SFT data or optimization settings.

We report results on four OR benchmarks. NL4OPT and OptiBench evaluate natural-language optimization modeling, B4O-Feasible evaluates whether the generated Gurobi program executes successfully and produces a feasible solution, and B4O-ORGEval evaluates whether the generated optimization model is structurally equivalent to the reference model, independent from implementations. Table 12 shows that SFT alone is already a strong baseline. Compared with the base model, direct SFT improves NL4OPT from 84.08% to 86.93%, OptiBench from 63.33% to 64.17%, B4O-Feasible from 60.47% to 65.93%, and B4O-ORGEval from 34.26% to 48.73%. This confirms that the 10K CoT OR modeling samples provide effective task-level supervision for activating optimization modeling capabilities.

CPT initialization provides further gains on top of this SFT baseline across all four benchmarks: +2.59 pp on NL4OPT, +2.95 pp on OptiBench, +5.29 pp on B4O-Feasible, and +10.66 pp on B4O-ORGEval. The gain is present on both executability and structure but is largest on B4O-ORGEval, indicating that CPT supplies OR-domain modeling priors that improve solver-facing feasibility and, most strongly, structural equivalence—capabilities that matched-condition SFT alone does not fully activate.

Since B4O-ORGEval emphasizes mathematical equivalence between the generated and reference optimization models, it is more directly aligned with the domain knowledge and modeling patterns introduced during CPT.

Overall, these results support the value of domain-adaptive CPT as a preparation stage for OR-oriented SFT. While direct SFT already improves task performance, CPT further strengthens the structurally demanding capabilities required for optimization model equivalence. This motivates scaling CPT with broader OR task coverage, higher-quality solver-verified documents, and retention-aware data mixtures in future experiments.

Table 13: Performance of Base, SFT, and CPT+SFT models on operations research benchmarks. The SFT model and the SFT stage of CPT+SFT both use the Clean-CoT dataset. All numbers are percentages. The best result in each column is in **bold**, and the second best is underlined. Gains are absolute differences (percentage points) compared to the Base model.

Dataset	Model	0-shot Pass@1		0-shot Pass@4		5-shot Pass@1	
		Value	Gain	Value	Gain	Value	Gain
NL4OPT	Base	84.08	—	91.35	—	88.24	—
	SFT	<u>86.93</u>	+2.85	<u>92.39</u>	+1.04	<u>88.97</u>	+0.73
	CPT+SFT	89.52	+5.44	93.15	+1.80	90.66	+2.42
OptiBench	Base	63.33	—	<u>72.50</u>	—	<u>66.00</u>	—
	SFT	<u>64.17</u>	+0.84	71.50	-1.00	<u>65.67</u>	-0.33
	CPT+SFT	67.12	+3.79	72.83	+0.33	66.89	+0.89
B4O-Feasible	Base	60.47	—	66.86	—	78.20	—
	SFT	<u>65.93</u>	+5.46	<u>68.69</u>	+1.83	73.55	-4.65
	CPT+SFT	71.22	+10.75	82.41	+15.55	<u>74.78</u>	-3.42
B4O-ORGEval	Base	34.26	—	49.49	—	71.57	—
	SFT	<u>48.73</u>	+14.47	<u>56.35</u>	+6.86	<u>82.99</u>	+11.42
	CPT+SFT	59.39	+25.13	67.28	+17.79	84.52	+12.95

4.6 Performance of the Proposed Post-Training Pipeline

Evaluation Setup. We evaluate on four OR benchmarks—NL4OPT (Ramamonjison et al., 2022) and OptiBench (Yang et al., 2025c) for natural-language optimization modeling and problem understanding, B4O-Feasible for executability, and B4O-ORGEval for structural equivalence (Wang et al., 2025b)—alongside general benchmarks MMLU (Hendrycks et al., 2021a), MMLU-Pro (Wang et al., 2024), CMMLU (Hendrycks et al., 2021a), HumanEval (Chen et al., 2021), GSM8K (Cobbe et al., 2021), and MATH (Hendrycks et al., 2021b) (Table 14). For decoding, we use greedy for Pass@1 and sample at $T = 0.8$ with top- $p = 0.95$ for Pass@4, applying unbiased estimation (Zhao et al., 2023). Correctness for B4O-Feasible requires runtime execution and contract adherence; B4O-ORGEval uses the ORGEval WL reward on the normalized formulation graph to measure semantic structural match. All evaluations share the same prompt template and decoding budget for fair comparison.

While SFT improves extraction tasks, structural equivalence, and zero-shot B4O-Feasible performance, it does not consistently enhance executability across all decoding settings; the 5-shot B4O-Feasible result remains below the base model. The full CPT+SFT pipeline consistently improves 0-shot Pass@1 across the OR benchmarks and yields its strongest gains on B4O-ORGEval (structural equivalence) and B4O-Feasible (executability), but it should not be interpreted as dominating every decoding regime. This indicates that CPT supplies domain knowledge critical for both deep structural reasoning and the generation of executable code, while SFT aligns outputs with format and protocol requirements.

To evaluate the complementary roles of knowledge injection and instruction tuning, we conduct end-to-end experiments comparing Base, SFT, and CPT+SFT under identical SFT conditions. Our central hypothesis is that CPT supplies domain-specific structural priors, while SFT grounds execution protocols and format compliance. The results in Table 13 confirm that the full pipeline (CPT+SFT) consistently outperforms both Base and SFT across all OR benchmarks in 0-shot Pass@1. The gain is largest on B4O-ORGEval (e.g., +10.66 pp in 0-shot Pass@1 and +7.71 pp in the three-setting mean over SFT) and is also substantial on B4O-Feasible (+5.29 pp in 0-shot Pass@1 and +6.75 pp in the three-setting mean over SFT). SFT alone improves B4O-Feasible in zero-shot settings but drops below Base in 5-shot, indicating that instruction tuning by itself is insufficient to stabilize executability across prompting regimes.

Disaggregating by task type reveals the distinct contributions of each stage. For structural equivalence tasks (B4O-ORGEval), SFT alone improves moderately over the base model, but the additional CPT prefix yields a substantial leap. This suggests that optimization modeling judgments—which require understanding mathematical intent independent of surface form—benefit from dense exposure to OR corpora during CPT, an exposure that SFT cannot fully replicate due to its instruction-focused, smaller-scale format. For code-execution and feasibility tasks (B4O-Feasible), SFT improves 0-shot

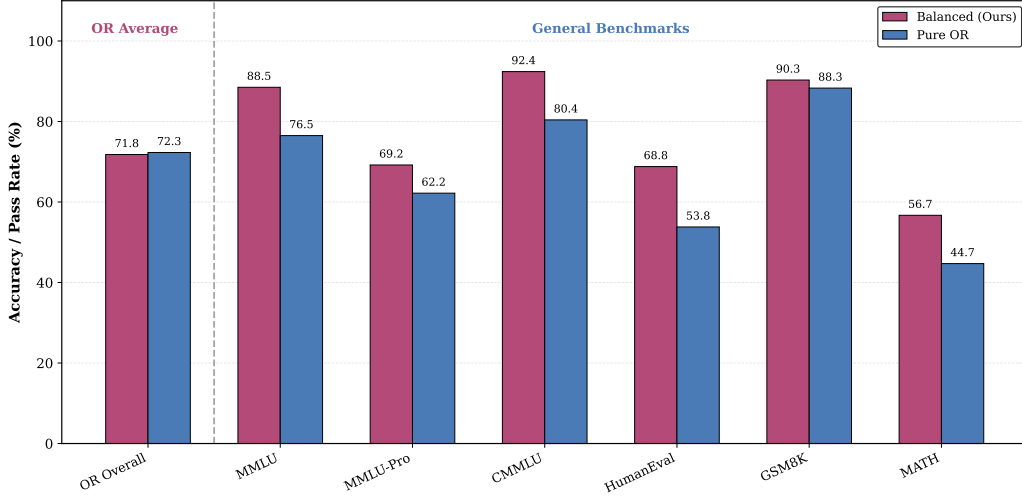


Figure 21: Ablation on data mixture under the end-to-end (CPT→SFT) pipeline.

Table 14: Performance comparison between Ours and Base on general benchmarks.

Category	Benchmark	# Shots	Model	
			SLAI T-Rex(Ours)	Base
World Knowledge	MMLU (Hendrycks et al., 2021a)	5-shot	88.5 (+0.9)	87.6
	MMLU-Pro (Wang et al., 2024)	5-shot	69.2 (-1.8)	71.0
	CMMLU (Hendrycks et al., 2021a)	5-shot	92.4 (+0.3)	92.1
Code & Math	HumanEval (Chen et al., 2021)	0-shot	68.8 (-0.6)	69.4
	GSM8K (Cobbe et al., 2021)	8-shot	90.3 (+0.5)	89.8
	MATH (Hendrycks et al., 2021b)	4-shot	56.7 (-1.7)	58.4

Table 15: Performance of various models on operations research benchmarks. The best result in each column is in **bold**, and the second best is underlined.

Model	Operations Research Benchmarks				Overall
	NL4OPT	OptiBench	B4O-Feasible	B4O-ORGEval	
GLM-5.2 (GLM-5-Team et al., 2026)	78.55	55.33	61.05	31.47	56.60
DeepSeek-V4-Flash (DeepSeek-AI et al., 2026)	84.08	63.33	60.47	34.26	60.54
MiMo-V2.5-Pro (Xiaomi MiMo Team, 2026)	80.97	56.83	70.35	41.88	62.51
Kimi-K2.6 (Kimi Team et al., 2026)	78.89	59.83	<u>72.67</u>	<u>46.95</u>	64.59
Gemini-3-Flash (Google Cloud, 2026)	79.93	68.83	<u>71.80</u>	<u>46.70</u>	66.82
GPT-5.4-Mini (OpenAI, 2026)	<u>87.89</u>	66.17	73.84	43.40	<u>67.83</u>
SLAI T-Rex (Ours)	89.52	<u>67.12</u>	71.22	59.39	71.81

Pass@1 and Pass@4 but remains below Base in 5-shot, yielding only a modest three-setting mean gain (69.39 vs. 68.51), whereas CPT+SFT reaches 76.14. This suggests that domain-specific pretraining is crucial for generating executable optimization code, beyond what format and protocol alignment alone can provide. On general benchmarks (Table 14), CPT+SFT achieves performance comparable to Base, with slight fluctuations in both directions (e.g., +0.9 on MMLU (Hendrycks et al., 2021a), -1.7 on MATH (Hendrycks et al., 2021b)), and no signs of catastrophic forgetting. This confirms that the staged data mixture (~75% OR + retained general corpora) preserves broad reasoning capabilities. As shown in Table 15, our CPT+SFT model attains the highest overall score across all four OR benchmarks, outperforming strong baselines such as GPT-5.4-Mini (OpenAI, 2026) and Gemini-3-Flash (Google Cloud, 2026) by a substantial margin.

To dissect the sources of CPT’s effectiveness, we ablate the CPT data mixture under the same end-to-end pipeline (Figure 21). We compare two configurations: Balanced (our mixture of OR and general data) and Pure OR (only OR data). As shown, Pure OR causes a drastic drop (10–15 points) on general

benchmarks while yielding no improvement on OR Overall compared to Balanced—in fact, it slightly underperforms Balanced. This indicates that exclusive domain pretraining leads to overfitting that harms general capabilities without benefiting domain performance. In contrast, Balanced achieves strong results on both OR and general benchmarks, demonstrating that interleaving domain-specific and general data during CPT is essential to inject structural priors while retaining broad reasoning abilities and guarding against catastrophic forgetting.

In summary, the end-to-end experiments show a complementary pattern between SFT and CPT. SFT establishes supervised task alignment and protocol-oriented output behavior, while CPT supplies domain-structural priors that drive the largest additional gains, most clearly on B4O-ORGEval and also on B4O-Feasible. Together, the two stages form a complementary pipeline for improving OR-domain performance while preserving general capability.

5 Conclusion, Limitations, and Future Directions

This technical report presents a first-stage practice for full-parameter post-training of the DeepSeek-V4 model family on Ascend NPU SuperPOD. We refer to the overall framework as **SLAI T-Rex**, which couples full-stack Ascend SuperPOD training optimization with solver-grounded CPT-SFT specialization for OR.

At the system level, we demonstrate one of the first full-stack optimization efforts for trillion-parameter MoE LLM training on a non-GPU computing platform. The proposed end-to-end optimization strategy spans system-level training orchestration and bottleneck-level kernel optimization, achieving up to 34.22% MFU. These results validate both the training efficiency and practical feasibility of large-scale post-training on Ascend NPU SuperPOD.

For post-training tasks, we establish an OR-oriented CPT-SFT workflow on DeepSeek-V4-Flash. Under identical SFT settings, direct SFT improves B4O-Feasible from 60.47% to 65.93% and B4O-ORGEval from 34.26% to 48.73%. With CPT initialization, the corresponding scores further increase to 71.22% and 59.39%, respectively. The full benchmark results show consistent trends across NL4OPT and OptiBench. These results suggest that direct SFT provides supervised task alignment, while CPT initialization introduces additional OR-domain modeling priors that improve both solver-facing feasibility and structural equivalence. The most significant improvement is observed on B4O-ORGEval, highlighting the value of domain-adaptive pre-training for complex OR modeling.

The SFT experiments further highlight the importance of data quality. Scaling self-distilled data from 10K to 50K does not lead to monotonic improvements and may even degrade performance on natural-language modeling benchmarks. In contrast, contract-aware cleaning and concise CoT-oriented enhancement improve the 10K data more reliably, especially on NL4OPT, OptiBench, and ORGEval. This indicates that OR-domain SFT benefits more from targeted repair of modeling errors, strict output-contract enforcement, compact reasoning checklists, and solver-grounded verification than from simply increasing the volume of synthetic training data. Admittedly, the current post-training experiments are conducted on DeepSeek-V4-Flash. In future work, we will extend the same training pipeline to DeepSeek-V4-Pro and release the corresponding training results to further validate the scalability of the proposed infrastructure and post-training workflow.

Reproducibility and artifact release. To support full reproducibility and facilitate follow-up research, we release the key artifacts underlying the proposed optimization and training workflow, including prompt templates, format-contract specifications, curriculum scheduling configurations, and monitoring callbacks. The AI-assisted data and evaluation tools used in our pipeline, including Cleaner, Reviewer, and Diagnostic Resolver, are implemented as configurable prompt- and rule-driven modules. Their templates and configuration files also are released, allowing future users to inspect, reproduce, adapt, and extend the data-cleaning, sample-review, diagnostic-resolution, and monitoring procedures introduced in this work.

In addition, we plan to separately release the optimized AscendC kernels developed through our kernel-optimization pipeline. This release will provide concrete kernel-level implementations and optimization examples, enabling future work to reproduce the reported operator-level improvements and further adapt the optimized kernels to related Ascend-based training workloads.

Future directions. Building on the proposed high-performance training infrastructure for Ascend SuperPOD, future work will further improve the efficiency of end-to-end full-parameter training while extending the post-training workflow toward agentic reinforcement learning (AgenticRL). In particular, we will investigate operations-research-based mathematical modeling for training parallelism optimization, aiming to automate the exploration of multi-dimensional parallel configurations under memory, communication, and orchestration constraints. In parallel, we will continue scaling OR-oriented continued pre-training (CPT) and supervised fine-tuning (SFT) to broader solver-verified corpora, extend evaluation to more diverse OR problem families and general-capability benchmarks, and strengthen structural verification for nonlinear expressions, ratio constraints, quadratic terms, and Gurobi-specific modeling patterns.

Overall, this report demonstrates Ascend NPU SuperPOD can support stable full-parameter post-training of trillion-scale models with MoE and hybrid attention architectures. Furthermore, the proposed CPT-SFT workflow can produce measurable gains in OR-oriented mathematical modeling. The key finding is that effective vertical domain adaptation depends on the alignment among domain knowledge acquisition, solver-grounded data construction, supervised data quality, system stability, and task-specific evaluation.

References

- Ali Ahmaditeshnizi, Wenzhi Gao, and Madeleine Udell. OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 577–596. PMLR, 2024. URL <https://proceedings.mlr.press/v235/ahmaditeshnizi24a.html>.
- Shiyi Cao, Ziming Mao, Joseph E Gonzalez, and Ion Stoica. K-search: Llm kernel generation via co-evolving intrinsic world model. *arXiv preprint arXiv:2602.19128*, 2026a.
- Xinzi Cao, Jianyang Zhai, Pengfei Li, Zhiheng Hu, Cen Yan, Bingxu Mu, Guanghuan Fang, Bin She, Jiayu Li, Yihan Su, Dongyang Tao, Xiansong Huang, Fan Xu, Feidiao Yang, Yao Lu, Chang-Dong Wang, Yutong Lu, Weicheng Xue, Bin Zhou, and Yonghong Tian. AscendKernelGen: A systematic study of llm-based kernel generation for neural processing units, 2026b. URL <https://arxiv.org/abs/2601.07160>.
- Hongyang Chen, Zhongwu Sun, Hongfei Ye, Kunchi Li, and Xuemin Lin. Continual learning in large language models: Methods, challenges, and opportunities. *CoRR*, abs/2603.12658, 2026a. URL <https://arxiv.org/abs/2603.12658>.
- Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. Evaluating large language models trained on code. 2021.
- Terry Chen, Zhifan Ye, Bing Xu, Zihao Ye, Timmy Liu, Ali Hassani, Tianqi Chen, Andrew Kerr, Haicheng Wu, Yang Xu, et al. Avo: agentic variation operators for autonomous evolutionary search. *arXiv preprint arXiv:2603.24517*, 2026b.
- Zhuoen Chen, Dongfang Li, Meishan Zhang, Baotian Hu, and Min Zhang. Dynamic long context reasoning over compressed memory via end-to-end reinforcement learning. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 8064–8083, San Diego, California, United States, July 2026c. Association for Computational Linguistics. ISBN 979-8-89176-390-6. doi: 10.18653/v1/2026.acl-long.365. URL <https://aclanthology.org/2026.acl-long.365/>.

- Fengxiang Cheng, Haoxuan Li, Fenrong Liu, Robert Van Rooij, Kun Zhang, and Zhouchen Lin. Empowering llms with logical reasoning: a comprehensive survey. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, pp. 10400–10408, 2025.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*, 2024. URL <https://arxiv.org/abs/2307.08691>.
- Tri Dao and Albert Gu. Transformers are ssms: Generalized models and efficient algorithms through structured state space duality. *arXiv preprint arXiv:2405.21060*, 2024.
- Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems 35 (NeurIPS 2022)*, 2022. URL <https://arxiv.org/abs/2205.14135>.
- DeepSeek-AI, Aixin Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Haowei Zhang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Li, Hui Qu, J. L. Cai, Jian Liang, Jianzhong Guo, Jiaqi Ni, Jiashi Li, Jiawei Wang, Jin Chen, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, Junxiao Song, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei Xu, Leyi Xia, Liang Zhao, Litong Wang, Liyue Zhang, Meng Li, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Ning Tian, Panpan Huang, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qinyu Chen, Qiushi Du, R. J. Chen, R. L. Jin, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runxin Xu, Ruoyu Zhang, Ruyi Chen, S. S. Li, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shaoqing Wu, Shengfeng Ye, Shengfeng Ye, Shirong Ma, Shiyu Wang, Shuang Zhou, Shuiping Yu, Shunfeng Zhou, Shuting Pan, T. Wang, Tao Yun, Tian Pei, Tianyu Sun, W. L. Xiao, Wangding Zeng, Wanbiao Zhao, Wei An, Wen Liu, Wenfeng Liang, Wenjun Gao, Wenqin Yu, Wentao Zhang, X. Q. Li, Xiangyue Jin, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaojin Shen, Xiaokang Chen, Xiaokang Zhang, Xiaosha Chen, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingkai Yu, Xinnan Song, Xinxia Shan, Xinyi Zhou, Xinyu Yang, Xinyuan Li, Xuecheng Su, Xuheng Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Yang Zhang, Yanhong Xu, Yanhong Xu, Yanping Huang, Yao Li, Yao Zhao, Yaofeng Sun, Yaohui Li, Yaohui Wang, Yi Yu, Yi Zheng, Yichao Zhang, Yifan Shi, Yiliang Xiong, Ying He, Ying Tang, Yishi Piao, Yisong Wang, Yixuan Tan, Yiyang Ma, Yiyuan Liu, Yongqiang Guo, Yu Wu, Yuan Ou, Yuchen Zhu, Yudian Wang, Yue Gong, Yuheng Zou, Yujia He, Yukun Zha, Yunfan Xiong, Yunxian Ma, Yuting Yan, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuyang Zhou, Z. F. Wu, Z. Z. Ren, Zehui Ren, Zhangli Sha, Zhe Fu, Zhean Xu, Zhen Huang, Zhen Zhang, Zhenda Xie, Zhengyan Zhang, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhipeng Xu, Zhiyu Wu, Zhongyu Zhang, Zhuoshu Li, Zihui Gu, Zijia Zhu, Zijun Liu, Zilin Li, Ziwei Xie, Ziyang Song, Ziyi Gao, and Zizheng Pan. DeepSeek-V3 technical report, 2025. URL <https://arxiv.org/abs/2412.19437>.
- DeepSeek-AI, Anyi Xu, Bangcai Lin, Bing Xue, Bingxuan Wang, Bingzheng Xu, Bochao Wu, Bowei Zhang, Chaofan Lin, Chen Dong, Chenchen Ling, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chengyu Hou, Chenhao Xu, Chenze Shao, Chong Ruan, Conner Sun, Damai Dai, Daya Guo, Dejian Yang, Deli Chen, Donghao Li, Dongjie Ji, Erhang Li, Fang Wei, Fangyun Lin, Fangzhou Yuan, Feiyu Xia, Fucong Dai, Guangbo Hao, Guanting Chen, Guoai Cao, Guolai Meng, Guowei Li, Han Yu, Han Zhang, Hanwei Xu, Hao Li, Haofen Liang, Haoling Zhang, Haoming Luo, Haoran Wei, Haotian Yuan, Haowei Zhang, Haowen Luo, Haoyu Chen, Haozhe Ji, Hengqing Zhang, Honghui Ding, Hongxuan Tang, Huanqi Cao, Huazuo Gao, Hui Qu, Hui Zeng, J Yang, JQ Zhu, Jia Luo, Jia Song, Jia Yu, Jialiang Huang, Jialu Cai, Jian Liang, Jiangting Zhou, Jiasheng Ye, Jiashi Li, Jiabin Xu, Jiewen Hu, Jieyu Yang, Jin Chen, Jin Yan, Jingchang Chen, Jingli Zhou, Jingting Xiang, Jingyang Yuan, Jingyuan Cheng, Jingzi Zhou, Jinhua Zhu, Jiping Yu, Joseph Sun, Jun Ran, Junguang Jiang, Junjie Qiu, Junlong Li, Junmin Zheng, Junxiao Song, Kai Dong, Kaige Gao, Kang Guan, Kexing Zhou, Kezhao Huang, Kuai Yu, Lean Wang, Lecong Zhang, Lei

- Wang, Leyi Xia, Li Zhang, Liang Zhao, Lihua Guo, Lingxiao Luo, Linwang Ma, Linyan Zhu, Litong Wang, Liyu Cai, Liyue Zhang, Longhao Chen, MS Di, MY Xu, Max Mei, Miaojun Wang, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Mingming Li, Mingxu Zhou, Minmin Han, Ning Wang, Panpan Huang, Panpan Wang, Peixin Cong, Peiyi Wang, Peng Zhang, Qiancheng Wang, Qihao Zhu, Qingyang Li, Qinyu Chen, Qiushi Du, Qiwei Jiang, Rui Tian, Ruifan Xu, Ruijie Lu, Ruiling Xu, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, Runqian Chen, Runqiu Yin, Runxin Xu, Ruomeng Shen, Ruoyu Zhang, Ruyi Chen, SH Liu, Shanghao Lu, Shangmian Sun, Shangyan Zhou, Shanhuang Chen, Shaofei Cai, Shaoheng Nie, Shaoqing Wu, Shaoyuan Chen, Shengding Hu, Shengyu Liu, Shiqiang Hu, Shirong Ma, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, Shuying Yu, Songyang Zhou, Tao Ni, Tao Yun, Tian Jin, Tian Pei, Tian Ye, Tianle Lin, Tianran Ji, Tianyi Cui, Tianyuan Yue, Tingting Yu, Tun Wang, W Zhang, WL Xiao, Wangding Zeng, Wei An, Weilin Zhao, Wen Liu, Wenfeng Liang, Wenjie Pang, Wenjing Luo, Wenjing Yao, Wenjun Gao, Wenkai Yang, Wenlve Huang, Wenqing Hou, Wentao Zhang, Wenting Ma, Xi Gao, Xiang He, Xiangwen Wang, Xianzu Wang, Xiao Bi, Xiaodong Liu, Xiaohan Wang, Xiaokang Chen, Xiaokang Zhang, Xiaotao Nie, Xiaowen Sun, Xiaoxiang Wang, Xin Cheng, Xin Liu, Xin Xie, Xingchao Liu, Xingchen Liu, Xingkai Yu, Xingyou Li, Xinyu Yang, Xinyu Zhang, Xu Chen, Xuanyu Wang, Xuecheng Su, Xueyin Chen, Xuheng Lin, Xuwei Fu, YC Yan, YQ Wang, YW Ma, Yanfeng Luo, Yang Zhang, Yanhong Xu, Yanru Ma, Yanwen Huang, Yao Li, Yao Li, Yao Xu, Yao Zhao, Yaofeng Sun, Yaohui Wang, Yi Qian, Yi Shao, Yi Yu, Yichao Zhang, Yifan Ding, Yifan Shi, Yijia Wu, Yiliang Xiong, Yiling Ma, Ying He, Ying Tang, Ying Zhou, Yingjia Luo, Yinmin Zhong, Yishi Piao, Yisong Wang, Yixiang Zhang, Yixiao Chen, Yixuan Tan, Yixuan Wei, Yiyang Ma, Yiyuan Liu, Yonglun Yang, Yongqiang Guo, Yongtong Wu, Yu Wu, YuKun Li, Yuan Cheng, Yuan Ou, Yuanfan Xu, Yuanhao Li, Yudian Wang, Yuehan Yang, Yuer Xu, Yuhan Wu, Yuhao Meng, Yuheng Zou, Yukun Zha, Yunfan Xiong, Yupeng Chen, Yuping Lin, Yuqian Cao, Yuqian Wang, Yushun Zhang, Yuting Yan, Yutong Lin, Yuxian Gu, Yuxiang Luo, Yuxiang You, Yuxuan Liu, Yuxuan Zhou, Yuyang Zhou, Yuzhen Huang, ZF Wu, Zehao Wang, Zehua Zhao, Zehui Ren, Zekai Zhang, Zhangli Sha, Zhe Fu, Zhe Ju, Zhean Xu, Zhenda Xie, Zhengyan Zhang, Zheren Gao, Zhewen Hao, Zhibin Gou, Zhicheng Ma, Zhigang Yan, Zhihong Shao, Zhixian Huang, Zhixuan Chen, Zhiyu Wu, Zhizhou Ren, Zhongyu Wu, Zhuoshu Li, Zhuping Zhang, Zian Xu, Zihao Wang, Zihua Qu, Zihui Gu, Zijia Zhu, Zilin Li, Zipeng Zhang, Ziwei Xie, Ziyi Gao, Ziyi Wan, Zizheng Pan, and Zongqing Yao. DeepSeek-V4: Towards highly efficient million-token context intelligence, 2026. URL <https://arxiv.org/abs/2606.19348>.
- Zezen Ding, Zhen Tan, Jiheng Zhang, and Tianlong Chen. OR-R1: Automating modeling and solving of operations research optimization problem via test-time reinforcement learning. In *Proceedings of the Thirty-Ninth AAAI Conference on Artificial Intelligence*, 2026. URL <https://arxiv.org/abs/2511.09092>.
- Guanting Dong, Junting Lu, Junjie Huang, Wanjuan Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, et al. Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence. *arXiv preprint arXiv:2604.18292*, 2026.
- Juncheng Dong, Yang Yang, Tao Liu, Yang Wang, Feng Qi, Vahid Tarokh, Kaushik Rangadurai, and Shuang Yang. STARK: Strategic team of agents for refining kernels. *CoRR*, abs/2510.16996, 2025. URL <https://arxiv.org/abs/2510.16996>.
- William Fedus, Barret Zoph, and Noam Shazeer. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research*, 23(120):1–39, 2022.
- Trevor Gale, Deepak Narayanan, Cliff Young, and Matei Zaharia. Megablocks: Efficient sparse training with mixture-of-experts. *Proceedings of Machine Learning and Systems*, 5:288–304, 2023.
- Zhiqi Gao, Albert Ge, Alexander Berenbeim, Nathaniel D. Bastian, and Frederic Sala. Models can model, but can't bind: Structured grounding in text-to-optimization. *CoRR*, abs/2605.21751, 2026. doi: 10.48550/ARXIV.2605.21751.
- Albert Ge. Synthetic operations research question answering dataset. <https://huggingface.co/datasets/albertge/synthetic-orqa>, 2025. Hugging Face dataset.
- GLM-5-Team, Aohan Zeng, Xin Lv, Zhenyu Hou, Zhengxiao Du, Qinkai Zheng, Bin Chen, Da Yin, Chendi Ge, Chenghua Huang, Chengxing Xie, Chenzheng Zhu, Congfeng Yin, Cunxiang Wang,

- Gengzheng Pan, Hao Zeng, Haoke Zhang, Haoran Wang, Huilong Chen, Jiajie Zhang, Jian Jiao, Jiaqi Guo, Jingsen Wang, Jingzhao Du, Jinzhu Wu, Kedong Wang, Lei Li, Lin Fan, Lucen Zhong, Mingdao Liu, Mingming Zhao, Pengfan Du, Qian Dong, Rui Lu, Shuang-Li, Shulin Cao, Song Liu, Ting Jiang, Xiaodong Chen, Xiaohan Zhang, Xuancheng Huang, Xuezhen Dong, Yabo Xu, Yao Wei, Yifan An, Yilin Niu, Yitong Zhu, Yuanhao Wen, Yukuo Cen, Yushi Bai, Zhongpei Qiao, Zihan Wang, Zikang Wang, Zilin Zhu, Ziqiang Liu, Zixuan Li, Bojie Wang, Bosi Wen, Can Huang, Changpeng Cai, Chao Yu, Chen Li, Chengwei Hu, Chenhui Zhang, Dan Zhang, Daoyan Lin, Dayong Yang, Di Wang, Ding Ai, Erle Zhu, Fangzhou Yi, Feiyu Chen, Guohong Wen, Hailong Sun, Haisha Zhao, Haiyi Hu, Hanchen Zhang, Hanrui Liu, Hanyu Zhang, Hao Peng, Hao Tai, Haobo Zhang, He Liu, Hongwei Wang, Hongxi Yan, Hongyu Ge, Huan Liu, Huanpeng Chu, Jia'ni Zhao, Jiachen Wang, Jiajing Zhao, Jiamin Ren, Jiapeng Wang, Jiaxin Zhang, Jiayi Gui, Jiayue Zhao, Jijie Li, Jing An, Jing Li, Jingwei Yuan, Jinhua Du, Jinxin Liu, Junkai Zhi, Junwen Duan, Kaiyue Zhou, Kangjian Wei, Ke Wang, Keyun Luo, Laiqiang Zhang, Leigang Sha, Liang Xu, Lindong Wu, Lintao Ding, Lu Chen, Minghao Li, Nianyi Lin, Pan Ta, Qiang Zou, Rongjun Song, Ruiqi Yang, Shangqing Tu, Shangtong Yang, Shaoxiang Wu, Shengyan Zhang, Shijie Li, Shuang Li, Shuyi Fan, Wei Qin, Wei Tian, Weining Zhang, Wenbo Yu, Wenjie Liang, Xiang Kuang, Xiangmeng Cheng, Xiangyang Li, Xiaoquan Yan, Xiaowei Hu, Xiaoying Ling, Xing Fan, Xingye Xia, Xinyuan Zhang, Xinze Zhang, Xirui Pan, Xu Zou, Xunkai Zhang, Yadi Liu, Yandong Wu, Yanfu Li, Yidong Wang, Yifan Zhu, Yijun Tan, Yilin Zhou, Yiming Pan, Ying Zhang, Yinpei Su, Yipeng Geng, Yong Yan, Yonglin Tan, Yuean Bi, Yuhao Yang, Yuhao Yang, Yujiang Li, Yunan Liu, Yunqing Wang, Yuntao Li, Yurong Wu, Yutao Zhang, Yuxi Duan, Yuxuan Zhang, Zezhen Liu, Zhengtao Jiang, Zhenhe Yan, Zheyu Zhang, Zhixiang Wei, Zhuo Chen, Zhuoer Feng, Zijun Yao, Ziwei Chai, Ziyuan Wang, Zuzhou Zhang, Bin Xu, Minlie Huang, Hongning Wang, Juanzi Li, Yuxiao Dong, and Jie Tang. GLM-5: from vibe coding to agentic engineering, 2026. URL <https://arxiv.org/abs/2602.15763>.
- Google Cloud. Gemini 3-Flash. <https://docs.cloud.google.com/gemini-enterprise-agent-platform/models/gemini/3-flash>, 2026. Accessed: 2026-07-08.
- Gurobi Optimization, LLC. Gurobi modeling examples. <https://github.com/Gurobi/modeling-examples>, 2026. Official Jupyter Notebook repository; accessed June 23, 2026.
- Suchin Gururangan, Ana Marasović, Swabha Swayamdipta, Kyle Lo, Iz Beltagy, Doug Downey, and Noah A. Smith. Don't stop pretraining: Adapt language models to domains and tasks. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 8342–8360. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.acl-main.740. URL <https://aclanthology.org/2020.acl-main.740>.
- Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. *Proceedings of the International Conference on Learning Representations (ICLR)*, 2021a.
- Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the math dataset, 2021b. URL <https://arxiv.org/abs/2103.03874>.
- Pin-Lun Hsu, Yun Dai, Vignesh Kothapalli, Qingquan Song, Shao Tang, Siyu Zhu, Steven Shimizu, Shivam Sahni, Haowen Ning, and Yanning Chen. Liger kernel: Efficient Triton kernels for LLM training. *CoRR*, abs/2410.10989, 2024. URL <https://arxiv.org/abs/2410.10989>.
- Chenyu Huang, Zhengyang Tang, Shixi Hu, Ruqing Jiang, Xin Zheng, Dongdong Ge, Benyou Wang, and Zizhuo Wang. ORLM: A customizable framework in training large models for automated optimization modeling. *Operations Research*, 73(6):2986–3009, 2025. doi: 10.1287/OPRE.2024.1233.
- Xuhan Huang, Qingning Shen, Yan Hu, Anningzhe Gao, and Benyou Wang. Mamo: A mathematical modeling benchmark with solvers. *CoRR*, abs/2405.13144, 2024. doi: 10.48550/ARXIV.2405.13144.
- Huawei Ascend. MindSpeed-LLM. <https://github.com/Ascend/MindSpeed-LLM>, 2024.
- Jinhao Jiang, Junyi Li, Xin Zhao, Yang Song, Tao Zhang, and Ji-Rong Wen. Mix-cpt: A domain adaptation framework via decoupling knowledge learning and format alignment. In *International Conference on Learning Representations*, volume 2025, pp. 82198–82216, 2025.

- Ishan Jindal, Chandana Badrinath, Pranjal Bharti, Lakkidi Vinay, and Sachin Dev Sharma. Balancing continuous pre-training and instruction fine-tuning: Optimizing instruction-following in llms. arXiv preprint arXiv:2410.10739, 2024.
- Zixuan Ke, Yifei Ming, Xuan-Phi Nguyen, Caiming Xiong, and Shafiq Joty. Demystifying domain-adaptive post-training for financial llms. In Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing, pp. 31021–31047, 2025.
- Kimi Team, Yifan Bai, Yiping Bao, Y. Charles, Cheng Chen, Guanduo Chen, Haiting Chen, Huarong Chen, Jiahao Chen, Ningxin Chen, Ruijue Chen, Yanru Chen, Yuankun Chen, Yutian Chen, Zhuofu Chen, Jialei Cui, Hao Ding, Mengnan Dong, Angang Du, Chenzhuang Du, Dikang Du, Yulun Du, Yu Fan, Yichen Feng, Kelin Fu, Bofei Gao, Chenxiao Gao, Hongcheng Gao, Peizhong Gao, Tong Gao, Yuyao Ge, Shangyi Geng, Qizheng Gu, Xinran Gu, Longyu Guan, Haiqing Guo, Jianhang Guo, Xiaoru Hao, Tianhong He, Weiran He, Wenyang He, Yunjia He, Chao Hong, Hao Hu, Yangyang Hu, Zhenxing Hu, Weixiao Huang, Zhiqi Huang, Zihao Huang, Tao Jiang, Zhejun Jiang, Xinyi Jin, Yongsheng Kang, Guokun Lai, Cheng Li, Fang Li, Haoyang Li, Ming Li, Wentao Li, Yang Li, Yanhao Li, Yiwei Li, Zhaowei Li, Zheming Li, Hongzhan Lin, Xiaohan Lin, Zongyu Lin, Chengyin Liu, Chenyu Liu, Hongzhang Liu, Jingyuan Liu, Junqi Liu, Liang Liu, Shaowei Liu, T. Y. Liu, Tianwei Liu, Weizhou Liu, Yangyang Liu, Yibo Liu, Yiping Liu, Yue Liu, Zhengying Liu, Enzhe Lu, Haoyu Lu, Lijun Lu, Yashuo Luo, Shengling Ma, Xinyu Ma, Yingwei Ma, Shaoguang Mao, Jie Mei, Xin Men, Yibo Miao, Siyuan Pan, Yebo Peng, Ruoyu Qin, Zeyu Qin, Bowen Qu, Zeyu Shang, Lidong Shi, Shengyuan Shi, Feifan Song, Jianlin Su, Zhengyuan Su, Lin Sui, Xinjie Sun, Flood Sung, Yunpeng Tai, Heyi Tang, Jiawen Tao, Qifeng Teng, Chaoran Tian, Chensi Wang, Dinglu Wang, Feng Wang, Hailong Wang, Haiming Wang, Jianzhou Wang, Jiaxing Wang, Jinhong Wang, Shengjie Wang, Shuyi Wang, Si Wang, Xinyuan Wang, Yao Wang, Yejie Wang, Yiqin Wang, Yuxin Wang, Yuzhi Wang, Zhaoji Wang, Zhengtao Wang, Zhengtao Wang, Zhexu Wang, Chu Wei, Qianqian Wei, Haoning Wu, Wenhao Wu, Xingzhe Wu, Yuxin Wu, Chenjun Xiao, Jin Xie, Xiaotong Xie, Weimin Xiong, Boyu Xu, Jinjing Xu, L. H. Xu, Lin Xu, Suting Xu, Weixin Xu, Xinran Xu, Yangchuan Xu, Ziyao Xu, Jing Xu, Jing Xu, Junjie Yan, Yuzi Yan, Hao Yang, Xiaofei Yang, Yi Yang, Ying Yang, Zhen Yang, Zhilin Yang, Zonghan Yang, Haotian Yao, Xingcheng Yao, Wenjie Ye, Zhuorui Ye, Bohong Yin, Longhui Yu, Enming Yuan, Hongbang Yuan, Mengjie Yuan, Siyu Yuan, Haobing Zhan, Dehao Zhang, Hao Zhang, Wanlu Zhang, Xiaobin Zhang, Yadong Zhang, Yangkun Zhang, Yichi Zhang, Yizhi Zhang, Yongting Zhang, Yu Zhang, Yutao Zhang, Yutong Zhang, Zheng Zhang, Haotian Zhao, Yikai Zhao, Zijia Zhao, Huabin Zheng, Shaojie Zheng, Longguang Zhong, Jianren Zhou, Xinyu Zhou, Zaida Zhou, Jinguo Zhu, Zhen Zhu, Weiyu Zhuang, and Xinxing Zu. Kimi K2: Open agentic intelligence, 2026. URL <https://arxiv.org/abs/2507.20534>.
- Minwei Kong, Chonghe Jiang, Ao Qu, Wenbin Ouyang, Zhaoming Zeng, Xiaotong Guo, Zhekai Li, Junyi Li, Yi Fan, Xinshou Zheng, Xi Jing, Yikai Zhang, Zhiwei Liang, Seonghoo Kim, Runqing Yang, Zijian Zhou, Sirui Li, Han Zheng, Wangyang Ying, Ou Zheng, Chonghuan Wang, Jinglong Zhao, Hanzhang Qin, Cathy Wu, Paul Pu Liang, Jinhua Zhao, and Hai Wang. FrontierOR: Benchmarking LLMs’ capacity for efficient algorithm design in large-scale optimization. CoRR, abs/2605.25246, 2026. doi: 10.48550/ARXIV.2605.25246.
- Jakub Krajewski, Jan Ludziejewski, Kamil Adamczewski, Maciej Pióro, Michał Krutul, Szymon Antoniak, Kamil Ciebia, Krystian Król, Tomasz Odrzygóźdź, Piotr Sankowski, et al. Scaling laws for fine-grained mixture of experts. arXiv preprint arXiv:2402.07871, 2024.
- Hanyu Lai, Xiao Liu, Junjie Gao, Jiale Cheng, Zehan Qi, Yifan Xu, Shuntian Yao, Dan Zhang, Jinhua Du, Zhenyu Hou, et al. A survey of post-training scaling in large language models. In Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers), pp. 2771–2791, 2025.
- Tony Lee and Percy Liang. Self-verified distillation: Your language model is secretly its own synthetic data pipeline. arXiv preprint arXiv:2605.26132, 2026.
- Chao Li, Yifan Zhang, Yanzhen Lu, and Andrew C. Yao. AutoMathText-V2: A 2.46 trillion token ai-curated stem pretraining dataset. Technical report, Institute for Interdisciplinary Information Sciences, Tsinghua University, August 2025. URL <https://iis-ai.github.io/AutoMathText-V2/AutoMathText-V2.pdf>.

- Dongfang Li, Zixuan Liu, Gang Lin, Baotian Hu, and Min Zhang. LycheeCluster: Efficient long-context inference with structure-aware chunking and hierarchical KV indexing. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 7607–7623, San Diego, California, United States, July 2026a. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.376. URL <https://aclanthology.org/2026.findings-acl.376/>.
- Yaxuan Li, Yuxin Zuo, Bingxiang He, Jinqian Zhang, Chaojun Xiao, Cheng Qian, Tianyu Yu, Huanang Gao, Wenkai Yang, Zhiyuan Liu, et al. Rethinking on-policy distillation of large language models: Phenomenology, mechanism, and recipe. *arXiv preprint arXiv:2604.13016*, 2026b.
- Xize Liang, Lin Yang, Jie Wang, Yiyang Lu, Runyu Wu, Hanzhu Chen, and Jianye Hao. Boosting multi-domain fine-tuning of large language models through evolving interactions between samples. In *Forty-second International Conference on Machine Learning*, 2025.
- Jingyang Lin, Andy Wong, Tian Xia, Shenghua He, Hui Wei, Mei Han, and Jiebo Luo. Facilitating long context understanding via supervised chain-of-thought reasoning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 12243–12259, 2025.
- Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017.
- Nicholas Lownes. Operations research in civil and environmental engineering. University of Connecticut Open Educational Resource, 2024. URL https://digitalcommons.lib.uconn.edu/open_educational_resource/1.
- Hongliang Lu, Zhonglin Xie, Yaoyu Wu, Can Ren, Yuxuan Chen, and Zaiwen Wen. OptMATH: A scalable bidirectional data synthesis framework for optimization modeling. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 40769–40802. PMLR, 2025. URL <https://proceedings.mlr.press/v267/lu25o.html>.
- Nick Mecklenburg, Yiyu Lin, Xiaoxiao Li, Daniel Holstein, Leonardo Nunes, Sara Malvar, Bruno Silva, Ranveer Chandra, Vijay Aski, Pavan Kumar Reddy Yannam, et al. Injecting new knowledge into large language models via supervised fine-tuning. *arXiv preprint arXiv:2404.00213*, 2024.
- MIT HAN Lab. Kernel Design Agents. <https://github.com/mit-han-lab/kernel-design-agents>, 2026. GitHub repository. Accessed: 2026-07-10.
- Mahdi Mostajabdaveh, Timothy Tin Long Yu, Samarendra Chandan Bindu Dash, Rindra Ramamonjison, Jabo Serge Byusa, Giuseppe Carenini, Zirui Zhou, and Yong Zhang. Evaluating llm reasoning in the operations research domain with orqa. In *Thirty-Ninth AAAI Conference on Artificial Intelligence, AAAI 2025*, pp. 24902–24910. AAAI Press, 2025. doi: 10.1609/AAAI.V39I23.34673.
- Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the international conference for high performance computing, networking, storage and analysis*, pp. 1–15, 2021.
- NVIDIA, Aarti Basant, Abhijit Khairnar, Abhijit Paithankar, Abhinav Khattar, Adithya Renduchintala, Aditya Malte, Akhiad Bercovich, Akshay Hazare, Alejandra Rico, Aleksander Ficek, Alex Kondratenko, Alex Shaposhnikov, Alexander Bukharin, Ali Taghibakhshi, Amelia Barton, Ameya Sunil Mahabaleshwarkar, Amy Shen, Andrew Tao, Ann Guan, Anna Shors, Anubhav Mandarwal, Arham Mehta, Arun Venkatesan, Ashton Sharabiani, Ashwath Aithal, Ashwin Poojary, Ayush Dattagupta, Balaram Buddharaju, Banghua Zhu, Barnaby Simkin, Bilal Kartal, Bitu Darvish Rouhani, Bobby Chen, Boris Ginsburg, Brandon Norick, Brian Yu, Bryan Catanzaro, Charles Wang, Charlie Truong, Chetan Mungekar, Chintan Patel, Chris Alexiuk, Christian Munley, Christopher Parisien, Dan Su, Daniel Afrimi, Daniel Korzekwa, Daniel Rohrer, Daria Gitman, David Mosalanezhad, Deepak Narayanan, Dima Rekesh, Dina Yared, Dmytro Pykhtar, Dong Ahn, Duncan Riach, Eileen Long, Elliott Ning, Eric Chung, Erick Galinkin, Evelina Bakhturina, Gargi Prasad, Gerald Shen, Haifeng Qian, Haim Elisha, Harsh Sharma, Hayley Ross, Helen Ngo, Herman Sahota,

Hexin Wang, Hoo Chang Shin, Hua Huang, Iain Cunningham, Igor Gitman, Ivan Moshkov, Jaehun Jung, Jan Kautz, Jane Polak Scowcroft, Jared Casper, Jian Zhang, Jiaqi Zeng, Jimmy Zhang, Jinze Xue, Jocelyn Huang, Joey Conway, John Kamalu, Jonathan Cohen, Joseph Jennings, Julien Veron Vialard, Junkeun Yi, Jupinder Parmar, Kari Briski, Katherine Cheung, Katherine Luna, Keith Wyss, Keshav Santhanam, Kezhi Kong, Krzysztof Pawelec, Kumar Anik, Kunlun Li, Kushan Ahmadian, Lawrence McAfee, Laya Sleiman, Leon Derczynski, Luis Vega, Maer Rodrigues de Melo, Makesh Narsimhan Sreedhar, Marcin Chochowski, Mark Cai, Markus Kliegl, Marta Stepniewska-Dziubinska, Matvei Novikov, Mehrzad Samadi, Meredith Price, Meriem Boubdir, Michael Boone, Michael Evans, Michal Bien, Michal Zawalski, Miguel Martinez, Mike Chrzanowski, Mohammad Shoeibi, Mostofa Patwary, Namit Dhameja, Nave Assaf, Negar Habibi, Nidhi Bhatia, Nikki Pope, Nima Tajbakhsh, Nirmal Kumar Juluru, Oleg Rybakov, Oleksii Hrinchuk, Oleksii Kuchaiev, Oluwatobi Olabiyi, Pablo Ribalta, Padmavathy Subramanian, Parth Chadha, Pavlo Molchanov, Peter Dykas, Peter Jin, Piotr Bialecki, Piotr Januszewski, Pradeep Thalasta, Prashant Gaikwad, Prasoon Varshney, Pritam Gundecha, Przemek Tredak, Rabeeh Karimi Mahabadi, Rajen Patel, Ran El-Yaniv, Ranjit Rajan, Ria Cheruvu, Rima Shahbazyan, Ritika Borkar, Ritu Gala, Roger Waleffe, Ruoxi Zhang, Russell J. Hewett, Ryan Prenger, Sahil Jain, Samuel Krizan, Sanjeev Satheesh, Saori Kaji, Sarah Yurick, Saurav Muralidharan, Sean Narenthiran, Seonmyeong Bak, Sepehr Sameni, Seungju Han, Shanmugam Ramasamy, Shaona Ghosh, Sharath Turuvekere Sreenivas, Shelby Thomas, Shizhe Diao, Shreya Gopal, Shrimai Prabhumoye, Shubham Toshniwal, Shuoyang Ding, Siddharth Singh, Siddhartha Jain, Somshubra Majumdar, Soumye Singhal, Stefania Alborghetti, Syeda Nahida Akter, Terry Kong, Tim Moon, Tomas Hliwiak, Tomer Asida, Tony Wang, Tugrul Konuk, Twinkle Vashishth, Tyler Poon, Udi Karpas, Vahid Noroozi, Venkat Srinivasan, Vijay Korthikanti, Vikram Fugro, Vineeth Kalluru, Vitaly Kurin, Vitaly Lavrukhin, Wasi Uddin Ahmad, Wei Du, Wonmin Byeon, Ximing Lu, Xin Dong, Yashaswi Karnati, Yejin Choi, Yian Zhang, Ying Lin, Yonggan Fu, Yoshi Suhara, Zhen Dong, Zhiyu Li, Zhongbo Zhu, and Zijia Chen. NVIDIA Nemotron Nano 2: An accurate and efficient hybrid mamba-transformer reasoning model, 2025. URL <https://arxiv.org/abs/2508.14444>.

NVIDIA, Aakshita Chandiramani, Aaron Blakeman, Abdullahi Olaoye, Abhibha Gupta, Abhilash Somasamudramath, Abhinav Khattar, Adeola Adesoba, Adi Renduchintala, Adil Asif, Aditya Agrawal, Aditya Vavre, Ahmad Kiswani, Aishwarya Padmakumar, Ajay Hotchandani, Akanksha Shukla, Akhiad Bercovich, Aleksander Ficek, Aleksandr Shaposhnikov, Alex Gronskiy, Alex Kondratenko, Alex Neefus, Alex Steiner, Alex Yang, Alexander Bukharin, Alexander Young, Ali Hatamizadeh, Ali Taghibakhshi, Alina Galiutdinova, Alisa Liu, Alok Kumar, Ameya Sunil Mahabaleshwarkar, Amir Klein, Amit Zuker, Amnon Geifman, Anahita Bhiwandiwalla, Ananth Subramaniam, Andrew Tao, Anjaney Shrivastava, Anjulie Agrusa, Ankur Srivastava, Ankur Verma, Ann Guan, Anna Shors, Annamalai Chockalingam, Anubhav Mandarwal, Aparna Ramani, Arham Mehta, Arti Jain, Arun Venkatesan, Asha Anoosheh, Ashwath Aithal, Ashwin Poojary, Asif Ahamed, Asit Mishra, Asli Sabanci Demiroz, Asma Kuriparambil Thekkumpate, Atefeh Sohrabizadeh, Avinash Kaur, Ayush Dattagupta, Barath Subramaniam Anandan, Bardiya Sadeghi, Barnaby Simkin, Ben Lanir, Benedikt Schifferer, Benjamin Chislett, Besmira Nushi, Bilal Kartal, Bill Thiede, Bitu Darvish Rouhani, Bobby Chen, Boris Ginsburg, Brandon Norick, Branislav Kisacanin, Brian Yu, Bryan Catanzaro, Buvaneshwari Mani, Carlo del Mundo, Chankyu Lee, Chanran Kim, Chantal Hwang, Chao Ni, Charles Wang, Charlie Truong, Cheng-Ping Hsieh, Chenhan Yu, Chenjie Luo, Cherie Wang, Chetan Mungekar, Chintan Patel, Chris Alexiuk, Chris Holguin, Chris Wing, Christian Munley, Christopher Parisien, Chuck Desai, Chunyang Sheng, Collin Neale, Cyril Meurillon, Dakshi Kumar, Dan Gil, Dan Su, Dane Corneil, Daniel Afrimi, Daniel Burkhardt Eliuth Triana, Daniel Egert, Daniel Fatade, Daniel Lo, Daniel Rohrer, Daniel Serebrenik, Daniil Sorokin, Daria Gitman, Daria Levy, Darko Stosic, David Edelsohn, David Messina, David Mosallanezhad, David Tamok, Deena Donia, Deepak Narayanan, Devin O'Kelly, Dheeraj Peri, Dhruv Nathawani, Di Wu, Dima Rekesh, Dina Yared, Divyanshu Kakwani, Dmitry Konyagin Brandon Tuttle, Dong Ahn, Dongfu Jiang, Dorrin Poorkay, Douglas O'Flaherty, Duncan Riach, Dusan Stosic, Dustin Van Stee, Edgar Minasyan, Edward Lin, Eileen Peters Long, Elad Segal, Elena Lantz, Elena Lewis, Ellie Evans, Elliott Ning, Eric Chung, Eric Harper, Eric Pham-Hung, Eric W. Tramel, Erick Galinkin, Erik Pounds, Esti Etrog, Evan Briones, Evan Wu, Evelina Bakhturina, Evgeny Tsykunov, Ewa Dobrowolska, Farshad Saberi Movahed, Farzan Memarian, Fay Wang, Fei Jia, Felipe Soares, Felipe Vieira Frujeri, Feng Chen, Fengguang Lin, Ferenc Galko, Fortuna Zhang, Frankie Siino, Frida Hou, Gantavya Bhatt, Gargi Prasad, Geethapriya Venkataramani, Geetika Gupta, George Armstrong, Gerald Shen, Giulio Borghesi, Gordana Neskovic, Gorkem

Batmaz, Grace Lam, Grace Wu, Greg Pauloski, Greyson Davis, Grigor Nalbandyan, Guoming Zhang, Guy Farber, Guyue Huang, Haifeng Qian, Haran Kumar Shiv Kumar, Harry Kim, Harsh Sharma, Hayate Iso, Hayley Ross, Herbert Hum, Herman Sahota, Hexin Wang, Himanshu Soni, Hiren Upadhyay, Huy Nguyen, Iain Cunningham, Ido Galil, Ido Shahaf, Igino Padovani, Igor Gitman, Igor Shovkun, Ikroop Dhillon, Ilya Loshchilov, Ingrid Kelly, Itamar Schen, Itay Levy, Ivan Moshkov, Izik Golan, Izzy Putterman, Jain Tu, Jan Baczek, Jan Kautz, Jane Polak Scowcroft, Janica Rosenberg, Jared Casper, Jarrod Pflum, Jason Grant, Jason Sewall, Jatin Mitra, Jeffrey Glick, Jenny Chen, Jesse Oliver, Jiacheng Xu, Jiafan Zhu, Jialin Song, Jian Zhang, Jiaqi Zeng, Jie Lou, Jill Milton, Jim Chow, Jimmy Zhang, Jinhang Choi, Jining Huang, Jocelyn Huang, Joel Caruso, Joey Conway, Joey Guman, Johan Jatko, John Kamalu, Johnny Greco, Jonathan Cohen, Jonathan Raiman, Joseph Jennings, Joyjit Daw, Juan Yu, Julio Tapia, Junkeun Yi, Jupinder Parmar, Jyothi Achar, Kari Briski, Kartik Mattoo, Katherine Cheung, Katherine Luna, Keith Wyss, Kevin Shih, Kezhi Kong, Khanh Nguyen, Khushi Bhardwaj, Kirill Buryak, Kirthi Shankar Sivamani, Konstantinos Krommydas, Kris Murphy, Krishna C. Puvvada, Krzysztof Pawelec, Kumar Anik, Laikh Tewari, Laya Sleiman, Leo Du, Leon Derczynski, Li Ding, Lilach Ilan, Lingjie Wu, Lizzie Wei, Luis Vega, Lun Su, Maarten Van Segbroeck, Maer Rodrigues de Melo, Magaret Zhang, Mahan Fathi, Makesh Narsimhan Sreedhar, Makesh Sreedhar, Makesh Tarun Chandran, Manuel Reyes Gomez, Maor Ashkenazi, Marc Cuevas, Marc Romeijn, Margaret Zhang, Mark Cai, Mark Gabel, Markus Kliegl, Martyna Patelka, Maryam Moosaei, Matthew Varacalli, Matvei Novikov, Mauricio Ferrato, Mehrzad Samadi, Melissa Corpuz, Meng Xin, Mengdi Wang, Mengru Wang, Meredith Price, Micah Schaffer, Michael Andersch, Michael Boone, Michael Evans, Michael Z Wang, Miguel Martinez, Mikail Khona, Mike Chrzanowski, Mike Hollinger, Mingyuan Ma, Minseok Lee, Mohammad Dabbah, Mohammad Shoeybi, Mostofa Patwary, Nabin Mulepati, Nader Khalil, Najeel Nabwani, Nancy Agarwal, Nanthini Balasubramaniam, Narimane Hennouni, Narsi Kodukula, Natalie Hereth, Nathaniel Pinckney, Nave Assaf, Negar Habibi, Nestor Qin, Neta Zmora, Netanel Haber, Nick Reamaroon, Nickson Quak, Nidhi Bhatia, Nikhil Jukar, Nikki Pope, Nikolai Ludwig, Nima Tajbakhsh, Nir Ailon, Nirmal Juluru, Nirmalya De, Nowel Pitt, Oleg Rybakov, Oleksii Hrinchuk, Oleksii Kuchaiev, Olivier Delalleau, Oluwatobi Olabiyi, Omer Ullman Argov, Omri Almog, Omri Puny, Oren Tropp, Otavio Padovani, Ouye Xie, Parth Chadha, Pasha Shamis, Paul Gibbons, Pavlo Molchanov, Peter Belcak, Peter Jin, Pinky Xu, Piotr Januszewski, Pooya Jannaty, Prachi Shevate, Pradeep Thalasta, Pranav Prashant Thombre, Prasoon Varshney, Prerana Gambhir, Pritam Gundecha, Przemek Tredak, Qing Miao, Qiyu Wan, Quan Tran Minh, Rabeel Karimi Mahabadi, Rachel Oberman, Rachit Garg, Rahul Kandu, Raina Zhong, Ran El-Yaniv, Ran Zilberstein, Rasoul Shafipour, Renee Yao, Renjie Pi, Richard Mazzaresse, Richard Wang, Rick Izzo, Ridhima Singla, Rima Shahbazyan, Rishabh Garg, Ritika Borkar, Ritu Gala, Riyadh Islam, Robert Clark, Robert Hesse, Roger Waleffe, Rohit Varma Kalidindi, Rohit Watve, Roi Koren, Ron Fan, Ruchika Kharwar, Ruisi Cai, Ruoxi Zhang, Russell J. Hewett, Ryan Prenger, Ryan Timbrook, Ryota Egashira, Sadegh Mahdavi, Sagar Singh Ashutosh Joshi, Sahil Modi, Samuel Krizan, Sandeep Pombra, Sanjay Kariyappa, Sanjeev Satheesh, Santiago Pombo, Saori Kaji, Satish Pasumarthi, Saurav Mishra, Saurav Muralidharan, Scott Hara, Sean Narenthiran, Sebastian Rogawski, Seonjin Na, Seonmyeong Bak, Sepehr Sameni, Seth Poulos, Shahar Mor, Shantanu Acharya, Shaona Ghosh Adam Lord, Sharath Turuvekere Sreenivas, Shaun Kotek, Shaya Gharghabi, Shelby Thomas, Sheng-Chieh Lin, Shibani Likhite, Shiqing Fan, Shiyang Chen, Shreya Gopal, Shrimai Prabhume, Shubham Pachori, Shubham Toshniwal, Shuo Zhang, Shuoyang Ding, Shyam Renjith, Shyamala Prayaga, Siddhartha Jain, Simeng Sun, Sirisha Rella, Sirshak Das, Smita Ithape, Sneha Harishchandra S, Somshubra Majumdar, Soumye Singhal, Sri Harsha Singudasu, Sriharsha Niverty, Stas Sergienko, Stefania Gloginich, Stefania Alborghetti, Stephen Ge, Stephen McCullough, Sugam Dipak Devare, Suguna Varshini Velury, Sukrit Rao, Sumeet Kumar Barua, Sunny Gai, Suseella Panguluri, Sushil Koundinyan, Swathi Patnam, Sweta Priyadarshi, Swetha Bhendigeri, Syeda Nahida Akter, Sylendran Arunagiri, Tailling Yuan, Talor Abramovich, Tan Bui, Tan Yu, Terry Kong, Thanh Do, Thomas Gburek, Thorgane Marques, Tiffany Moore, Tijmen Blankevoort, Tim Moon, Timothy Ma, Tiya Mitra, Tomasz Grzegorzec, Tomer Asida, Tomer Bar Natan, Tomer Keren, Tomer Ronen, Traian Rebedea, Trenton Starkey, Tugrul Konuk, Twinkle Vashishth, Tyler Condensa, Udi Karpas, Ushnish De, Vahid Noorzi, Vahid Noroozi, Vanshil Atul Shah, Veena Vaidyanathan, Venkat Srinivasan, Venmugil Elango, Victor Cui, Vijay Korthikanti, Vikas Mehta, Virginia Adams, Virginia Wu, Vitaly Kurin, Vitaly Lavrukhin, Vladimir Anisimov, Wan Seo, Wanli Jiang, Wasi Uddin Ahmad, Wei Du, Wei Ping, Wei-Ming Chen, Wendy Quan, Wenliang Dai, Wenwen Gao, Will Jennings, William Zhang, Xiaowei Ren, Xiaowen Xin, Xin Li, Yang Yu, Yangyi Chen, Yaniv Galron, Yashaswi Karnati, Yejin Choi, Yev Meyer, Yi-Fu Wu, Yian Zhang, Ying Lin, Yonatan Geifman,

- Yonggan Fu, Yoshi Suhara, Youngeun Kwon, Yuan Zhang, Yuki Huang, Zach Moshe, Zhilin Wang, Zhiyu Cheng, Zhongbo Zhu, Zhuolin Yang, Zihan Liu, Zijia Chen, Zijie Yan, and Zuhair Ahmed. Nemotron 3 Super: Open, efficient mixture-of-experts hybrid mamba-transformer model for agentic reasoning, 2026. URL <https://arxiv.org/abs/2604.12374>.
- OpenAI. GPT-5.4-mini. <https://developers.openai.com/api/docs/models/gpt-5.4-mini>, 2026. Accessed: 2026-07-08.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F. Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems* 35, 2022. URL http://papers.nips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.
- Penghui Qi, Xinyi Wan, Guangxing Huang, and Min Lin. Dualpipe could be better without the dual. <https://hackmd.io/@ufotalent/r1lVXsa9Jg>, 2025. Blog.
- Zhenting Qi, Fan Nie, Alexandre Alahi, James Zou, Himabindu Lakkaraju, Yilun Du, Eric P. Xing, Sham M. Kakade, and Hanlin Zhang. EvoLM: In search of lost training dynamics for language model reasoning. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026. URL <https://openreview.net/forum?id=B6bE2GC71a>.
- Rindranirina Ramamonjison, Timothy Yu, Raymond Li, Haley Li, Giuseppe Carenini, Bissan Ghaddar, Shiqi He, Mahdi Mostajabdaveh, Amin Banitalebi-Dehkordi, Zirui Zhou, and Yong Zhang. Nl4opt competition: Formulating optimization problems based on their natural language descriptions. In Marco Ciccone, Gustavo Stolovitzky, and Jacob Albrecht (eds.), *Proceedings of the NeurIPS 2022 Competitions Track*, volume 220 of *Proceedings of Machine Learning Research*, pp. 189–203. PMLR, 28 Nov–09 Dec 2022. URL <https://proceedings.mlr.press/v220/ramamonjison23a.html>.
- Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. Zero-offload: Democratizing {billion-scale} model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*, pp. 551–564, 2021.
- Victor Sanh, Albert Webson, Colin Raffel, Stephen H Bach, Lintang Sutawika, Zaid Alyafeai, Antoine Chaffin, Arnaud Stiegler, Teven Le Scao, Arun Raja, et al. Multitask prompted training enables zero-shot task generalization. *arXiv preprint arXiv:2110.08207*, 2021.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. FlashAttention-3: Fast and accurate attention with asynchrony and low-precision. *CoRR*, abs/2407.08608, 2024. URL <https://arxiv.org/abs/2407.08608>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, YK Li, Yang Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- Shamane Siriwardhana, Mark McQuade, Thomas Gauthier, Lucas Atkins, Fernando Fernandes Neto, Luke Meyers, Anneketh Vij, Tyler Odenthal, Charles Goddard, Mary MacCarthy, et al. Domain adaptation of llama3-70b-instruct through continual pre-training and model merging: A comprehensive evaluation. *arXiv preprint arXiv:2406.14971*, 2024.
- Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. doi: 10.1016/j.neucom.2023.127063. URL <https://arxiv.org/abs/2104.09864>. arXiv:2104.09864.

- Philippe Tillet, H. T. Kung, and David Cox. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL 2019)*, pp. 10–19. ACM, 2019. doi: 10.1145/3315508.3329973.
- Georgios Tzannetos, Parameswaran Kamalaruban, and Adish Singla. Curriculum design for trajectory-constrained agent: Compressing chain-of-thought tokens in llms. *Advances in Neural Information Processing Systems*, 38:19881–19907, 2026.
- Sunith Vallabhaneni, Thomas Berkane, and Maimuna S Majumder. The ai committee: A multi-agent framework for automated validation and remediation of web-sourced data. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, pp. 583–590, 2026.
- Yang Wang and Kai Li. Large language models and operations research: A structured survey. *CoRR*, abs/2509.18180, 2025. doi: 10.48550/arXiv.2509.18180.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, et al. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *Advances in Neural Information Processing Systems*, 37:95266–95290, 2024.
- Yudong Wang, Zixuan Fu, Jie Cai, Peijun Tang, Hongya Lyu, Yewei Fang, Zhi Zheng, Jie Zhou, Guoyang Zeng, Chaojun Xiao, Xu Han, and Zhiyuan Liu. Ultra-FineWeb: Efficient data filtering and verification for high-quality llm training data. *arXiv preprint arXiv:2505.05427*, 2025a.
- Yudong Wang, Zixuan Fu, Hengyu Zhao, Chen Zhao, Chuyue Zhou, Hongya Lyu, Shuaikang Xue, Yi Yi, Yingjiao Wang, Zhi Zheng, Yuzhou Zhang, Jie Zhou, Chaojun Xiao, Xu Han, Zhiyuan Liu, and Maosong Sun. Data science and technology towards agi part i: Tiered data management. *arXiv preprint arXiv:2602.09003*, 2026.
- Zhuohan Wang, Ziwei Zhu, Ziniu Li, Congliang Chen, Yizhou Han, Yufeng Lin, Zhihang Lin, Angyang Gu, Xinglin Hu, Ruoyu Sun, and Tian Ding. Orgeval: Graph-theoretic evaluation of llms in optimization modeling. *CoRR*, abs/2510.27610, 2025b. doi: 10.48550/ARXIV.2510.27610.
- Anjiang Wei, Tianran Sun, Yogesh Seenichamy, Hang Song, Anne Ouyang, Azalia Mirhoseini, Ke Wang, and Alex Aiken. Astra: A multi-agent system for GPU kernel performance optimization. *CoRR*, abs/2509.07506, 2025. URL <https://arxiv.org/abs/2509.07506>.
- Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V Le. Finetuned language models are zero-shot learners. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=gEZrGCozdqR>.
- Zhongzhen Wen, Shudi Shao, Zhong Li, Yu Ge, Tongtong Xu, Yuanyu Lin, and Tian Zhang. Ascend-Craft: Automatic ascend NPU kernel generation via DSL-guided transcompilation, 2026. URL <https://arxiv.org/abs/2601.22760>.
- Jiehao Wu, Zixiao Huang, Wenhao Li, Chuyun Shen, Junjie Sheng, and Xiangfeng Wang. AscendOptimizer: Episodic agent for ascend NPU operator optimization. *CoRR*, abs/2603.23566, 2025a. URL <https://arxiv.org/abs/2603.23566>.
- Yang Wu, Yifan Zhang, Yurong Wu, Yuran Wang, Junkai Zhang, and Jian Cheng. Training llms for optimization modeling via iterative data synthesis and structured validation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 12880–12896, 2025b.
- Ziyang Xiao, Dongxiang Zhang, Yangjun Wu, Lilin Xu, Yuan Wang, Xiongwei Han, Xiaojin Fu, Tao Zhong, Jia Zeng, Mingli Song, et al. Chain-of-experts: When llms meet complex operations research problems. In *International Conference on Learning Representations*, volume 2024, pp. 48519–48537, 2024.

- Ziyang Xiao, Jingrong Xie, Lilin Xu, Shisi Guan, Jingyan Zhu, Xiongwei Han, Xiaojin Fu, WingYin Yu, Han Wu, Wei Shi, Qingcan Kang, Jiahui Duan, Tao Zhong, Mingxuan Yuan, Jia Zeng, Yuan Wang, Gang Chen, and Dongxiang Zhang. A survey of optimization modeling meets LLMs: Progress and future directions. In *Proceedings of the Thirty-Fourth International Joint Conference on Artificial Intelligence*, pp. 10771–10779. International Joint Conferences on Artificial Intelligence Organization, 2025. doi: 10.24963/ijcai.2025/1192.
- Ziyang Xiao, Yuan Jessica Wang, Xiongwei Han, Shisi Guan, Jingyan Zhu, Jingrong Xie, Lilin Xu, Han Wu, Wing Yin Yu, Zehua Liu, et al. Deepor: A deep reasoning foundation model for optimization modeling. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pp. 34052–34060, 2026.
- Xiaomi MiMo Team. MiMo-V2.5-Pro. <https://huggingface.co/collections/XiaomiMiMo/mimo-v25>, 2026.
- Zhenda Xie, Yixuan Wei, Huanqi Cao, Chenggang Zhao, Chengqi Deng, Jiashi Li, Damai Dai, Huazuo Gao, Jiang Chang, Kuai Yu, Liang Zhao, Shangyan Zhou, Zhean Xu, Zhengyan Zhang, Wangding Zeng, Shengding Hu, Yuqing Wang, Jingyang Yuan, Lean Wang, and Wenfeng Liang. mHC: Manifold-constrained hyper-connections, 2025. URL <https://arxiv.org/abs/2512.24880>.
- Zijie Yan, Hongxiao Bai, Xin Yao, Dennis Liu, Tong Liu, Hongbin Liu, Pingtian Li, Evan Wu, Shiqing Fan, Li Tao, et al. Scalable training of mixture-of-experts models with megatron core. *arXiv preprint arXiv:2603.07685*, 2026.
- Fanyi Yang, Jianfeng Liu, Xin Zhang, Haoyu Liu, Xixin Cao, Yuefeng Zhan, Hao Sun, Weiwei Deng, Feng Sun, and Qi Zhang. Main: Mutual alignment is necessary for instruction tuning. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 12768–12780, 2025a.
- Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *International Conference on Learning Representations*, volume 2025, pp. 29687–29707, 2025b.
- Zhicheng Yang, Yiwei Wang, Yinya Huang, Zhijiang Guo, Shi Shi, Xiongwei Han, Liang Feng, Linqi Song, Xiaodan Liang, and Jing Tang. Optibench meets resocratic: Measure and improve llms for optimization modeling. In *International Conference on Learning Representations*, volume 2025, pp. 24726–24759, 2025c.
- Zihao Ye, Lequn Chen, Ruihang Lai, Wuwei Lin, Yineng Zhang, Stephanie Wang, Tianqi Chen, Baris Kasikci, Vinod Grover, Arvind Krishnamurthy, and Luis Ceze. FlashInfer: Efficient and customizable attention engine for LLM inference serving. In *Proceedings of Machine Learning and Systems 7 (MLSys 2025)*, 2025. URL <https://arxiv.org/abs/2501.01005>. Best Paper Award.
- Edward Yeo, Yuxuan Tong, Morry Niu, Graham Neubig, and Xiang Yue. Demystifying long chain-of-thought reasoning in llms. *arXiv preprint arXiv:2502.03373*, 2025.
- Çağatay Yıldız, Nishaanth Kanna Ravichandran, Nitin Sharma, Matthias Bethge, and Beyza Ermiş. Investigating continual pretraining in large language models: Insights and implications. *arXiv preprint arXiv:2402.17400*, 2024.
- Bin Yu, Hang Yuan, Haotian Li, Xueyin Xu, Yuliang Wei, Bailing Wang, Weizhen Qi, and Kai Chen. Long-short chain-of-thought mixture supervised fine-tuning eliciting efficient reasoning in large language models. *arXiv preprint arXiv:2505.03469*, 2025.
- Bowen Zhang and Pengcheng Luo. Or-llm-agent: Automating modeling and solving of operations research optimization problem with reasoning large language model. *arXiv e-prints*, pp. arXiv–2503, 2025.
- Jinyang Zhang, Yue Fang, Hongxin Ding, Weibin Liao, Muyang Ye, Xu Chu, Junfeng Zhao, and Yasha Wang. Adept: Continual pretraining via adaptive expansion and dynamic decoupled tuning. *arXiv preprint arXiv:2510.10071*, 2025a.

- Xinyu Zhang, Changzhi Zhou, Linmei Hu, Luhao Zhang, Xiancai Chen, Haomin Fu, Yang Yang, and Mengdi Zhang. Scoder: Progressive self-distillation for bootstrapping small-scale data synthesizers to empower code llms. In Findings of the Association for Computational Linguistics: EMNLP 2025, pp. 20825–20841, 2025b.
- Yifan Zhang, Yifan Luo, Yang Yuan, and Andrew C. Yao. Autonomous data selection with zero-shot generative classifiers for mathematical texts. In Findings of the Association for Computational Linguistics: ACL 2025, 2025c. URL <https://arxiv.org/abs/2402.07625>.
- Haiyuan Zhao, Jun Xu, Xiao Zhang, Guohao Cai, Zhenhua Dong, and Ji-Rong Wen. Unbiased top-k learning to rank with causal likelihood decomposition. In Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region, pp. 129–138, 2023.
- Wanru Zhao, Yihong Chen, Yuzhi Tang, Wentao Ma, Shengchao Hu, Shell Xu Hu, Alex Jacob, Abhinav Mehrotra, and Nicholas D. Lane. Rethinking data curation in LLM training: Online reweighting offers better generalization than offline methods. In The Fourteenth International Conference on Learning Representations, 2026. URL <https://openreview.net/forum?id=UFwnsmFZ6R>.
- Pengfei Zuo, Huimin Lin, Junbo Deng, Nan Zou, Xingkun Yang, Yingyu Diao, Weifeng Gao, Ke Xu, Zhangyu Chen, Shirui Lu, et al. Serving large language models on huawei cloudmatrix384. arXiv preprint arXiv:2506.12708, 2025.

Appendix

A Author List

Core Contributors Dongfang Li, Xiaodong Luo, Ruoyu Sun, Xuhui Chen, Linyuan Qiu, Jian Meng, Zhengxuan Lu, Yiting Wang, Yucheng Xie, Tao Guo, Tianxiang Fang, Jing Li, Sihang Chen, Shihao Hong, Chang Liu, Weihua Dai, Zirong Zeng, Ziwei Zhu, Zhuohan Wang, Zhengjun Yue, Igor Vasilyev, Min Liu, Weijian Sun, Xin Chen

Contributors Yingmeng Gao, Jinhua Zhou, Taolue Chen, Chenwei Wu, Dong Zhang, Wenlong Jin, Jinmin Xiang, Barkova Maria, Ushakov Anton, Xianfei Jin, Tian Ding, Zhihang Lin, Qian Chen, Linxin Yang, Mingzhe Yang, Bingwei Zhang, Hongzhang Yang, Fangxue Zhang, Shijun Qin, Jie Yu, Cuihua Hu, Tolstykh Vasiliy, Nosov Ivan, Abdullin Amir, Zhichen Zhou, Xin Zhang, Zhixiong Ning, Xutong Zhao, Junjie Huang, Jiajun Liu, Weiyan Kong, Zheng Zhang, Wenhan Luo, Lin Hu, Yangbo Guo, Li Zeng, Shihao Zeng

Project Leaders Baotian Hu, Min Zhang, Haizhou Li, Zhiquan Luo

B Post-training Recipe Details

This appendix summarizes the main recipe-level settings used in the post-training experiments, focusing on configurations that affect optimization, parallel execution, memory usage, and reproducibility.

CPT recipe. Table 16 reports the CPT recipe used for the Ascend-based DeepSeek-V4 continued-pretraining run, including the long-context setup, optimization configuration, parallelism strategy, and memory-saving mechanisms.

SFT recipe. Table 17 reports the main recipe-level configuration used for supervised fine-tuning. In contrast to CPT, which optimizes autoregressive language modeling over domain-adaptive corpora, SFT optimizes instruction-response trajectories and masks the loss on prompt tokens so that gradients are applied only to the target response. The reported items are selected to match the capabilities discussed in Section 3.3, including instruction-following alignment, solver-oriented modeling, and pipeline-optimized distributed execution on Ascend NPUs.

Agentic rollout configuration. The agentic rollout pipeline is evaluated in inference mode and does not update model parameters in this work. Its configuration is therefore separated from the CPT and SFT training recipes. If reported, the rollout configuration should include only parameters that affect inference-time behavior and future reinforcement-learning reproducibility, such as the number of sampled formulation candidates, maximum self-review rounds, maximum code-repair rounds, solver timeout, execution success criterion, and objective-matching tolerance. These settings are rollout parameters rather than optimizer or training hyperparameters.

C Solver-Verified OR-CPT Data Synthesis: Engine Design and Illustrative Cases

Section 3.2 introduced the synthesized component of the operations-research (OR) corpus and its bidirectional construction pipeline. This appendix focuses on the method: how a parameterized optimization generator is converted into a self-contained CPT document, how mathematical invariants are preserved across language-model transformations, and how deterministic checks prevent plausible but incorrect samples from entering the training corpus.

The central design principle is that language-model fluency is not used as a proxy for mathematical correctness. Language models perform two semantic transformations—from a verified optimization instance to a business problem, and from that business problem back to a mathematical model and executable program. Generator contracts, static checks, and independent solver execution surround these transformations and determine whether the resulting document is admissible for CPT.

Table 16: Main recipe-level configuration for the Ascend-based CPT run under the DeepSeek-V4-Flash long-context setting. The table reports settings that affect optimization, parallel execution, memory behavior, and NPU-specific training support. Dataset composition, checkpoint paths, logging options, profiling settings, and environment-specific launch details are omitted.

Configuration item	Setting
<i>Training Setup</i>	
Training stage	Full-parameter continued pre-training
Sequence length	65,536 tokens
Global batch size	128 sequences
MTP	Enabled
<i>Optimization</i>	
Optimizer	AdamW
Peak learning rate	1.0×10^{-6}
Learning-rate schedule	Cosine decay
Warmup rate	20%
Minimum learning-rate factor	0.01
Optimizer epsilon	1.0×10^{-6}
<i>Parallelism</i>	
Tensor parallel degree	1
Pipeline parallel degree	1
Context parallel degree	16
Expert parallel degree	32
FSDP-style sharding	Enabled with reshard-after-forward
<i>Memory and NPU Support</i>	
Activation checkpointing	Full
Optimizer memory strategy	Swap optimizer
Model-specific NPU kernels	Adapted for attention and MoE computation

C.1 Role in the post-training workflow

The OR-CPT Engine is an upstream data-construction system. It is separate from the Ascend 910C training stack: the engine establishes mathematical fidelity, semantic coverage, and provenance, while the training stack tokenizes, packs, samples, and optimizes the model on the resulting corpus. The engine exports plain self-contained documents for autoregressive continued pre-training rather than instruction-response conversations.

This separation also clarifies the relationship between CPT and SFT. CPT documents expose the model to OR terminology, formulation patterns, solver APIs, and verification habits. SFT subsequently teaches the model to elicit that knowledge under task instructions and output contracts. The same OR taxonomy can guide both stages, but the data representations and optimization objectives remain distinct.

C.2 End-to-end engine architecture

The engine follows a bidirectional verification path. It first samples a structured optimization instance from a parameterized generator, solves the source model independently, and filters infeasible, degenerate, numerically unstable, or duplicated seeds. The verified seed is then backtranslated into a self-contained business problem, where a natural-language quality filter checks numeric coverage, unit consistency, business grounding, and solution leakage. Finally, the engine reconstructs the model from the generated problem, applies static and contract-based validation, executes the reconstructed model, compares its status and objective against the independent reference, and renders accepted pairs into CPT documents with lineage metadata.

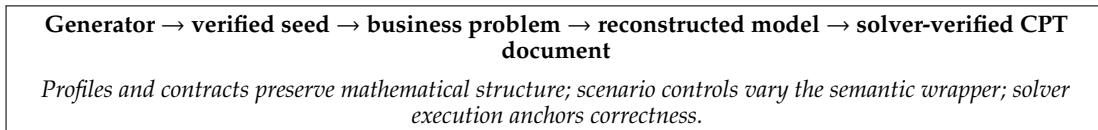
Table 17: Main recipe-level configuration for the SFT run. The table reports settings that affect optimization behavior, reproducibility, and the instruction-tuning experiments discussed in Section 3.3.

Configuration item	Setting
<i>Training Setup</i>	
Training stage	Supervised fine-tuning
Sequence length	8,192 tokens
Global batch size	128 sequences
MTP	Enabled
<i>Optimization</i>	
Optimizer	AdamW
Peak learning rate	5.0×10^{-6}
Minimum learning rate	5.0×10^{-8}
Learning-rate schedule	Cosine decay
Weight decay	1.0×10^{-2}
Gradient clipping	Maximum norm of 1.0
Initial loss scale	65,536
<i>Parallelism</i>	
Tensor parallel degree	1
Pipeline parallel degree	4
Context parallel degree	1
Context parallel algorithm	Ulysses CP
Expert parallel degree	32
Sequence parallel	Enabled
<i>Memory and Execution</i>	
Activation checkpointing	Full
Optimizer memory strategy	Swap optimizer
MTP memory-efficient logits	Enabled

For a seed s , let M_s denote the source model and z_s^* its reference objective. Backtranslation yields a natural-language problem q_s , and forward modeling yields an executable reconstruction $\widehat{M}_s$ with objective $\widehat{z}_s$. The core admission condition can be summarized as

$$\text{Accept}(s) = \mathbb{I}[Q_{\text{seed}}(M_s) \wedge Q_{\text{NL}}(q_s) \wedge Q_{\text{contract}}(\widehat{M}_s) \wedge Q_{\text{exec}}(\widehat{M}_s) \wedge Q_{\text{obj}}(z_s^*, \widehat{z}_s)]. \quad (18)$$

The objective comparison uses absolute and relative tolerances. It is a strong executable consistency signal, but not a proof of symbolic equivalence: two different formulations can share the same optimum on a particular numerical instance. The static and contract-based checks therefore remain necessary even when the objective agrees. Figure 22 summarizes the resulting end-to-end trust path from a generated seed to an accepted, solver-verified CPT document.

**Figure 22:** The trust path of a synthesized OR-CPT document.

C.3 Mathematical seeds and domain contracts

Seed representation. A seed is a structured optimization instance, not merely a textual question. It contains declared sets, coefficient tables, objective direction, variable domains, bounds, constraint families, and executable or LP artifacts. It also carries semantic metadata such as task family, sub-

family, modeling concepts, difficulty axes, and formulation variant. Deterministic random seeds support regeneration when a generator must be audited or refined.

Generator profiles and family contracts. A generator profile describes the canonical mathematical signature of one generator, including variable types, objective terms, core constraints, valid indices, and forbidden reinterpretations. A family contract describes invariants shared across related generators, such as balance equations, linking rules, conservation laws, and required variable families. These layers are grounded in the source facts of the seed—declared sets, coefficient tables, bounds, units, and executable artifacts—so that the language model cannot invent numbers or refer to nonexistent indices. Seed-quality records further capture feasibility, objective recomputation, solution activity, degeneracy, and numerical stability, while lineage metadata records the generator, scenario configuration, prompt revision, validation status, and rendering status outside the training text.

Seed-quality filtering. Feasibility alone does not guarantee training value. A model in which all meaningful decisions are zero, every decision is fixed at a bound, or no constraint participates in a genuine trade-off may be solvable but unhelpful. The engine therefore checks solution activity, constraint activity, objective recomputation, numerical precision, and duplicate signatures before a seed is eligible for language generation.

C.4 Prompt-controlled bidirectional synthesis

Backtranslation and forward modeling have different information boundaries and are controlled by complementary fidelity requirements. Backtranslation may inspect the verified source facts, but it must preserve coefficients, bounds, units, and indexed relations while hiding solver status, objective values, variable solutions, LP artifacts, and code. It may vary the industry setting, stakeholder perspective, planning trigger, horizon, naming style, and units, but it must not add or remove mathematical structure. Forward modeling receives the generated problem and the applicable structural contracts; it must reconstruct the variables, objective sense, core constraints, index policy, explanation, formulation, standalone Gurobi code, and self-check using only the declared data, without hard-coding the reference answer or inferring missing coefficients from the optimum.

The production prompts are versioned implementation artifacts. Their essential logic can be represented without reproducing their full text:

Listing 1: Abstract prompt interface used by the bidirectional synthesis stage.

```
BACKTRANSLATE(verified_seed, profile, scenario):
    preserve source facts and mathematical structure
    hide reference solution and implementation artifacts
    return a complete business problem

FORWARD_MODEL(business_problem, profile, family_contract):
    reconstruct explanation, formulation, and executable code
    use only declared indices and coefficients
    return a structured self-check; do not hard-code the optimum
```

The self-check is treated as a claim made by the generated response, not as the validator. Static inspection and independent execution remain authoritative because prose may mention a required constraint that the code omits or indexes incorrectly.

C.5 Controlled scenarios and CPT rendering

The scenario catalog separates mathematical diversity from linguistic diversity. A base OR structure can be expressed through different industry lenses, decision triggers, organization types, planning frames, entity-naming styles, and unit conventions. These controls change the surface narrative—for example, whether the problem is framed as a healthcare allocation, logistics planning, energy scheduling, manufacturing review, outage response, budget revision, or service-network decision—while keeping the source coefficient tables, variable roles, objective terms, constraints, and mathematical periods invariant.

Negative guidance is equally important. A transportation matrix must not be turned into vehicle routing unless vehicles and sequencing exist. A facility- location task must retain its fixed opening decisions. A lot-sizing task must retain its period-by-period inventory balance. These restrictions prevent a fluent scenario from silently changing the model family.

After executable verification, the renderer converts the accepted pair into a self-contained CPT document. A full document can contain: (i) the business problem and data; (ii) a business-to-model explanation; (iii) a mathematical formulation; (iv) standalone solver code; (v) an interpreted solution; and (vi) feasibility and objective-recalculation checks. Style and section order may vary, but mandatory mathematical and validation content does not.

C.6 Illustrative case I: capacitated facility location

Suppose a regional service operator considers three candidate sites serving three demand zones. Opening a site incurs a fixed cost; service can be allocated only from opened sites and cannot exceed site capacity. The illustrative data are shown in Table 18.

Table 18: Manually constructed facility-location example.

Site	Fixed cost	Capacity	Cost to Z1	Cost to Z2	Cost to Z3
A	120	100	4	7	6
B	95	65	6	3	5
C	80	60	8	5	2
Demand	–	–	40	35	25

Let $y_i \in \{0, 1\}$ indicate whether site i is opened and let $x_{ij} \geq 0$ denote units served from site i to zone j . A faithful model is

$$\min_{x,y} \sum_{i \in I} f_i y_i + \sum_{i \in I} \sum_{j \in J} c_{ij} x_{ij}, \quad (19)$$

$$\text{s.t.} \quad \sum_{i \in I} x_{ij} = d_j, \quad \forall j \in J, \quad (20)$$

$$\sum_{j \in J} x_{ij} \leq u_i y_i, \quad \forall i \in I, \quad (21)$$

$$y_i \in \{0, 1\}, \quad x_{ij} \geq 0. \quad (22)$$

The optimal illustrative plan opens B and C. Site B serves 30 units of Z1 and all 35 units of Z2; site C serves the remaining 10 units of Z1 and all 25 units of Z3. Site B is exactly at its capacity of 65. The objective is

$$95 + 80 + 6(30) + 8(10) + 3(35) + 2(25) = 590. \quad (23)$$

This example contains a genuine fixed-charge trade-off. Site C has the lowest fixed cost and the cheapest service to Z3, while B is cheapest for Z2. Neither can serve total demand alone under the stated capacities. The model must jointly choose sites and allocate service rather than select the cheapest coefficient in each demand column.

The same mathematics could be rendered as clinic siting, regional repair depots, or computing-service hubs. The entities and units may change, but the three invariants—fixed activation, demand satisfaction, and capacity linking—must remain present. Forward reconstruction is accepted only if it recovers these invariants and reproduces the reference objective within tolerance.

C.7 Illustrative case II: executable but structurally wrong models

This example also shows why execution success is insufficient. Table 19 contrasts the faithful formulation with two executable but structurally incorrect reconstructions and identifies the validation gates that detect each error.

Table 19: Illustrative structural errors and the gates that detect them.

Reconstruction	Structural defect	Illustrative optimum	Detection
Faithful model	Fixed costs and capacity linking retained	590	Accepted after contract and objective checks.
Fixed costs omitted	Sites can be opened without paying activation cost	315	Contract reports a missing objective term; objective comparison fails.
Capacity constraints omitted	Site B serves all demand despite capacity 65	565	Contract reports a missing constraint; objective comparison fails.

All three programs can be syntactically valid, executable, and solved to an OPTIMAL status. That status certifies only the optimum of the program that was actually constructed. It does not certify faithfulness to the intended business problem. The example therefore motivates three distinct checks: static presence of required terms, family-level structural contracts, and independent objective comparison.

Repair follows the same rule. When a problem statement lacks a coefficient or bound, the safe action is regeneration, rejection, or recovery from an authoritative source table. A repair must not invent a value merely to make the model bounded or to approach the reference objective.

D CPT-SFT-Deployment-Evaluation Provenance

Post-training large-scale MoE models involves multiple stages—CPT, SFT, weight conversion, serving, and evaluation—each producing artifacts dependent on upstream outputs. We introduce a lightweight provenance system that links all stages via per-stage manifests and a local file-based registry, to avoid opacity between benchmark scores and training configurations, checkpoints, or deployed weights, thereby ensuring clear attribution of improvements, reproducibility, and auditability of evaluation claims. The system serves three functions: (i) it enforces a verifiable chain from training data to scores, tracing each result to its exact checkpoint; (ii) it enables root-cause analysis of regressions by tracing deployed models back through conversion and training to the upstream CPT run; (iii) it provides programmatic support for automated tracking, dashboard monitoring, and artifact management, without database dependencies or disruption to existing workflows.

D.1 Design principles

1. **Model identity is decoupled from the service endpoint.** A served model name, IP address, and port identify the inference entry point, not the model version. The authoritative model identity is carried by the artifact manifest at the weight directory, together with the upstream training lineage recorded at each stage. This separation allows multiple model versions to be served under the same endpoint name while preserving unambiguous identification of which weights are in use.
2. **Every stage records its ground truth at the point of execution.** Each stage writes the actual paths, key environment variables, and upstream relationships into a structured manifest immediately upon completion. This manifest, rather than any later reconstruction, serves as the authoritative record for that stage. The point-of-execution recording eliminates the fragility of post-hoc documentation and ensures that environmental details (e.g., CANN version, parallelism configuration, tokenizer path) are captured before they can be altered or forgotten.

3. **Historical experiments can be incorporated post-hoc.** Registration tooling accepts existing configuration snapshots, checkpoint directories, and converted artifacts, allowing runs completed before the provenance system was deployed to be retroactively linked into the provenance graph. This ensures that valuable legacy experiments are not excluded from cross-run comparison and trend analysis.

D.2 Provenance chain and registry layout

The end-to-end chain spans five stages: CPT training, SFT training, HF artifact conversion, model deployment, and benchmark evaluation. Each stage produces a manifest file at its output location and registers a copy in a centralized JSON registry. The registry is organized by stage type (training runs, artifacts, deployments, evaluation runs), with a lightweight index file maintaining aggregate counts. In addition to the centralized entries, manifests are always co-located with the artifacts they describe—the HF artifact manifest resides in the weight directory, the deployment manifest resides in the serving log directory, and the evaluation provenance resides in the benchmark summary directory. This dual placement ensures that the provenance record travels with the artifact even if the centralized registry is unavailable, and that any downstream consumer of the artifact (e.g., a separate evaluation pipeline) can discover its lineage without consulting the central registry.

D.3 Stage-level linkage mechanism

CPT → SFT. Both CPT and SFT runs produce a configuration snapshot and lineage manifest through a shared registration interface. The parent-child relationship is established by matching the SFT checkpoint load path against the CPT checkpoint save path, recording `linked` status with `path_match` confidence on success, or `missing` otherwise. This link is essential for CPT-to-SFT transfer analysis: without it, a reported transfer gain cannot be traced to the specific CPT checkpoint that provided the initialization.

SFT → HF artifact. Upon weight format conversion to HuggingFace-compatible shards, the converter writes an artifact manifest recording the artifact identifier, source training run, checkpoint directory and iteration, HF output directory, and upstream CPT lineage, with a copy registered centrally. This manifest is the authoritative record connecting deployable weights to their training provenance.

HF artifact → model deployment. The serving launch procedure reads the artifact manifest from the weight directory and generates a deployment manifest capturing the deployment identifier, artifact and training run references, model path, served model name, and inference endpoint URL, with copies in both the log directory and the central registry. This artifact-to-endpoint binding enables the dashboard to track which model version is served at each endpoint.

Model deployment → eval run. Evaluation accepts an optional deployment manifest reference via an environment variable. When provided, the harness extends the benchmark summary with full provenance fields spanning evaluation, deployment, artifact, SFT, and CPT. Without it, evaluation proceeds normally but scores cannot be automatically traced to specific weights or associated with the corresponding dashboard experiment.

D.4 Confidence levels

Each provenance link carries a confidence annotation, summarized in Table 20, that distinguishes manifest-based exact linkage from path-based or heuristic matching.

D.5 Dashboard monitoring and retroactive registration

The provenance registry directly feeds a monitoring dashboard that visualizes the full experiment graph, each row corresponds to a training run, and columns display the run identifier, job type, training iterations, key metrics (loss, benchmark scores), and—critically—provenance status and confidence annotations for each link in the chain. Runs with complete manifest coverage are marked with `exact_manifest` or `path_match` confidence; legacy runs that predate the provenance system are displayed with a `missing` annotation but are otherwise fully functional in the dashboard.

Table 20: Provenance confidence levels assigned to each link in the chain.

Level	Condition
<code>exact_manifest</code>	The downstream stage directly reads an explicit manifest produced by the upstream stage (e.g., the deployment script reads the artifact manifest from the HF weight directory).
<code>path_match</code>	The link is established by matching path fields across independently produced configuration records (e.g., the SFT checkpoint load path matches the CPT checkpoint save path).
<code>inferred_from_log</code>	Reserved for future use, where training or deployment logs may provide auxiliary evidence for a link.
<code>missing</code>	Insufficient information exists to establish the link; the downstream stage operates without a confirmed upstream connection.

The dashboard is refreshed by a build script that scans the training archive directory and the centralized registry, aggregating manifest contents into a single JSON consumed by the frontend. Newly completed runs appear automatically on the next rebuild; legacy runs without manifests can be registered retroactively by pointing the registration tooling at their existing configuration files, checkpoint directories, or artifact outputs. This design ensures that the dashboard reflects both actively tracked experiments and backfilled historical data within a unified view, making it the operational interface through which experiment provenance is queried, audited, and cross-referenced against benchmark claims.

E Training Schedule and Monitoring Design

While the data refinement pipeline improves individual sample quality, the macro-level organization of training data also requires careful design. Our data-mixture analysis shows that overly concentrated OR-domain adaptation can weaken general capabilities, motivating a training schedule that balances OR, mathematics, code, and general-domain data. We therefore design a predefined curriculum schedule together with a monitoring dashboard for tracking training stability and capability retention. The schedule, stage boundaries, and transition rules are specified before training, while the dashboard serves as an observability tool for manual inspection.

E.1 Static Curriculum Learning Design

We organize the training data mixture into a pre-defined, three-stage schedule. The schedule uses a mixing vector $\alpha(t) = [\alpha_{\text{OR}}, \alpha_{\text{math}}, \alpha_{\text{code}}, \alpha_{\text{gen}}]$ that is varied across training steps t according to a fixed plan:

- Foundation stage (first 30% of steps):** Heavy emphasis on mathematical reasoning and clean general code ($\alpha_{\text{math}} = 0.35, \alpha_{\text{code}} = 0.35, \alpha_{\text{gen}} = 0.20, \alpha_{\text{OR}} = 0.10$). This phase reinforces precise token generation and algorithmic logic.
- Domain ramp-up (30–70%):** α_{OR} is linearly increased to 0.40 while α_{code} gradually reduces to 0.15. Within this stage, OR samples are introduced by difficulty format: first Data-in-Problem (DP), then Data-in-Table (DT), and finally Data-Problem-Separate (DPS). The transition to a more difficult format is gated by a fixed evaluation threshold on a held-out subset of the current format: the model must achieve more than 85% execution success before the next format is unlocked. This threshold is a hard rule, not an adaptive algorithm.
- Contract hardening (70–100%):** The mixture shifts back toward code-only and LP-writing tasks (α_{code} raised to 0.25) to reinforce strict output contracts, using validated samples from the data refinement pipeline.

This schedule is a **default trajectory** whose parameters (proportions, thresholds, stage boundaries) were fixed based on preliminary runs and the data-mixture ablation in Section 4.4 (Figure 21). It is not modified dynamically during training. Curriculum-like scheduling strategies have been explored in other contexts (Tzannetos et al., 2026; Zhao et al., 2026; Liang et al., 2025); our specific instantiation is one manually designed configuration.

E.2 Monitoring Dashboard

To observe training behavior and detect potential problems early, we implemented a monitoring setup that logs two groups of metrics at regular intervals (every 500 training steps). This dashboard is not a closed-loop controller; it is an observability layer for human developers.

Training-health track. Standard optimization signals: loss, gradient norm, NaN counts, and Model FLOPs Utilization (MFU). A spike in gradient norm or a NaN loss immediately signals a data or training issue requiring manual investigation.

Capability-retention track. A set of metrics that track both domain and general performance:

- **OR capability:** validation perplexity on an OR-domain set (partitioned by format), and task-specific rewards (`code_reward`, `wl_reward`) on small proxy subsets of NL4OPT and OptiBench.
- **General code quality:** a *syntax health score* computed on a held-out set of 200 pure-Python algorithmic problems. The score checks for non-ASCII tokens, indentation errors, import correctness, and basic static validity.
- **General language fluency:** perplexity on a fixed general-domain corpus (Wikipedia, books) to detect catastrophic forgetting.

These metrics are logged and visualized in WandB dashboards. During development, they were used for manual inspection: for example, a persistent decline in syntax health could prompt the developer to pause OR-centric training, inspect recent data, or adjust the mixture. The specific thresholds and the resulting decisions were made by human engineers, not by an automated state machine. The dashboard design is inspired by systematic analyses of training dynamics (Qi et al., 2026), and its instrumentation code is included in the released artifacts.

F Representative Original-Checkpoint Error Cases

Section 3.1 summarizes the aggregate failure distribution of the original DeepSeek-V4-Flash checkpoint. This appendix records the concrete examples used in that diagnosis. The cases are traced back to the corresponding evaluation artifacts and are grouped by the six failure types discussed in the Inference-Setting Shift analysis.

Case Study: Canonical LP-Graph Mismatch under ORGEval

Benchmark	B4O ORGEval
Sample ID	BENCH4OPT_0; related: BENCH4OPT_1, BENCH4OPT_12
Evaluation	code_reward=1.0, wl_reward=0.0; var_num_check=True, cons_num_check=False
Failure Type	Executable LP with non-reference constraint family

Original Problem

The problem is a pharmaceutical blending task. The model must choose ingredient quantities to minimize cost while respecting ingredient availability, medication dosage requirements, and active-ingredient purity bounds. ORGEval does not only check whether the generated LP can be written; it compares the variable family, constraint family, and their graph structure against the reference LP.

Model Output

```
x = model.addVars(I, lb=0.0, ub=GRB.INFINITY, name='x')
model.setObjective(gp.quicksum(costs[i] * x[i] for i in range(I)), GRB.MINIMIZE)

for i in range(I):
    model.addConstr(x[i] <= availability[i], name=f'availability_{i}')

# The model assumes dosage bounds apply to the total medication amount.
for m in range(M):
    total_med = gp.quicksum(requirements[m][i] * x[i] for i in range(I))
    model.addConstr(total_med >= min_dosage[m], name=f'min_dosage_{m}')
    model.addConstr(total_med <= max_dosage[m], name=f'max_dosage_{m}')

total_ingredients = gp.quicksum(x[i] for i in range(I))
for i in range(I):
    model.addConstr(x[i] >= min_purity[i] * total_ingredients,
                    name=f'min_purity_{i}')
    model.addConstr(x[i] <= max_purity[i] * total_ingredients,
                    name=f'max_purity_{i}')
```

Error Annotation. The program is executable and writes an LP file, but the generated constraint family does not match the reference formulation. In this family of cases, BENCH4OPT_1 first fails variable-count matching, and BENCH4OPT_12 shows the next level of difficulty: even when variable and constraint counts match, the WL graph can still be wrong. These examples show why structural ORGEval failures motivate CPT for domain templates and SFT for benchmark-facing canonical formulations.

Case Study: Output-Contract and Schema Failures

Benchmark	B4O ORGEval
Sample ID	BENCH4OPT_4; BENCH4OPT_176
Evaluation	BENCH4OPT_4: syntax failure from markdown fence; BENCH4OPT_176: TypeError from list indexing with a string shift label
Failure Type	Code-only protocol violation and JSON/list schema mismatch

Original Problems

BENCH4OPT_4 is a capital-budgeting problem: select investment projects to maximize NPV under budget, risk, and dependency constraints. BENCH4OPT_176 is a hospital nurse-scheduling problem: assign nurses to shifts while satisfying shift coverage, skill requirements, and maximum working hours.

Model Output

```
# BENCH4OPT_4: code-only target, but the generated response starts with:
"python
import json
import gurobipy as gp
from gurobipy import GRB

# BENCH4OPT_176: shifts are string labels, while shift_requirements is a list.
for j in shifts:
    model.addConstr(
        gp.quicksum(x[i, j] for i in nurses) >=
        shift_requirements[j],
        name=f"shift_coverage_{j}"
    )
```

Error Annotation. The first error blocks evaluation before the model is even built: the executor sees the markdown fence token as Python syntax. The second error reaches model construction but fails when a string shift name is used as a list index. Both examples primarily motivate SFT: the model already has useful OR knowledge, but Pass@1 behavior must obey code-only contracts and schema-safe data-access templates.

Case Study: Discrete Semantics Error in NL4OPT

Benchmark	NL4OPT
Sample ID	NL4OPT_15
Evaluation	Ground truth: 28250; Prediction: 28125
Failure Type	Continuous relaxation of integer workforce variables

Original Problem

A store employs senior citizens who earn \$500 per week and young adults who earn \$750 per week. The store must keep the weekly wage bill below \$30000. On any day, the store requires at least 50 workers, of whom at least 10 must be young adults. To ensure the store runs smoothly, the number of young adults should be at least a third the number of senior citizens. Formulate a LP to minimize the wage bill.

Model Output

```
import gurobipy as gp
from gurobipy import GRB

m = gp.Model("MinimizeWageBill")

x = m.addVar(vtype=GRB.CONTINUOUS, name="seniors")
y = m.addVar(vtype=GRB.CONTINUOUS, name="young_adults")

m.setObjective(500 * x + 750 * y, sense=GRB.MINIMIZE)

m.addConstr(x + y >= 50, "total_workers")
m.addConstr(y >= 10, "min_young")
m.addConstr(3 * y >= x, "young_senior_ratio")
m.addConstr(500 * x + 750 * y <= 30000, "wage_budget")

m.optimize()

if m.status == GRB.OPTIMAL:
    print(f"Optimal solution: x = {x.X:.2f}, y = {y.X:.2f}")
    print(f"Minimum wage bill = ${m.ObjVal:.2f}")
else:
    print("No optimal solution found.")
```

Error Annotation. The problem asks for numbers of workers, but the model uses continuous variables. This relaxes the integer workforce decision and yields a fractional optimum.

Case Study: Loss/Yield-Driven Conservation Error

Benchmark	B4O Feasible
Sample ID	BENCH40PT_2
Evaluation	Pass@1 prediction: 265.5768398591736; ground truth: 531.1536797183472. 5-shot and Pass@16 recover the reference value.
Failure Type	Evaporation loss mixed into both cost and flow conservation

Original Problem

The task is a water-distribution network-flow problem. Each edge has a capacity, transportation cost, and evaporation rate. The model must satisfy node demand while minimizing the total cost induced by flow and evaporation.

Model Output

```

flow = {}
for i, (source, target, capacity, cost, evaporation_rate) in enumerate(edges):
    flow[(source, target)] = model.addVar(lb=0.0, ub=capacity,
                                           name=f"flow_{source}_{target}")

obj = gp.LinExpr()
for (source, target), var in flow.items():
    ...
    obj += (cost + evaporation_rate) * var
model.setObjective(obj, GRB.MINIMIZE)

for node_idx, node in enumerate(nodes):
    inflow = gp.LinExpr()
    outflow = gp.LinExpr()
    evaporation_loss = gp.LinExpr()
    ...
    model.addConstr(
        inflow - outflow - evaporation_loss == demand[node_idx],
        name=f"flow_conservation_{node}"
    )

```

Error Annotation. The code is syntactically valid and reaches an optimal solution, but it implements the wrong physical system. Evaporation is treated as an additive unit cost and also deducted from conservation, blurring whether loss affects delivered quantity, incurred cost, or both. This is a domain-pattern failure: network flow with loss, yield, shrinkage, and evaporation needs stable conservation templates rather than generic LP scaffolding.

Case Study: Ratio, Unit, and Total-Quantity Coupling Error

Benchmark	OptiBench
Sample ID	OPTIBENCH_6
Evaluation	Pass@1 prediction: 10000.0; 5-shot prediction: 28000.0; Pass@16 prediction: 10000.0; ground truth: 18000.0
Failure Type	Investment-return units and total savings are coupled incorrectly

Original Problem

A logistics company allocates 50 trucks among three goods. Each good has an initial per-truck delivery cost. Investment in route-optimization software can reduce per-truck cost, and total investment must not exceed the total savings generated by the investment. The objective is to minimize total delivery cost while satisfying minimum truck counts for Goods X and Y.

Model Output

```
x = m.addVars(goods, vtype=GRB.INTEGER, name="trucks")
I = m.addVars(goods, vtype=GRB.CONTINUOUS, name="investment")

# cost per truck = c_i - (r_i/100)*I_i
obj = gp.QuadExpr()
for i in goods:
    obj += (c[i] - (r[i]/100.0) * I[i]) * x[i]
m.setObjective(obj, GRB.MINIMIZE)

m.addConstr(gp.quicksum(I[i] for i in goods) <= B, name="budget")

# Savings constraint: total investment <= total cost savings
savings_lhs = gp.quicksum(I[i] for i in goods)
savings_rhs = gp.quicksum(
    (r[i]/100.0) * I[i] * x[i] for i in goods
)
m.addConstr(savings_lhs <= savings_rhs, name="savings")
m.params.NonConvex = 2
```

Error Annotation. The model recognizes that investment and truck allocation interact, but it does not correctly reconcile dollars, hundreds-of-dollars, per-truck reduction, and total savings. In 5-shot, the error changes shape: the model moves toward a linear-looking savings template but can drop the scale variable. This case therefore exposes a deeper semantic issue than API usage: ratio, ROI, average-cost, and efficiency statements require explicit tracking of numerator, denominator, unit, and total quantity.

Case Study: Solver-Compatible Nonlinear Reformulation Failure

Benchmark	OptiBench
Sample ID	OPTIBENCH_13
Evaluation	Pass@1: optimization failed with “Objective must be linear or quadratic”; 5-shot prediction: 0.125; Pass@16 prediction: 0.125; ground truth: 0.35355235026576626
Failure Type	Fourth-order objective passed directly to Gurobi

Original Problem

A person is trapped near a swamp boundary described by $y \geq x^2$ with $x \geq 0$. The task is to find the closest feasible rescue point to $(3/4, 0)$, equivalently the shortest rope length to a point on or above the parabolic boundary.

Model Output

```
m = gp.Model("SwampRescue")
x = m.addVar(lb=0.0, vtype=GRB.CONTINUOUS, name="x")

# Objective: minimize (x - 0.75)^2 + (x^2)^2
obj = (x - 0.75)*(x - 0.75) + x*x*x*x
m.setObjective(obj, GRB.MINIMIZE)

m.optimize()
```

Error Annotation. The mathematical expression is meaningful, but it is not a linear or quadratic objective. Later inference settings avoid the initial build error yet still return the wrong distance, so the missing capability is not merely a Gurobi API habit. The model needs solver-aware reformulation knowledge: when to introduce auxiliary variables, when nonconvex quadratic or general constraints are legitimate, and when analytic simplification is required before coding the optimization model.